



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт	ИВТИ
Кафедра	ВМСС

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(магистерская диссертация)

Направление 09.04.01 Информатика и вычислительная техника
(код и наименование)

Направленность (программа) Вычислительные машины, комплексы,
системы и сети

Форма обучения очная
(очная/очно-заочная/заочная)

Тема: Исследование способов компенсации нелинейных искажений
цифровых аудиозаписей

Студент	<u>А-07М-23</u>	<u>Винокуров Р.Н.</u>
	группа	подпись фамилия и инициалы

Научный руководитель	<u>к.т.н.</u>	<u>доцент</u>	<u>Гольцов А.Г.</u>
	уч. степень	должность	подпись фамилия и инициалы

«Работа допущена к защите»

Зав. кафедрой	<u>к.т.н.</u>	<u>доцент</u>	<u>Вишняков С.В.</u>
	уч. степень	звание	подпись фамилия и инициалы

Дата _____

Москва, 2025

АННОТАЦИЯ

В представленной работе рассматривается проблема компенсации нелинейных искажений цифрового звука на примере жесткого ограничения максимально допустимой амплитуды. Предложены два подхода к решению данной проблемы: компенсация нелинейных искажений с помощью сплайновой интерполяции и использование глубоких нейронных сетей. Представлены оригинальные реализации обоих подходов, произведена количественная и качественная оценка полученных результатов, проведен анализ более ранних исследований в предметной области.

Работа содержит 76 страниц текста, 12 рисунков, 2 приложения, 4 таблицы, и 26 использованных источников.

ABSTRACT

This work addresses the problem of compensating for nonlinear distortions in digital audio, using hard clipping of the maximum allowable amplitude as an example. Two approaches are proposed: compensation using spline interpolation and using deep neural networks. Original implementations of both approaches are presented, along with quantitative and qualitative evaluations of the results obtained, and an analysis of previous studies on the subject.

The work contains 76 pages of text, 12 figures, 2 appendices, 4 tables, and 26 references.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ: ПОСТАНОВКА ЗАДАЧИ И ТИПОВЫЕ ПОДХОДЫ К ЕЕ РЕШЕНИЮ	6
1.1. Базовые понятия предметной области	6
1.2. Постановка задачи	12
1.3. Существующие решения проблемы клиппинга.....	14
1.4. Выводы.....	17
2. СПЛАЙНОВАЯ ИНТЕРПОЛЯЦИЯ КАК ПОДХОД К ПОДАВЛЕНИЮ НЕЛИНЕЙНЫХ ИСКАЖЕНИЙ ЦИФРОВОГО ЗВУКА	18
2.1. Введение в сплайновую интерполяцию цифрового звука.....	18
2.2. Обнаружение клиппинга	18
2.3. Интерполяция кубическими сплайнами	19
2.4. Особенности имплементации алгоритма восстановления	23
2.5. Выводы.....	25
3. ПОДАВЛЕНИЕ НЕЛИНЕЙНЫХ ИСКАЖЕНИЙ ЦИФРОВОГО ЗВУКА С ПОМОЩЬЮ НЕЙРОСЕТЕЙ.....	26
3.1. Подготовка обучающей выборки нейросети.....	26
3.2. Оконное дискретное преобразование Фурье и его инверсия	30
3.3. Архитектура и обучение нейросети	39
3.4. Восстановление звуковых дорожек с помощью разработанной нейросети	51
3.5. Выводы.....	56
4. АНАЛИЗ ЭФФЕКТИВНОСТИ РАЗРАБОТАННЫХ МЕТОДОВ НА РЕАЛЬНЫХ ДАННЫХ	58
4.1. Методы оценки эффективности восстановления цифрового звука.....	58
4.2. Статистический анализ разработанных методов восстановления звука	66
4.3. Выводы.....	71
ЗАКЛЮЧЕНИЕ	73
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	74
ПРИЛОЖЕНИЕ А. Задание на магистерскую диссертацию.....	77
ПРИЛОЖЕНИЕ Б. Листинг кода разработанных программ и скриптов	82

ВВЕДЕНИЕ

В современном мире качественная передача и воспроизведение звука играет ключевую роль во множестве приложений — от студийной звукозаписи и телекоммуникаций до систем публичного оповещения и автомобильных аудиосистем. Однако реальные акустические среды и электронные трактовые устройства неизбежно вносят самые различные искажения: от нелинейных гармонических артефактов и фазовых смещений до резонансных провалов и фоновых шумов. Искажения звукового сигнала не только ухудшают разборчивость речи и восприятие музыкальных произведений, но и могут приводить к нежелательным акустическим эффектам.

В связи с этим важной задачей современной акустики и цифровой обработки сигналов становится разработка методик компенсации звуковых искажений. В представленной работе будет рассмотрена задача компенсации одного из видов нелинейных искажений цифрового звука с использованием сплайновой интерполяции и глубокого машинного обучения. Разработанные решения будут ориентированы на восстановление звуковых записей человеческой речи. Эффективность представленных решений будет оценена на обширной выборке звуковых фрагментов человеческой речи.

Основная часть диссертации состоит из четырех разделов.

В *первом разделе* рассматривается предметная область исследования, вводятся необходимые для постановки задачи определения и понятия, формулируется решаемая в диссертации проблема и обсуждаются более ранние исследования решаемой проблемы и их результаты.

Во *втором разделе* рассматриваются теоретические и практические аспекты интерполяции кубическими сплайнами как решения поставленной проблемы. Особое внимание уделяется практическим аспектам работы программы, связанным с хранением и обработкой звуковых данных.

В *третьем разделе* рассматривается предложенный подход к решению поставленной проблемы с использованием глубокого обучения. Обосновываются принятые при проектировании разработанной модели архитектурные решения и используемые подходы к подготовке обучающей выборки нейросети. В отдельном подразделе рассматривается реализация алгоритма восстановления звука с помощью разработанной модели.

В *четвертом разделе* производится анализ эффективности разработанных программных имплементаций восстановления звука при помощи набора объективных показателей качества восстановления звука. В ходе анализа рассматриваются оба представленных в работе подхода, проводится их сравнение, и обосновываются полученные результаты.

Задание на магистерскую диссертацию приведено в приложении А.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ: ПОСТАНОВКА ЗАДАЧИ И ТИПОВЫЕ ПОДХОДЫ К ЕЕ РЕШЕНИЮ

1.1. Базовые понятия предметной области

Перед тем как говорить о решаемой задаче, скажем несколько слов о предметной области представленной работы. Областью исследования данной работы в широком смысле является цифровая обработка сигналов. В более узком смысле ее можно сформулировать как цифровую обработку звука.

Прежде чем перейти к постановке задачи нам необходимо сформулировать некоторые фундаментальные понятия, которые потребуются для ее более точной постановки. Начать стоит с понятия о том, в каком виде хранится звуковая информация в памяти ЭВМ. Поскольку нам необходимо базовое понимание данного вопроса, мы не будем рассматривать сложные звуковые форматы со сжатием, наподобие FLAC или MP3. Упомянем лишь, что сжатие звуковой информации осуществляется похожими или зачастую в точности теми же самыми методами, которыми осуществляется сжатие изображений и точно так же по аналогии со сжатием изображений звуковую информацию можно сжимать с потерями и без потерь.

Из соображений простоты рассмотрим один из простейших форматов хранения звуковой информации – WAV (от англ. waveform – «в форме волны»). Данный формат позволяет использовать различные алгоритмы кодирования (то же самое сжатие вышеупомянутого формата MP3 можно встроить внутрь WAV-файла), но, как правило, он используется для хранения несжатого звука в импульсно-кодовой модуляции (англ. Pulse Code Modulation, PCM).

Поскольку PCM является самым простым видом модуляции, рассмотрим хранение оцифрованного звука на его примере. Это вид модуляции, используемый для оцифровки аналоговых сигналов и использующий квантование и дискретизацию [1, с.92]. Поскольку данные понятия являются базовыми понятиями цифровой обработки сигналов, приведем краткое объяснение их значения. Квантование – округление всех

значений сигнала до одного из значений, находящихся в конечном множестве возможных значений. Дискретизация – это процесс взятия отсчетов амплитуды сигнала в некоторые фиксированные моменты времени. Процесс импульсно-кодовой модуляции проще всего представить как последовательное применение к некоторой непрерывной функции сначала дискретизации, а потом квантование полученного дискретизированного сигнала. Результаты, получаемые при последовательной дискретизации и квантовании аналогового сигнала представлены на Рис. 1.

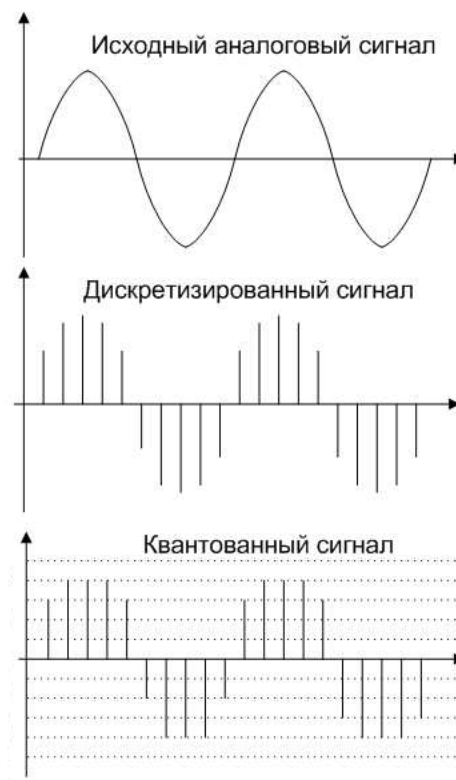


Рис. 1 Оцифровка аналогового сигнала [2]

Сверху на данном рисунке представлен исходный аналоговый сигнал, который математически выражается как некоторая непрерывная функция от времени, в середине – дискретизированный сигнал, выражаемый конечным набором значений данной функции в фиксированные моменты времени (отсчетов), и внизу – дискретизированный и квантованный сигнал, выражается набором округленных до ближайшего «допустимого» значения отсчетов исходной функции в фиксированные моменты времени.

Существует два варианта РСМ: линейная и нелинейная. Первый вид дискретизации подразумевает однородное квантование и кодирование, второй – неоднородное квантование и кодирование. Рассмотрим однородное квантование. Однородное квантование представляет собой такую схему квантования, где разница между двумя соседними по величине «допустимыми» амплитудами сигнала всегда постоянна. То есть:

$$\Delta A = A_{i+1} - A_i = \text{const}, i = 1 \dots n - 1 \quad (1.1)$$

Здесь:

A_i – i -ая допустимая амплитуда сигнала (считаем, что допустимые значения амплитуды сигнала упорядочены по возрастанию)

n – число допустимых амплитуд квантованного сигнала

Соответственно, неоднородное квантование — это квантование, при котором ΔA в формуле 1.1 не равна константе.

Дискретизация в обоих случаях осуществляется с фиксированным значением временного интервала между двумя соседними отсчетами исходного сигнала. Обозначим длительность этого интервала как T_s . Тогда обратная величина $f_s = \frac{1}{T_s}$ выражает количество отсчетов дискретизированного сигнала, приходящихся на одну секунду времени. Эту величину обычно называют частотой дискретизации сигнала.

Как было сказано ранее, линейная и нелинейная РСМ подразумевают однородное и неоднородное квантование. Несмотря на простоту схемы с однородным квантованием зачастую возникает необходимость использовать схему с неоднородным квантованием. Данная необходимость обусловлена тем, что в некоторых сигналах встречающиеся значения амплитуд могут быть сосредоточены в достаточно узком диапазоне. К примеру, в человеческой речи встречаемость малых амплитуд сигнала более вероятна, чем больших [1, с.94]. Таким образом, для экономии количества занимаемой такими сигналами памяти можно использовать схему с неоднородным квантованием.

Квантование всегда подразумевает искажение исходного сигнала и появление в нем некоторого шума (как правило, погрешность квантования не превышает значения половины МЗБ используемого АЦП[1, с.92]), но это не всегда подразумевает необходимость достижения как можно большего значения количества уровней квантования, поскольку такое повышение может лишь привести к повышению точности описания уже содержащегося в исходном сигнале шума[1, с.92-93]. В то же время операцию дискретизации можно осуществить таким образом, чтобы передать все необходимые гармоники исходного аналогового сигнала. Это следует из формулировки теоремы Котельникова (в западной литературе – теорема Найквиста-Шеннона): непрерывный сигнал с наибольшей частотой гармоники f может быть однозначно восстановлен по дискретному набору своих отсчетов, следующих с частотой $f_s = 2f$ [3, с.26].

Таким образом, чтобы передать любой воспринимаемый человеком звук (как мы знаем, человеческое ухо воспринимает звуковые колебания с частотой до 20 кГц), достаточно частоты дискретизации около 40 кГц. Такая частота дискретизации действительно является используемым в РСМ стандартом (точнее, 44 100 Гц). Как мы помним, в дополнение к частоте дискретизации РСМ также характеризуется количеством уровней квантования аналогового сигнала. Как правило, его выражают разрядностью используемого для оцифровки АЦП. Для РСМ в WAV-файле это в большинстве случаев либо 8, либо 16, либо 32 бита (что соответствует либо $2^8 = 256$, либо $2^{16} = 65\,536$, либо $2^{32} = 4\,294\,967\,296$ уровням квантования) [4]. Чаще всего значения семплов звукового файла хранятся со знаком (за исключением 8-битного аудио) [4], т.е. если количество уровней квантования в WAV-файле равно N , то значения семплов в нем это целые числа от $-\frac{N}{2}$ до $\frac{N}{2} - 1$ включительно. Важно учитывать диапазон значений семплов в WAV-файле при нормировке значений его семплов. Отметим, что как поля заголовка WAV-файла (за исключением полей, интерпретируемых как последовательности ASCII-

символов), так и сами данные WAV-файла используют порядок следования байтов от младшего к старшему.

Следующая важная концепция цифровой обработки сигналов, которой мы будем пользоваться далее – дискретное преобразование Фурье. Источником дискретного преобразования Фурье (ДПФ) является непрерывное преобразование Фурье, определяемое следующим образом [5, с.63]:

$$X(f) = \int_{-\infty}^{\infty} x(t) \cdot e^{-j2\pi ft} dt \quad (1.2)$$

Здесь:

j – мнимая единица ($j^2 = -1$)

e – число Эйлера

$x(t)$ – некоторый непрерывный сигнал во временной области

$X(f)$ – спектр сигнала $x(t)$

Отметим, что при дальнейшем рассмотрении преобразования Фурье и сопутствующих понятий мы будем придерживаться той же системы обозначений для мнимой единицы и числа Эйлера. В области обработки непрерывных сигналов формула 1.2 используется для преобразования аналитического выражения для непрерывной временной функции $x(t)$ в непрерывную функцию $X(f)$ в частотной области [5, с.63].

Поскольку обрабатываемый нами сигнал – дискретная последовательность отсчетов звукового сигнала, больший интерес для нас представляет дискретная версия преобразования Фурье. ДПФ определяется согласно следующей формуле [5, с.64]:

$$X(m) = \sum_{n=0}^{N-1} x(n) \cdot e^{\frac{-j2\pi nm}{N}} \quad (1.3)$$

Здесь:

$x(n)$ – дискретный сигнал во временной области (будем считать, что значение $x(n)$ определено для целых n принимающих значения от 0 до $N - 1$)

$X(m)$ – спектр сигнала $x(n)$ в частотной области (вообще говоря, определен для любого целого значения m , но как правило в качестве спектра используется только N его индексов)

Отметим, что, вообще говоря, отсчеты как входного сигнала x , так и его спектра X в формуле 1.3 – комплексные. Попробуем также объяснить смысл приведенного преобразования. Цель, ради которой мы будем производить данное преобразование – исследование исходного дискретного сигнала в частотной области. Поэтому мы будем придерживаться следующей интерпретации получаемого в результате ДПФ спектра: если исходный сигнал x был дискретизирован с частотой дискретизации f_s , то m -ый отсчет спектра исходного сигнала X представляет собой «вклад» составляющей с частотой $f = \frac{mf_s}{N}$ в исходное представление сигнала x в частотной области. Частоту f также называют частотой анализа ДПФ [5, с.66]. Постараемся разъяснить смысл приведенной выше фразы. Под «вкладом» подразумевается амплитуда и фаза синусоиды с частотой f в получаемом представлении дискретного сигнала как суммы его гармоник. Поскольку, как уже было упомянуто выше, отсчеты как исходного сигнала, так и его спектра в общем случае комплексные, амплитуде синусоиды с частотой f соответствует модуль m -ого комплексного отсчета получаемого спектра $|X(m)|$, а фазе синусоиды с частотой f – фаза m -ого комплексного отсчета получаемого спектра $\angle X(m)$. Хотя приведенная выше интерпретация и не является математически точной, она является достаточной для корректной интерпретации результатов ДПФ в практических приложениях, на которые мы будем ориентироваться в представленной работе. Также отметим, что в вычислительной технике как правило вместо вычисления ДПФ по формуле 1.3 напрямую пользуются различными подходами, которые ускоряют процесс вычисления ДПФ. Алгоритмы, ускоряющие вычисление ДПФ называют алгоритмами быстрого преобразования Фурье (БПФ).

Не меньшее значение чем само дискретное преобразование Фурье имеет его инверсия, обратное дискретное преобразование Фурье, позволяющее восстановить исходный сигнал по его спектру. Данное преобразование вычисляется по следующей формуле [6, с.289]:

$$x(n) = \frac{1}{N} \sum_{m=0}^{N-1} X(m) \cdot e^{\frac{j2\pi nm}{N}} \quad (1.4)$$

Заметим, что приведенное выше выражение отличается от формулы прямого ДПФ 1.3 лишь знаком в показателе комплексной экспоненты и наличием множителя $\frac{1}{N}$ перед оператором суммирования. Вообще говоря, в размещении множителя $\frac{1}{N}$ в формулах 1.3 и 1.4 нет полного единства. Встречаются 3 варианта: представленный выше, с множителем $\frac{1}{N}$ в выражении для ДПФ, и с множителем $\frac{1}{\sqrt{N}}$ как в выражении для ДПФ, так и в выражении для ОДПФ [6, с. 289]. В дальнейшем, при обсуждении ДПФ и ОДПФ мы будем придерживаться подхода, использованного в формулах 1.3 и 1.4.

1.2. Постановка задачи

Теперь поговорим непосредственно о решаемой задаче. Данная работа посвящена очищению человеческой речи от нелинейных искажений, связанных с ограничением сигнала по амплитуде. Мы сосредоточимся на конкретном виде таких искажений, называемом клиппингом. Клиппинг – это нелинейные искажения звукового сигнала вследствие жесткого ограничения его амплитуды [7]. При разработке различных программных решений для восстановления клиппинга в данной работе мы будем ориентироваться на следующий набор параметров, как наиболее характерный формат хранения цифрового звука (хотя ограниченная поддержка других форматов входных данных также будет присутствовать):

- звуковой файл формата WAV с одним каналом;
- знаковая 16-битная PCM;
- частота дискретизации – 44.1 кГц;

Под нелинейными искажениями мы будем понимать искажения, изменяющие спектр исходного сигнала (как правило, это выражается в появлении в сигнале гармоник, изначально не присутствовавших в нем). В случае с клиппингом звука в спектре сигнала появляются новые высокочастотные гармоники, из-за чего клиппинг более заметен на слух, нежели искажения иной природы с тем же коэффициентом нелинейных искажений [7].

Клиппинг может иметь различную природу, как цифровую, так и аналоговую. Клиппинг аналоговой природы происходит в ситуации, когда сигнал, поступающий с усилителя, выходит за пределы его динамического диапазона. Таким образом, верхняя часть формы звуковой волны оказывается усеченной, что вызывает появление в спектре гармоник высокой частоты (как известно функции похожей природы в результате разложения в ряд Фурье дают бесконечный ряд высокочастотных гармоник, при этом амплитуда высокочастотных гармоник убывает относительно медленно).

Клиппинг цифровой природы происходит по схожим причинам: амплитуда сигнала превышает пределы кодируемых АЦП значений, из-за чего, как правило, происходит фиксация амплитуды сигнала на уровне верхней или нижней границы шкалы [7]. Проиллюстрировать клиппинг можно изображением искаженного 8-битного сигнала со знаковой РСМ, представленным на рисунке 2.

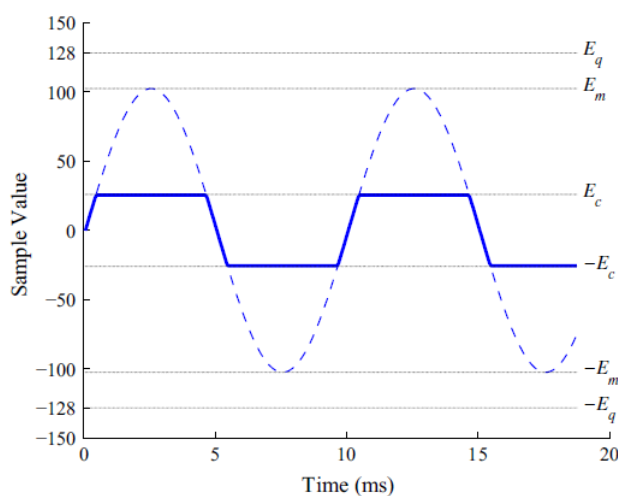


Рис. 2 Клиппинг синусоидального сигнала [8]

На рисунке 2 E_c – уровень клиппинга (уровень, при котором происходит отсечение амплитуды исходного сигнала), E_m – максимальное значение исходного сигнала, E_q – максимальное представимое в текущей модуляции значение амплитуды сигнала.

Таким образом, задача состоит в ликвидации описанного выше вида нелинейных искажений и восстановлении исходного вида звуковой волны по ее усеченной версии. Существующие способы решения данной проблемы будут описаны в следующем подразделе данной работы.

1.3. Существующие решения проблемы клиппинга

В данном подразделе нашей работы рассмотрим существующие решения проблемы клиппинга и полученные ранее результаты в изучаемой области. Вообще говоря, для простоты можно разделить решения для борьбы с клиппингом на 2 категории: с использованием нейронных сетей и без. Сначала рассмотрим подходы к решению данной проблемы без использования нейронных сетей. Среди самых популярных таких решений можно выделить следующие (к каждому подходу дадим краткое пояснение его сути):

- Интерполяция кубическими сплайнами. Подход, использующий предположение, что форма сигнала в области клиппинга может быть выражена алгебраическим полиномом третьей степени, зависящим от времени. Данный подход получил достаточно широкое распространение несмотря на свою ограниченную эффективность в силу простоты своей имплементации. Практическая имплементация компенсации клиппинга с использованием данного подхода будет рассмотрена во второй главе данной работы.
- Рекурсивное проецирование векторов (англ. RVP – Recursive Vector Projection). Данный метод работает с выборкой смешанных данных (т.е. входные данные алгоритма – клиппинг и прилегающие к нему области неискаженных семплов), представляемой в виде вектора. Данный вектор раскладывается в линейную комбинацию заранее подготовленных базисных векторов. При этом в разложении используется только часть

векторов исходного базиса, подбираемых итеративно на основании евклидовой нормы проекции вектора базиса на текущий вектор ошибки разложения [9, с. 788].

- SPADE (англ. SParse Audio DEclipper – разреженный деклиппер аудио) – итеративный алгоритм минимизации ошибки, похожий на RVP и также использующий подготовленный заранее набор векторов, представляющих собой базис разложения, но использующий различные слагаемые для регуляризации целевой минимизируемой функции, и минимизирующий норму разности между текущим комплексным вектором разложения спектра сигнала и его амплитудно-маскированной («разреженной») версией, в которой маскированы все отсчеты кроме k отсчетов с наибольшими по модулю значениями [10].
- Методы приближения спектра сигнала. Скорее не один, а группа методов, использующая в качестве отправной точки не само представление сигнала во временной области, а его спектр, получаемый в результате ДПФ. Одна из описанных реализаций этого подхода использует разбиение исходного сигнала во временной области на кадры из 512 семплов с наложением, вычисление спектра каждого кадра, и интерполяция амплитуд высокочастотных отсчетов спектра кадра с клиппингом с помощью спектров ближайших к нему во времени кадров, не содержащих клиппинг [11, с. 6].

Комплексная оценка эффективности представленных алгоритмов в изученных нами тематических статьях не производилась. В представленной работе будет произведена комплексная оценка качества восстановления с помощью алгоритма интерполяции кубическими сплайнами и сравнение полученных результатов с результатами интерполяции с помощью нейросети.

Теперь рассмотрим подходы к решению данной проблемы с использованием нейронных сетей. Среди самых популярных подходов к решению этой проблемы можно выделить следующие:

- Open-Unmix. Данное решение было представлено как модель для разделения музыкальных дорожек. Модель основана на трех двунаправленных LSTM-слоях, использует как входные данные амплитудную спектрограмму исходного сигнала (похожий подход будет использоваться в данной работе), и возвращает маску, поэлементное перемножение спектрограммы исходного сигнала на которую дает спектрограмму результирующего сигнала [12].
- Wave-U-Net. Это решение было предложено как один из подходов для разделения музыкальных дорожек, использующих представление сигнала во временной области. Следовательно, данное решение использует не только амплитуду, но и фазу музыкальных сигналов. Поскольку данная модель была успешно использована не только для разделения дорожек, но и для микширования, можно предположить, что она подходит для широкого спектра преобразований звука, включающего такую задачу как компенсация клиппинга [12].
- Demucs. Решение для разделения музыкальных дорожек на основе Wave-U-Net. Также как и Wave-U-Net, использует представление сигнала во временной области. Содержит ряд улучшений архитектуры относительно Wave-U-Net модели. Модель включает в себя кодировщик на основе сверточного слоя, структуру из двунаправленных LSTM-слоев, и декодировщик на основе сверточного слоя [12].

Для перечисленных выше моделей производилось сравнение их эффективности на датасете SignalTrain с внесенным в него клиппингом разных уровней. Сам датасет содержит более чем 24 часа музыки и сгенерированных случайных сигналов. Результаты оценивались с помощью показателей SI-SNR и показателя PEAQ. Второй из этих двух показателей более приближен к воспринимаемому человеческим ухом качеству звука, поэтому рассмотрим полученные для него результаты. В сильно искаженной выборке исходного звука (SI-SNR выборки с клиппингом примерно 5 децибел), Open-Unmix

показал небольшое ухудшение уровня PEAQ в исходной выборке, а Wave-U-Net и Demucs – значительное улучшение [12]. При этом Wave-U-Net демонстрирует улучшение показателя PEAQ порядка 2 баллов, а Demucs – порядка 2,5 баллов [12]. Таким образом, Demucs является перспективной моделью для компенсации клиппинга, о чем свидетельствуют активные исследования по дальнейшей оптимизации данной архитектуры для задач компенсации клиппинга [13].

1.4. Выводы

1. Базовые понятия предметной области представленной работы: дискретизация сигнала, квантование, преобразование Фурье, ДПФ.
2. Решаемая в представленной работе задача – компенсация нелинейных искажений, связанных с жестким ограничением амплитуды сигнала (клиппингом) в цифровом звуке.
3. Исследование разработанных решений проблемы клиппинга показывает, что использование нейросетевого подхода к данной проблеме является перспективной темой исследования.

Базовые понятия предметной области и формулировка задачи являются необходимым теоретическим минимумом для работы по решению поставленной задачи, которая будет проведена в последующих разделах. Знания о существующих решениях проблемы клиппинга будут полезны как при имплементации одного из таких решений во 2 разделе работы, так и при разработке собственного решения для компенсации клиппинга в 3 разделе.

2. СПЛАЙНОВАЯ ИНТЕРПОЛЯЦИЯ КАК ПОДХОД К ПОДАВЛЕНИЮ НЕЛИНЕЙНЫХ ИСКАЖЕНИЙ ЦИФРОВОГО ЗВУКА

2.1. Введение в сплайновую интерполяцию цифрового звука

Существует множество разных подходов к решению проблемы нелинейных искажений как таковой. Основным различием данных подходов является исходное предположение о действительной форме сигнала в восстанавливаемой области. В данном разделе работы мы рассмотрим один из подходов к сплайновой интерполяции искаженного звука. Соответственно, гипотезой о форме сигнала в восстанавливаемой области будет являться предположение о том, что данная форма может быть выражена через некоторый полином третьей степени, зависящий от времени. Нахождению действительных коэффициентов этого полинома и будет посвящено дальнейшее повествование этого раздела. Разработанное программное решение для компенсации клиппинга с использованием данного подхода является компилируемой программой на языке Си, исходный код которой содержится в двух файлах: `restore.c`, содержащем основной код разработанной программы, и заголовочном файле `wave.h`, содержащем набор объявлений, реализующих чтение заголовка и данных WAV-файлов.

2.2. Обнаружение клиппинга

Но перед тем, как восстанавливать поврежденную область, необходимо ее обнаружить. Поскольку в нашей работе рассматривается случай жесткого цифрового клиппинга, нам необходимо обнаружить каким-либо образом область «отсечения» пиков звукового файла. Мы будем предполагать, что пороги клиппинга звука симметричны – то есть мы имеем равное абсолютное значение порога отсечения сигнала как при достижении максимума, так и при достижении минимума. Так же нам может понадобится некоторый «зазор» между реальным значением максимума сигнала и значением, при котором фиксируется клиппинг (то есть фиксация клиппинга должна происходить в случае, когда значение семпла слабо отличается от абсолютного максимума сигнала). Для регулирования этого «зазора» программе передается

относительный порог фиксации клиппинга, представляющий собой действительное число в интервале $(0,1)$. Обозначим относительный порог фиксации клиппинга греческой буквой α . В соответствии с вышеописанными требованиями мы будем использовать следующий двухэтапный алгоритм выявления областей клиппинга. Первый этап – нахождение абсолютного максимума значений семпла звукового файла. Второй этап – нахождение во входных данных интервалов, требующих восстановления. Сами интервалы восстанавливаемых данных выражаются в виде индексов их начала и конца. Обозначим соответственно индекс начала интервала клиппинга как t_0 , а индекс конца как t_1 . Сами эти индексы находятся по следующим условиям:

$$t_0: \begin{cases} samples_{t_0-1} < \alpha \cdot peak \\ samples_{t_0} \geq \alpha \cdot peak \end{cases} \quad (2.1)$$

$$t_1: \begin{cases} samples_{t_1-1} \geq \alpha \cdot peak \\ samples_{t_1} < \alpha \cdot peak \end{cases} \quad (2.2)$$

Здесь:

samples – массив семплов исходного звукового файла, соответственно *samples_i* обозначает *i*-ый семпл исходного звукового файла

peak – пиковое значение семпла исходного звукового файла; обычно определяется как максимальное по модулю значение в массиве *samples*

Программа составляет два массива, содержащих соответственно индексы начала и индексы конца интервалов клиппинга (индексы начал определяются согласно условию 2.1, индексы концов – согласно условию 2.2). Именно семплы в этих интервалах будут восстанавливаться программой. В следующем подразделе мы рассмотрим сам процесс восстановления поврежденных семплов в области клиппинга.

2.3. Интерполяция кубическими сплайнами

Сначала введем необходимые понятия. Сплайн – функция в математике, область определения которой разбита на конечное число отрезков, на каждом из которых она совпадает с некоторым алгебраическим полиномом [14]. Соответственно, кубический сплайн – функция, заданная на каждом отрезке

полиномом, степень которого не превышает третью. Интерполяцией мы будем называть процесс нахождения значений сигнала во временном промежутке между известными значениями. Абстрактно опишем ситуацию, с которой мы сталкиваемся в случае клиппинга звукового файла. Клиппинг создает в исходном сигнале непересекающиеся отрезки из поврежденных (или недействительных) отсчетов. Самым очевидным решением для интерполяции в этом случае является использование одного кубического полинома $P(t)$ для одного «поврежденного» отрезка. Очевидно, чтобы найти 4 коэффициента этого полинома нужно составить систему из 4 уравнений. Предположим, нам известны координаты t_0 и t_1 начала и конца поврежденного отрезка во времени, значения амплитуды в этих концах f_0 и f_1 , а также значения f'_0 и f'_1 производной кубического полинома в соответствующие концам моменты времени. Преобразуем это в условия, из которых можно составить необходимую систему уравнений:

$$\begin{cases} P(t_0) = f_0 \\ P(t_1) = f_1 \\ P'(t_0) = f'_0 \\ P'(t_1) = f'_1 \end{cases} \quad (2.3)$$

Поскольку $P(t)$ в формуле 2.3 – полином третьей степени, его можно записать в виде $P(t) = d(t - t_0)^3 + c(t - t_0)^2 + b(t - t_0) + a$, а его производную в виде $P'(t) = 3d(t - t_0)^2 + 2c(t - t_0) + b$. Подставляя в эти выражения t_0 получаем, что $P(t_0) = a$, а также $P'(t_0) = b$. Таким образом, $a = f_0$ и $b = f'_0$.

Аналогично, подставляя в эти выражения t_1 , получаем:

$$f_1 = d(t_1 - t_0)^3 + c(t_1 - t_0)^2 + b(t_1 - t_0) + a \quad (2.4)$$

$$f'_1 = 3d(t_1 - t_0)^2 + 2c(t_1 - t_0) + b \quad (2.5)$$

Подставив в формулу 2.4 $a = f_0$ и в формулу 2.5 $b = f'_0$ и перегруппировав члены, получаем:

$$d(t_1 - t_0)^3 + c(t_1 - t_0)^2 = f_1 - f_0 - f'_0(t_1 - t_0) \quad (2.6)$$

$$3d(t_1 - t_0)^2 + 2c(t_1 - t_0) = f'_1 - f'_0 \quad (2.7)$$

Выразим значение c из равенства 2.6 и значение d из равенства 2.7. Для этого умножим равенство 2.6 на 3 и вычтем из него равенство 2.7 умноженное на $t_1 - t_0$:

$$c(t_1 - t_0)^2 = 3f_1 - 3f_0 - 3f'_0(t_1 - t_0) - f'_1(t_1 - t_0) + f'_0(t_1 - t_0) \quad (2.8)$$

Из равенства 2.8 получаем:

$$c = \frac{3(f_1 - f_0)}{(t_1 - t_0)^2} - \frac{f'_1 + 2f'_0}{t_1 - t_0} \quad (2.9)$$

Чтобы найти d , вычтем из умноженного на $t_1 - t_0$ уравнения 2.7 уравнение 2.6, умноженное на 2:

$$d(t_1 - t_0)^3 = (f'_1 - f'_0)(t_1 - t_0) - 2f_1 + 2f_0 + 2f'_0(t_1 - t_0) \quad (2.10)$$

Из равенства 2.10 получаем:

$$d = \frac{f'_1 + f'_0}{(t_1 - t_0)^2} - \frac{2(f_1 - f_0)}{(t_1 - t_0)^3} \quad (2.11)$$

Подставляя ранее полученные выражения для a и b , а также выражения 2.9 и 2.11 для c и d в формулу $P(t)$, получаем:

$$\begin{aligned} P(t) = f_0 + f'_0(t - t_0) + \left(\frac{3(f_1 - f_0)}{(t_1 - t_0)^2} - \frac{f'_1 + 2f'_0}{t_1 - t_0} \right) (t - t_0)^2 \\ + \left(\frac{f'_1 + f'_0}{(t_1 - t_0)^2} - \frac{2(f_1 - f_0)}{(t_1 - t_0)^3} \right) (t - t_0)^3 \end{aligned} \quad (2.12)$$

Выведенный способ интерполяции также называют «интерполяцией эрмитовыми кубическими сплайнами». Заметим, что в формуле 2.12 мы видим зависимость от значений производной на границах «поврежденного» отрезка f'_0 и f'_1 . Если точные значения f_0 и f_1 нам известны непосредственно из входных данных, то значения f'_0 и f'_1 – нет. Для их нахождения мы можем воспользоваться разностными формулами производной. Заметим, что использование точек справа от t_0 и слева от t_1 для нахождения производной является нежелательным, так как значения в этом диапазоне как раз и являются значениями, поврежденными клиппингом. Таким образом, для нахождения производной в точке t_0 мы будем использовать левую разностную производную, а для нахождения производной в точке t_1 – правую разностную

производную. Иными словами, в нашей программе f'_0 и f'_1 находятся по следующим двум формулам:

$$P'(t_i) \approx \frac{f_i - f_{i-d}}{d} \quad (2.13)$$

$$P'(t_j) \approx \frac{f_{j+d} - f_j}{d} \quad (2.14)$$

Здесь:

t_i – начало области клиппинга

t_j – конец области клиппинга

f_k – значение сигнала в момент времени t_k

d – некоторое натуральное число, выражающее расстояние между двумя точками, которые используются для вычисления производной

Отметим, что значение d может ощутимо повлиять на приближенное значение производной, получаемое по формулам 2.13 и 2.14 даже если остальные входные данные алгоритма аналогичны. В нашей программе в качестве компромиссного варианта используется $d = 3$.

На этом описание используемого алгоритма интерполяции можно считать завершенным. Отметим, что используемая данным алгоритмом гипотеза о форме сигнала в области клиппинга накладывает существенное ограничение на форму «восстановленного» сигнала. Действительно, предположим, что «эталонный» сигнал имел форму, представленную на рисунке 3.

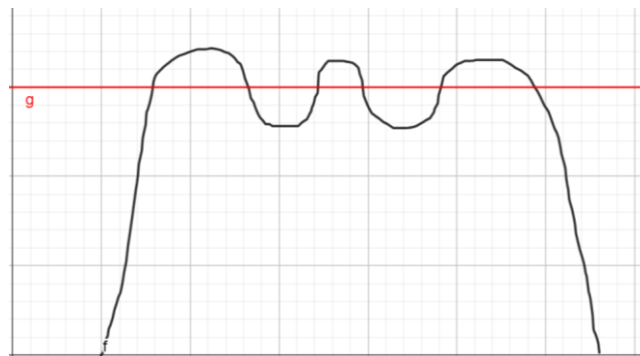


Рис. 3 Форма «эталонного» сигнала, которая не может быть восстановлена кубическим сплайном

На рисунке 3 мы видим некоторую кривую f , пересекаемую прямой g . Предположим, что g параллельна оси абсцисс, иными, словами, равняется некоторой константе (т.е. $g = c$). Из рисунка нетрудно заметить, что f пересекает g в 6 точках. Иными словами, если кривая f задается некоторой функцией от времени f^* , она равна c для 6 различных значений времени t (или, аналогично, функция $f^* - c$ имеет 6 действительных корней). Как известно, полином третьей степени не может иметь более 3 корней (вычитание константы не влияет на степень полинома), следовательно, поведение кривой f не может быть выражено никаким полиномом третьей степени P . Вышеприведенное рассуждение показывает, что интерполяция кубическими сплайнами подходит только для восстановления простых по форме «колоколообразных» пиков, но не подходит для восстановления пиков сложной формы, что и является основным ограничением рассмотренного способа интерполяции.

2.4. Особенности имплементации алгоритма восстановления

Описанный выше алгоритм имплементирован в виде компилируемой программы на языке Си. Программа принимает на вход 4 аргумента: имя исходного восстанавливаемого файла, имя файла в который будут записаны результаты восстановления, относительный порог фиксации клиппинга, изменение громкости в результирующем файле в децибелах. Разработанная программа поддерживает работу с 8, 16, и 24-битными WAV-файлами с PCM-модуляцией, содержащими не более четырех звуковых каналов. Код, связанный с чтением заголовка и сырых данных WAV-файлов с PCM-модуляцией был вынесен в заголовочный файл «wave.h», используемый данной программой. Исходный код данной программы (файлы restore.c и wave.h) может быть найден в приложениях данной работы. Для многоканального аудио все этапы восстановления производятся полностью отдельно (иными словами, в случае, если в разных каналах аудио был разный уровень клиппинга, клиппинг в каждом канале будет фиксироваться при небольшом отклонении от соответствующего конкретно этому каналу

максимуму значения сигнала). Отметим, что при восстановлении описанным способом также может возникнуть клиппинг: интерполированное значение амплитуды в одной из точек, соответствующих области клиппинга может превысить диапазон представимых значений шкалы звукового файла. В такой ситуации нужно передать программе восстановления отрицательный четвертый аргумент: соответствующее изменение громкости будет производиться для массива «сырых» значений перед записью в файл, таким образом, клиппинг при записи результата в файл не возникнет. Формула пересчета значения в децибелах в множитель семплов исходного файла выглядит следующим образом:

$$gain = 10^{\frac{val}{20}} \quad (2.15)$$

Здесь:

gain – значение, на которое будут умножены все семплы перед записью окончательного результата

val – уровень изменения громкости в результирующем файле в децибелах

Таким образом, все результирующие семплы перед записью окончательного будут умножены на значение *gain*, вычисляемое согласно формуле 2.15.

Чтобы резюмировать приведенное выше объяснение особенностей имплементации программы, рассмотрим основной блок кода, связанный с компенсацией клиппинга. Данный блок кода находится в функции `process_buffer` исходного файла `restore.c` (код данной функции можно найти в приложениях к работе).

В теле функции `process_buffer` можно заметить 2 цикла `for`. Первый цикл представляет собой нахождение индексов начала и конца фрагментов клиппинга, и, соответственно, заполнение двух массивов: `exit_list` для индексов начала областей клиппинга, и `return_list` для индексов конца областей клиппинга. При этом переменные `exit_count` и `return_count` хранят длины массивов `exit_list` и `return_list` соответственно. Заметим, что, если клиппинг происходит в самом начале или конце звуковой дорожки, где отсутствуют

необходимые точки слева или справа для разностного вычисления производной, соответствующие области клиппинга исключаются из массивов обрабатываемых индексов. Второй цикл реализует восстановление областей клиппинга, извлекая последовательные пары индексов из массивов `exit_list` и `return_list` соответственно и изменяя значения некорректных семплов в массиве `buffer` (указатель на начало этого массива является аргументом функции `process_buffer`) напрямую.

2.5. Выводы

Во втором разделе были получены следующие результаты:

1. Предложен способ обнаружения клиппинга с использованием относительного порога обнаружения клиппинга α .
2. Выведена формула кубической эрмитовой интерполяции, а также предложен способ применения данной формулы для решения задачи компенсации искажений, вызванных клиппингом.
3. Алгоритм интерполяции цифрового звука с использованием кубических сплайнов имплементирован в виде компилируемой программы на языке Си.

Рассмотренные в данном разделе теоретические аспекты интерполяции кубическими сплайнами являются основами разработанной программы для интерполяции (исходный код данной программы представлен в приложениях к данной работе). Данная имплементация будет использована в качестве базового метода для оценки результатов восстановления с использованием разработанной архитектуры нейросетей в 4 разделе данной работы.

3. ПОДАВЛЕНИЕ НЕЛИНЕЙНЫХ ИСКАЖЕНИЙ ЦИФРОВОГО ЗВУКА ПРИ ПОМОЩИ НЕЙРОСЕТЕЙ

В данном разделе мы рассмотрим обучение и применение Seq2Seq нейросети для восстановления цифрового звука, искаженного клиппингом. Для реализации скриптов, с помощью которых производится создание и обучение нейросети мы будем применять фреймворк Keras с бекэндом Tensorflow. Отметим, что в отличие от разработанной на языке Си программы для интерполяции реализуемые скрипты будут работать только с WAV-аудио с 16 или 32-битной РСМ-модуляцией с одним каналом. Представленный набор скриптов включает:

- скрипт для подготовки обучающей выборки `rf64_clip.py`;
- скрипт с базовыми объявлениями, необходимыми для обучения и восстановления с помощью обученной нейросети `base.py`;
- скрипт обучения нейросети `script.py`;
- скрипт для восстановления звука с помощью нейросети `inference_batch_prediction.py`;

Но прежде, чем обучать, и тем более применять нейросеть, нужно разобраться со способом получения обучающей выборки. Поэтому мы рассмотрим наш подход к восстановлению звука с помощью глубокого машинного обучения в следующей хронологии: подготовка обучающей выборки, архитектура нейросети и ее обучение, применение нейросети для восстановления звука.

3.1. Подготовка обучающей выборки

Для обучения нейросети мы будем использовать человеческую речь с искусственно внесенным в нее клиппингом. Для этого мы напишем специальный скрипт для создания такой выборки. Разработанный скрипт будет работать с одноканальной речью с 16 или 32-битной РСМ-модуляцией. При этом обучаемая нейросеть будет принимать на вход нормированные последовательности длиной 512 семплов исходной звуковой дорожки, возвращая последовательности длиной 512 семплов в нормированной форме.

В соответствии с этим, обучающая выборка будет состоять из нормированных последовательностей звука с внесенным клиппингом и нормированных последовательностей исходной звуковой дорожки до внесения в нее клиппинга. Однако, выборки речи на которых будет производиться обучение могут оказаться достаточно большими (для хороших результатов работы нейросети нужно много данных). Принципиальное ограничение RIFF контейнера, используемого в формате WAV – не более 4 Гб аудиоданных (в данном контейнере используется 32-битное поле, выражающее размер блока данных в байтах)[15]. Из-за этого для обучения нашей нейросети нам придется использовать либо множество WAV-файлов, размер которых не превышает допустимого предела, либо какой-то другой медиаконтейнер. Мы предпочтем второй подход, и вместо стандартного RIFF мы будем использовать RIFF 64 для хранения обучающей выборки. В скрипте обучения `script.py` для чтения последовательностей из WAV-файла с медиаконтейнером RIFF 64 мы будем использовать библиотеку `wave-bwf-rf64`, но в скрипте для создания обучающей выборки `rf64_clip.py` (код данного скрипта можно найти в приложениях к работе) мы будем использовать ручное чтение заголовка файла. Мы используем более сложный способ в силу того, что в случае со скриптом для генерации клиппинга нам нужно произвести не просто чтение сырых семплов исходного файла, а извлечь заголовок исходного файла как массив байтов, который будет записан в результирующий файл. Типовая структура звукового файла с медиаконтейнером RIFF 64 представлена на рисунке 4.

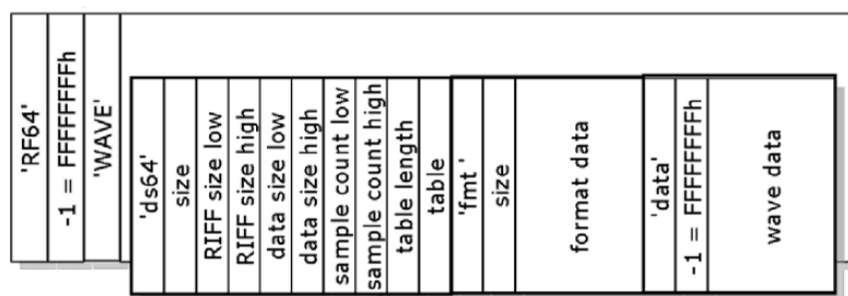


Рис. 4 Структура файла с медиаконтейнером RIFF 64 [15]

Каждое из полей на рисунке 4 кроме полей 'table', 'format data', и 'wave data' 32-битное [15]. Код для чтения заголовочных данных RIFF 64 файла в скрипте `rf64_clip.py` имплементирует описанный далее алгоритм. Основная часть этого алгоритма – считывание чанков исходного файла пока не будет встречен чанк 'data', содержащий непосредственно звуковые данные файла. При этом, если имя чанка 'ds64' или 'fmt', из него извлекаются соответствующие данные. Из чанка 'ds64' извлекаются 3 следующих поля: размер блока RIFF 64, размер чанка 'data' (если поле размера файла равно 0xFFFFFFFFh, это означает, что вместо его значения нужно использовать значение из данного поля [15]), и фактическое количество семплов в файле; из чанка 'fmt' извлекается число байт на семпл в исходном файле. Если имя чанка не 'data', и ни одно из вышеперечисленных, данный чанк просто считывается из входного файла без парсинга и копируется в результирующий файл в том же виде в каком он был прочитан из входного файла.

Скрипт для создания обучающей выборки принимает на вход следующие 4 параметра: имя файла с исходной (неискаженной выборкой) в формате WAV с контейнером RIFF 64, имя файла, куда будет записана эта выборка с наложенным клиппингом, минимальная глубина клиппинга в децибелах, максимальная глубина клиппинга в децибелах. Скрипт читает исходный файл кусками размера *chunk_size* байт, генерирует псевдослучайное значение уровня клиппинга в децибелах в заданном диапазоне и производит симметричное отсечение со сгенерированным значением отсечения. Пересчет сгенерированного уровня клиппинга в

децибелах в пороговое значение для семплов сигнала вычисляется по формуле:

$$val = maxval \cdot 10^{-\frac{dbvalue}{20}} \quad (3.1)$$

Здесь:

val – предел отсечения, который будет накладываться для конкретного куска (получившееся нецелое значение будет округляться до ближайшего целого)

dbvalue – случайный уровень клиппинга в текущем куске в децибелах

maxval – модуль максимально представимого в текущей шкале значения семпла (например, 32768 для 16-битной РСМ со знаком)

Отметим, что случайная величина, выражающая значение псевдослучайного порога отсечения имеет равномерное распределение. Это означает, что при разбиении исходного отрезка допустимых значений на непересекающиеся отрезки равной длины вероятность попадания псевдослучайного значения в каждый из отрезков одинакова. Таким образом, в обучающей выборке в равной степени будет представлен каждый из уровней клиппинга в заданном диапазоне.

Еще одной проблемой, связанной с самой постановкой задачи обучения модели для большой выборки исходных данных является обеспечение правильной генерации обучающей выборки нейросети. Очевидно, что при работе с большим количеством обучающих данных загрузка исходных данных в оперативную память является не лучшей идеей – скорее всего обучающая выборка в нее не влезет, что приведет к пробуксовке (состояние, когда подсистема виртуальной памяти компьютера находится в состоянии постоянного свопинга, часто обменивая данные в памяти и данные на диске, в ущерб выполнению приложений) и ощутимому замедлению самого процесса обучения. Чтобы избежать этого, нам необходимо загружать исходные данные из файла на диске непрерывными кусками относительно небольшого размера. Мы будем использовать для этой цели разработанную имплементацию генератора обучающей выборки. Данный генератор будет читать файлы

обучающей выборки с диска и «кэшировать» их небольшими кусками в оперативной памяти. Важное замечание: обычно при обучении с помощью градиентного спуска используется перемешивание данных в обучающей выборке. Но если мы попытаемся перемешать вообще все последовательности в обучающей выборке мы не сможем обеспечить эффективного кэширования: действительно, каждую пачку для нашего кэша нам придется читать из разных частей исходного файла, что существенно замедлит работу генератора обучающей выборки. Поэтому мы считываем с диска непрерывный блок данных, соответствующий размеру кэша, производим перемешивание пачек в нем, и изменяем порядок последовательностей в рамках каждой пачки. Таким образом, данная имплементация достигает компромисса между максимально возможным перемешиванием исходной выборки (что важно для эффективной работы алгоритма стохастического градиентного спуска) и обеспечением максимально возможной скорости работы генератора. Параметры, которые мы будем использовать при создании обучающей выборки нейросети: размер пачки – 512 пар последовательностей (напомним, что одна последовательность содержит 512 нормированных семплов из звуковых файлов с человеческой речью), размер кэша – 20 пачек, производится перемешивание как последовательностей в пачке, так и пачек в кэше. Сами исходные данные для создания обучающей выборки – 20 часов студийных подкастов при частоте дискретизации 44,1 кГц, с 16-битной РСМ-модуляцией. Соответственно, в обучающую выборку вносится клиппинг с уровнем от 5 до 15 дБ, изменяемым каждую секунду (т.е. через каждые 44100 семплов) исходной выборки. Согласно формуле 3.1, это соответствует уровню клиппинга от $10^{\frac{-15}{20}} \approx 17,78\%$ до $10^{\frac{-5}{20}} \approx 56,23\%$ от максимального уровня представимых значений в знаковой РСМ.

3.2. Оконное дискретное преобразование Фурье и его инверсия

Разработанная нами архитектура нейросети будет восстанавливать искаженный сигнал используя преобразования, осуществляемые как во

временной, так и в частотной областях. Как правило, обработка сигнала в частотной области ассоциируется с преобразованием спектра сигнала, получаемого с помощью преобразования Фурье. Однако, восстанавливаемый сигнал содержит также локализованную во времени составляющую искажений (области клиппинга локализованы в конкретных временных промежутках исходного сигнала), тогда как стандартное ДПФ возвращает отсчеты спектра для всего временного промежутка исходного сигнала. Все это обуславливает необходимость использовать преобразование, отсчеты которого выражают не просто спектр исходного сигнала, а то, как этот спектр изменялся во времени. Именно таким преобразованием является оконное ДПФ, о котором пойдет речь в данном разделе.

Оконное дискретное преобразование Фурье может быть найдено по следующей формуле:

$$F(n, \omega) = \sum_{m=-\infty}^{\infty} x[m] \cdot w[n - m] \cdot e^{-j\omega m} \quad (3.2)$$

Здесь:

$F(n, \omega)$ – комплексное значение, соответствующее отсчету с индексом n преобразуемого сигнала и частоте преобразования ω

$x[m]$ – m -ый отсчет исходного сигнала во временной области

$w[n - m]$ – $(n - m)$ -ый отсчет оконной функции w

Очевидно, что для вычисления оконного ДПФ с помощью формулы 3.2 значения x и w должны быть определены для любого целого индекса. Поскольку рассмотрение бесконечно длинных сигналов и окон является непрактичным, в дальнейшем повествовании мы явным образом определим значения только для конечного числа индексов этих последовательностей, при этом их значения для остальных индексов будем полагать равными нулю.

Как видно из приведенной формулы, окно w в данном преобразовании используется для извлечения части исходного сигнала x конечной длины, такой что спектральные характеристики извлеченной части приблизительно

стационарны. Предположим, что оконная функция w содержит конечное число R ненулевых отсчетов, причем индекс ненулевого отсчета удовлетворяет неравенству $0 \leq m \leq R - 1$, в то время как исходный сигнал x содержит N ненулевых отсчетов, индексы которых удовлетворяют неравенству $0 \leq n \leq N - 1$. Используя это положение, а также равенство $\omega = \frac{2\pi k}{N}$, приведенное выше выражение для оконного дискретного преобразования Фурье можно переписать в следующем виде:

$$F(n, k) = \sum_{m=n-(R-1)}^{m=n} x[m] \cdot w[n - m] \cdot e^{-\frac{j2\pi km}{N}} \quad (3.3)$$

Заметим, что, если считать n в формуле 3.3 константой, мы получаем ДПФ от поэлементного произведения исходного сигнала с окном $x[m] \cdot w[n - m]$. Таким образом, оконное преобразование Фурье можно считать эквивалентным многократному ДПФ от поэлементных произведений исходного сигнала с перемещаемым во времени окном [16, с. 768]. Нетрудно увидеть, что формула 3.3 выражает ДПФ для отсчетов x с индексами от $n - (R - 1)$ до n включительно, т.е. отсчеты слева от отсчета с индексом n . В практических имплементациях оконного преобразования Фурье как правило производится «центрирование» окна таким образом, чтобы преобразуемое поэлементное произведение включало примерно одинаковое число отсчетов сигнала слева и справа от «опорного» отсчета с индексом n .

Следует также отметить два важных момента. ДПФ, соответствующий срезу получаемой по формуле 3.3 спектрограммы по времени $F(n, \cdot)$ является интерполированным ДПФ от R отсчетов произведения исходного сигнала с окном. Иными словами, получаемый результат эквивалентен ДПФ от R отсчетов произведения исходного сигнала с окном, дополненных $N - R$ нулевыми отсчетами на конце. Чтобы получить результат, соответствующий простому ДПФ от R отсчетов произведения исходного сигнала с окном, нужно заменить N на R в знаменателе показателя экспоненты в формуле 3.3. Вторым моментом, который мы хотели отметить является тот факт, что при

вычислении спектрограммы по формуле 3.3 переход от $F(n, k)$ к $F(n + 1, k)$ эквивалентен сдвигу окна на один отсчет вперед, т.е. вычисление оконного ДПФ с единичным шагом по времени приводит к получению комплексной спектрограммы с большим уровнем избыточности. В силу этого, можно использовать вместо единичного шага некоторое целое значение шага L , поменяв индексацию отсчетов по времени в исходной формуле. С учетом двух вышеприведенных поправок формулу 3.3 можно переписать в следующем виде:

$$F(l, k) = \sum_{m=l \cdot L - (R-1)}^{m=l \cdot L} x[m] \cdot w[l \cdot L - m] \cdot e^{-\frac{j2\pi k(m-l \cdot L + R-1)}{R}} \quad (3.4)$$

Важно понимать, что формула 3.4 учитывает большинство аспектов практической имплементации оконного дискретного преобразования Фурье в вычислительной технике за исключением вышеупомянутого центрирования окна, которое будет присутствовать в имплементации оконного ДПФ из данной работы.

Напомним, что разрабатываемая нами нейросеть преобразует последовательности из 512 семплов. Мы будем использовать встроенные в Keras имплементации прямого и обратного оконного преобразования Фурье, доступные с помощью обращений `tf.keras.ops.stft` и `tf.keras.ops.istft`. Мы будем использовать следующий набор параметров дискретного оконного преобразования Фурье:

- окно – Ханна (Хэннинга) с центрированием;
- размер окна R (и используемого ДПФ) – 16 отсчетов;
- шаг (сдвиг) окна L – 8 отсчетов;

Выбор окна Ханна обусловлен его простотой, умеренно низким уровнем растекания получаемого с его использованием спектра [17], и инвертируемостью получаемого результата с помощью OLA (англ. Overlap-add method – метод совмещения и сложения) для интересующих нас уровней наложения. Заметим, что инвертируемость дискретного оконного

преобразования Фурье с помощью OLA обеспечивается не для всех наборов исходных параметров. Необходимым и достаточным условием инвертируемости дискретного оконного преобразования Фурье является следующее условие [18]:

$$\sum_l w^{\alpha+1}[n - lL] \neq 0 \quad (3.5)$$

Здесь:

w – используемая при вычислении дискретного оконного преобразования Фурье оконная функция

α – целый параметр, определяющий как будет выглядеть формула восстановления семплов исходного сигнала

Окно Ханна, используемое в дискретном оконном преобразовании Фурье рассчитывается по следующей формуле:

$$w_{hann}[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{R} \right) \right) \quad (3.6)$$

Заметим, что согласно формуле двойного угла, выражение 3.6 может быть переписано в следующем виде:

$$w_{hann}[n] = \sin^2 \left(\frac{\pi n}{R} \right) \quad (3.7)$$

Поскольку окно должно содержать конечное число ненулевых отсчетов, а выражение 3.7 не удовлетворяет этому условию в силу периодичности синусоидальной функции, определим окно для любого целого индекса следующим образом:

$$w_{hann}[n] = \begin{cases} \sin^2 \left(\frac{\pi n}{R} \right), & 1 \leq n \leq R - 1 \\ 0, & \text{в противном случае} \end{cases} \quad (3.8)$$

Таким образом, формула 3.8 задает значение оконной функции для любого целого значения индекса n . Попробуем доказать инвертируемость осуществляемого преобразования через условие 3.5 используя значение $\alpha = 0$. Заметим, что поскольку шаг $L = \frac{R}{2}$ в сумме в выражении 3.5 только два отсчета окна для соседних l окажутся ненулевыми. Обозначим большее из этих двух

значений за $m = n - l \frac{R}{2}$, тогда меньшее будет равно $k = n - (l + 1) \frac{R}{2} = m - \frac{R}{2}$. Найдем сумму значений окна Ханна для этих двух отсчетов:

$$\sin^2\left(\frac{\pi m}{R}\right) + \sin^2\left(\frac{\pi k}{R}\right) = \sin^2\left(\frac{\pi m}{R}\right) + \sin^2\left(\frac{\pi(m - \frac{R}{2})}{R}\right) \quad (3.9)$$

Второе слагаемое в выражении 3.9 можно переписать следующим образом:

$$\sin^2\left(\frac{\pi(m - \frac{R}{2})}{R}\right) = \sin^2\left(\frac{\pi m}{R} - \frac{\pi}{2}\right) = \cos^2\left(\frac{\pi m}{R}\right) \quad (3.10)$$

Подставляя результат в 3.10 в формулу 3.9, получаем:

$$\sin^2\left(\frac{\pi m}{R}\right) + \sin^2\left(\frac{\pi(m - \frac{R}{2})}{R}\right) = \sin^2\left(\frac{\pi m}{R}\right) + \cos^2\left(\frac{\pi m}{R}\right) = 1 \quad (3.11)$$

Результат, полученный в выражении 3.11 приводит нас к равенству 3.12:

$$\sum_l w^{\alpha+1}[n - lL] = 1 \neq 0 \quad (3.12)$$

Таким образом, для подобранных параметров дискретного оконного преобразования Фурье соблюдается необходимое и достаточное условие его инвертируемости, что доказывает корректность используемых нами параметров.

Отметим, что если условие 3.5 выполняется для какого-либо значения α , оконной функции w , и шага окна L , то для этого значения α , этой оконной функции w и этого шага окна L справедлива следующая формула инвертирования оконного дискретного преобразования Фурье [18]:

$$x[n] = \frac{\sum_l x_l[n] \cdot w^\alpha[n - l \cdot L]}{\sum_l w^{\alpha+1}[n - l \cdot L]} \quad (3.13)$$

Здесь:

$x[n]$ – n -ый отсчет исходного сигнала x

$x_l[n]$ – отсчет, соответствующий n -ому отсчету исходного сигнала x в ОДПФ

l -ого среза спектрограммы по времени

w – оконная функция, использованная для прямого оконного дискретного преобразования Фурье

α – некоторое целое значение

В нашем случае (при использовании окна Ханна со значением $\alpha = 0$) знаменатель формулы 3.13 равен единице (согласно равенству 3.12) и, следовательно, может быть отброшен. Также заметим, что при $\alpha = 0$ выражение $w^\alpha[n - l \cdot L]$ обращается в единицу и может быть отброшено. Таким образом, равенство 3.13 можно переписать в виде (обозначения оставляем теми же):

$$x[n] = \sum_l x_l[n] \quad (3.14)$$

Анализируя формулу 3.14, можно заметить ряд интересных деталей. Во-первых, как и в случае с формулой 3.5, только два отсчета x_l для соседних l окажутся ненулевыми. Во-вторых, фактически, x_l эквивалентен простому поэлементному произведению исходного сигнала с окном (как уже было сказано выше, оконное дискретное преобразование Фурье можно интерпретировать как ДПФ от поэлементных произведений исходного сигнала с перемещаемым во времени окном). Заметим, что поскольку в нашем случае $L = \frac{R}{2}$, эти два соседних поэлементных произведения являются произведением одних и тех же отсчетов x с левой и правой половинами окна Ханна. Таким образом, инвертируемость по формуле 3.14 в данном случае достигается за счет того, что левые и правые половины окна при поэлементном суммировании дают единицу. Этот факт проиллюстрирован на рисунке 5.

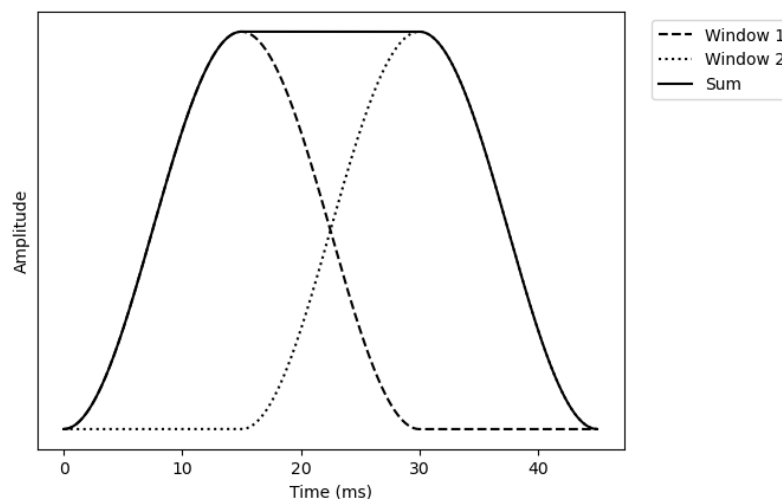


Рис. 5 Суммирование левой и правой половин окна Ханна [19]

Поскольку отсчеты исходного сигнала действительные, имплементация оконного преобразования Фурье в Keras вычисляет действительное дискретное преобразование Фурье поэлементного произведения исходного сигнала с окном, отбрасывая комплексно-сопряженные отсчеты получаемого спектра. Напомним, что действительное дискретное преобразование Фурье преобразует N действительных отсчетов исходного сигнала в $\left\lfloor \frac{N}{2} \right\rfloor + 1$ комплексных отсчетов спектра [20, с.52]. Таким образом, 16 отсчетов поэлементного произведения сигнала с окном будут преобразованы в 9 комплексных отсчетов получаемого спектра. Размер измерения времени оконного дискретного преобразования Фурье для окна с центрированием определяется по формуле $T = \left\lfloor \frac{N_{eff} - R}{L} \right\rfloor + 1$. В этой формуле $N_{eff} = N + 2 \left\lfloor \frac{R}{2} \right\rfloor$.

Как мы помним, согласно формуле 3.13 для инвертирования дискретного оконного преобразования Фурье нужно сложить ОДПФ двух соседних срезов спектрограммы по времени, что соответствует сложению одних и тех же отсчетов сигнала, поэлементно умноженных на левую и правую половины окна Ханна. Таким образом, для обеспечения инвертируемости исходного сигнала во временной области для краевых отсчетов нужно, чтобы преобразуемая последовательность содержала дополнительные $\left\lfloor \frac{R}{2} \right\rfloor$ отсчетов слева и $\left\lfloor \frac{R}{2} \right\rfloor$ отсчетов справа от своих соответствующих концов, что обеспечит

наличие ДПФ как от поэлементного произведения краевых отсчетов с левой половиной окна Ханна, так и с правой. Именно это объясняет форму выражения для N_{eff} , приведенную выше. При этом такая форма выражения для T обусловлена тем, что первый срез спектрограммы по времени покрывает R отсчетов исходного сигнала, а каждый последующий — L новых отсчетов исходного сигнала во временной области.

В нашем случае получаем $N_{eff} = N + 2 \left\lfloor \frac{R}{2} \right\rfloor = 512 + 2 \left\lfloor \frac{16}{2} \right\rfloor = 528$. Тогда $T = \left\lfloor \frac{528-16}{8} \right\rfloor + 1 = 64 + 1 = 65$. Таким образом, из 512 отсчетов исходного сигнала мы получаем двумерную комплексную спектрограмму, состоящую из 9 отсчетов по оси частоты и 65 отсчетов по оси времени. Обозначим получаемую в результате дискретного оконного преобразования Фурье комплексную спектрограмму как $F(f, t)$, где f — измерение частоты исходной спектрограммы ($0 \leq f \leq 8$), а t — измерение времени исходной спектрограммы ($0 \leq t \leq 64$). Обозначим амплитудную часть (т.е. поэлементное взятие модуля комплексных отсчетов спектрограммы) получаемой спектрограммы как $|F(f, t)|$, а ее фазовую часть (т.е. поэлементное извлечение фазы комплексных отсчетов спектрограммы) как $\angle F(f, t)$. Тогда часть алгоритма восстановления звука с помощью оконного преобразования Фурье в нашем алгоритме будет выглядеть следующим образом:

1. Вычисляем дискретное оконное преобразование Фурье от исходного сигнала (форма сигнала изменяется следующим образом: $512 \rightarrow 65 \times 9$, при этом исходный вектор состоит из действительных отсчетов, в то время как спектрограмма — это двумерный массив комплексных отсчетов).
2. Разделяем полученную спектрограмму $F(f, t)$ на амплитудную $|F(f, t)|$ и фазовую $\angle F(f, t)$ составляющие.
3. Вычисляем аддитивную маску с амплитудной спектрограммы с помощью разработанной нейросети, т.е. $|F(f, t)| \rightarrow |F_{add}(f, t)|$.

4. Получаем восстановленную спектрограмму с помощью обратной связи через поэлементное сложение исходной спектрограммы и аддитивной маски, т.е. $|F_{rest}(f, t)| = |F(f, t)| + |F_{add}(f, t)|$.

5. Объединяем восстановленную амплитудную составляющую сигнала и фазовую составляющую исходного сигнала: $F_{rest}(f, t) = |F_{rest}(f, t)| \cdot e^{j\angle F(f, t)}$.

6. Вычисляем обратное оконное дискретное преобразование Фурье для полученной комплексной спектрограммы $F_{rest}(f, t)$, т.е. производим обратный шаг 1 переход (форма сигнала изменяется следующим образом: $65 \times 9 \rightarrow 512$, при этом исходный вектор состоит из комплексных отсчетов, а восстановленный сигнал содержит действительные отсчеты).

3.3. Архитектура и обучение нейросети

В данном подразделе мы постараемся охватить все особенности архитектуры разработанной нейросети в совокупности. Мы будем использовать архитектуру разработанной нейросети в двух вариантах: с использованием простых рекуррентных слоев Keras (тип `tf.keras.layers.SimpleRNN`) и продвинутые рекуррентные слои с использованием длинной цепи элементов краткосрочной памяти (тип `tf.keras.layers.LSTM`). В целом два варианта будут отличаться только типом используемых рекуррентных слоев и функциями активации, используемыми в этих слоях. В SimpleRNN-слое мы будем использовать ReLU-активацию для выхода, а в LSTM-слоев – tanh-активацию для выхода. Схематичное изображение разработанной архитектуры представлено на Рис. 6.



Рис. 6 Архитектура разработанной нейросети для восстановления звука

Также для большей ясности приведем полные данные по слоям разработанной архитектуры. На рисунке 7 представлена структура нейросети

в архитектуре со слоем SimpleRNN, а на рисунке 8 структура по слоям в архитектуре со слоем LSTM.

Model: "SpectrogramModel"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 512)	0	-
stft (STFT)	[(None, 65, 9), (None, 65, 9)]	0	input_signal[0][...]
bidirectional (Bidirectional)	(None, 65, 512)	534,528	stft[0][0]
dropout (Dropout)	(None, 512, 32)	0	bidirectional[0]... simple_rnn_2[0][...]
bidirectional_1 (Bidirectional)	(None, 65, 512)	1,049,600	dropout[0][0]
dense (Dense)	(None, 65, 9)	4,617	bidirectional_1[...]
add (Add)	(None, 65, 9)	0	stft[0][0], dense[0][0]
istft (ISTFT)	(None, 512)	0	add[0][0], stft[0][1]
add_inner_dim_1 (AddInnerDim)	(None, 512, 1)	0	istft[0][0]
conv1d (Conv1D)	(None, 512, 32)	128	add_inner_dim_1[...]
simple_rnn_2 (SimpleRNN)	(None, 512, 32)	2,080	conv1d[0][0]
simple_rnn_3 (SimpleRNN)	(None, 512, 256)	73,984	dropout[1][0]
dense_1 (Dense)	(None, 512, 256)	65,792	simple_rnn_3[0][...]
conv1d_1 (Conv1D)	(None, 512, 1)	769	dense_1[0][0]
squeeze_1 (Squeeze)	(None, 512)	0	conv1d_1[0][0]
add_1 (Add)	(None, 512)	0	input_signal[0][...] squeeze_1[0][0]

Total params: 1,731,498 (6.61 MB)

Trainable params: 1,731,498 (6.61 MB)

Non-trainable params: 0 (0.00 B)

Рис. 7 Структура нейросети в архитектуре со слоем SimpleRNN

Model: "SpectrogramModel"

Layer (type)	Output Shape	Param #	Connected to
input_signal (InputLayer)	(None, 512)	0	-
stft (STFT)	[(None, 65, 9), (None, 65, 9)]	0	input_signal[0][...]
bidirectional (Bidirectional)	(None, 65, 512)	2,138,112	stft[0][0]
dropout (Dropout)	(None, 512, 32)	0	bidirectional[0]... lstm_2[0][0]
bidirectional_1 (Bidirectional)	(None, 65, 512)	4,198,400	dropout[0][0]
dense (Dense)	(None, 65, 9)	4,617	bidirectional_1[...]
add (Add)	(None, 65, 9)	0	stft[0][0], dense[0][0]
istft (ISTFT)	(None, 512)	0	add[0][0], stft[0][1]
add_inner_dim_1 (AddInnerDim)	(None, 512, 1)	0	istft[0][0]
conv1d (Conv1D)	(None, 512, 32)	128	add_inner_dim_1[...]
lstm_2 (LSTM)	(None, 512, 32)	8,320	conv1d[0][0]
lstm_3 (LSTM)	(None, 512, 256)	295,936	dropout[1][0]
dense_1 (Dense)	(None, 512, 256)	65,792	lstm_3[0][0]
conv1d_1 (Conv1D)	(None, 512, 1)	769	dense_1[0][0]
squeeze_1 (Squeeze)	(None, 512)	0	conv1d_1[0][0]
add_1 (Add)	(None, 512)	0	input_signal[0][...] squeeze_1[0][0]

Total params: 6,712,074 (25.60 MB)

Trainable params: 6,712,074 (25.60 MB)

Non-trainable params: 0 (0.00 B)

Рис. 8 Структура нейросети в архитектуре со слоем LSTM

Красный блок на рисунке 6 обозначает часть архитектуры, связанную с восстановлением сигнала в частотной области. Соответственно, к сигналу применяется оконное дискретное преобразование Фурье и осуществляется восстановление полученной спектрограммы. Напомним, что восстановление производится только для амплитудной части получаемой спектрограммы, и

при обратном преобразовании применяется фазовая часть спектрограммы исходного сигнала в неизменном виде (параметры дискретного оконного преобразования Фурье, применяемого к исходному сигналу более подробно описаны в главе 3.2). Отметим, что обрабатываемый блок исходного сигнала при частоте 44,1 кГц соответствует всего $\frac{512}{44100} \approx 11,61$ мс исходного сигнала.

Теперь скажем несколько слов об устройстве двунаправленного рекуррентного блока исходной нейросети. Основа данного блока – рекуррентный слой из 512 нейронов, возвращающий последовательности (параметр `return_sequences=True`). Двунаправленная версия рекуррентного блока состоит из двух таких слоев, только один слой применяется к исходной последовательности семплов сигнала, другой слой применяется к семплам исходного сигнала, идущим в обратном порядке. Полученные результаты объединяются в одну последовательность тем или иным образом. Для реализации данного фреймворка мы используем вызов `Keras tf.keras.layers.Bidirectional` для объявленного рекуррентного слоя. Код этого блока в объявлении нейросети выглядит следующим образом:

```
#прототипы используемых рекуррентных слоев, преобразуют спектрограммы
сигнала
self.rnn = self.rnn_layer(units=sq_length, activation=self.activation,
return_sequences=True)
.....
self.rnn1 = tf.keras.layers.Bidirectional(self.rnn,merge_mode='ave')
#двунаправленный слой с усреднением результатов работы в двух
направлениях
```

Здесь `merge_mode` определяет режим объединения последовательностей. В приведенном куске кода `merge_mode='ave'`, что означает усреднение результатов применения прямого и обратного рекуррентных слоев. На рисунке 4 «x2» означает двухкратное применение данного преобразования к получаемой спектрограмме. Его же можно увидеть на рисунках 7 и 8 в виде слоев с названиями `bidirectional` и `bidirectional_1`. Еще раз напомним, что получаемая после дискретного оконного преобразования Фурье спектрограмма представляет собой двумерный массив размерности 65 × 9.

Также отметим, что рекуррентный слой представляет собой функциональную зависимость вида $f(s, x)$, где s обозначает внутреннее состояние рекуррентного слоя, определяемое предыдущими входными данными этого слоя, а x – текущий входной вектор рекуррентного слоя. В то же время типовой слой нейросетей представляет собой функциональную зависимость вида $f(x)$, т.е. его выход после обучения всегда будет одинаков для одинакового входного вектора. В Keras рекуррентные слои рассматривают предпоследнее измерение входного вектора как измерение времени. Это означает, что рекуррентный слой будет обрабатывать получаемую на вход амплитудную spectrogramму входа как набор из 65 последовательных векторов в пространстве $\mathbb{R}^9$, интерпретируя каждый такой вектор из 9 отсчетов как входной вектор в данный момент времени. Заметим, что порядок измерений в данном случае имеет решающее значение, поскольку измерения spectrogramмы неоднородны: одно измерение spectrogramмы выражает частоту, а другое время. В нашем случае, измерение времени spectrogramмы будет интерпретироваться как измерение времени рекуррентным слоем, что является залогом эффективной обработки spectrogramм. Размерность вывода рекуррентного слоя равна количеству нейронов в данном слое. Таким образом, входная амплитудная spectrogramма после применения к ней рекуррентных слоев будет преобразована в двумерный массив отсчетов размерности 65×512 (см. форму вывода слоя `bidirectional` на рисунках 7 и 8). Поскольку эти данные не соответствуют размеру исходной амплитудной spectrogramмы, нужно произвести преобразование, устраняющее введенную в данные избыточность. Для этого применяется полносвязный слой из 9 нейронов (слой `dense` на рисунках 7 и 8), устраняющий избыточность и возвращающий двумерный массив действительных отсчетов размерности 65×9 . Далее полученный двумерный массив складывается с амплитудной spectrogramмой для исходной сигнала (слой `add` на рисунках 7 и 8). Это нужно для того, чтобы обеспечить максимальную точность восстановления «корректных» отсчетов spectrogramмы. Как мы видим, наш алгоритм восстановления амплитудной

спектрограммы содержит в себе множество сложно связанных друг с другом слоев, из-за чего очевидно функциональная зависимость входа и выхода становится нелинейной. В то же время вполне логично предположить, что искажения спектрограммы, вызванные клиппингом затрагивают только часть отсчетов исходного сигнала. Если мы не будем использовать обратную аддитивную связь в представленном алгоритме, фактически мы будем пытаться воспроизвести линейную зависимость $f(x) = x$ с помощью нелинейной зависимости входа и выхода для неискаженных отсчетов спектрограммы. В случае добавления обратной аддитивной связи эта зависимость превращается в зависимость $f(x) = 0$, которой представленный набор слоев достаточно легко можно научить, применяя либо ReLU-активацию (что мы делаем в варианте архитектуры со слоями SimpleRNN), либо механизм внимания (что мы делаем в варианте архитектуры со слоями LSTM).

На этом рассмотрение части алгоритма, связанной с обработкой спектрограммы исходного сигнала будем считать завершенным. Перейдем к рассмотрению следующей части алгоритма, которая выражает преобразование сигнала во временной области. На рисунке 6 эта часть алгоритма находится в зеленой рамке. Сразу представим набор объявлений слоев данного блока в исходном коде разработанных скриптов:

```
#слои, используемые для преобразования сигнала во временной области
self.convout = tf.keras.layers.Conv1D(filters=32, kernel_size=3,
activation='relu', padding='same') #сверточный слой с relu-активацией,
32 фильтра
self.rnnout = self.rnn_layer(units=32, activation=self.activation,
return_sequences=True) #рекуррентный слой из 32 нейронов
self.rnnout2 = self.rnn_layer(units=sq_length//2,
activation=self.activation, return_sequences=True) #рекуррентный слой
из sq_length//2 нейронов
self.denseout = tf.keras.layers.Dense(units=sq_length//2,
activation='linear') #полносвязный слой из sq_length//2 нейронов
self.convout2 = tf.keras.layers.Conv1D(filters=1, kernel_size=3,
activation='linear', padding='same') #сверточный слой для устранения
искажений во временной области
```

Первый из слоев, которые мы рассмотрим – сверточный слой, сохраняемый в поле `self.convout` (слой `conv1d` на рисунках 7 и 8). Как можно понять из объявления, данный слой имеет длину равную трем и состоит из 32 сверточных фильтров. Поскольку данный слой интерпретирует последнее измерение входного вектора как количество каналов входа, перед его применением к входным данным мы производим дополнительное добавление измерения к входному вектору (иными словами, размерность входа изменяется следующим образом $batch_size \times 512 \rightarrow batch_size \times 512 \times 1$). Соответственно, обработку входных данных этим слоем мы можем интерпретировать как применение к входу 32 различных фильтров и объединение всех результатов в массив размерности $batch_size \times 512 \times 32$, где последнее измерение выражает индекс примененного к входным данным фильтра. Следующий слой, сохраняемый в поле `self.rnnout` (слой `simple_rnn_2` на рисунке 7 или слой `lstm_2` на рисунке 8), состоит из 32 нейронов. Напомним, что рекуррентный слой в Keras интерпретирует предпоследнее измерение входного вектора как измерение времени отсчетов. Таким образом, форма вектора после применения данного слоя останется равной $batch_size \times 512 \times 32$. После данного слоя мы применяем второй рекуррентный и полносвязный слои из 256 нейронов (слои `self.rnnout2` и `self.denseout`, соответственно на рисунке 7 `self.rnnout2` это слой `simple_rnn_3`, а на рисунке 8 это слой `lstm_3`, тогда как слой `self.denseout` это слой `dense_1` на обоих рисунках), чтобы получить более сложную зависимость входного сигнала от выходного. Чтобы устранить возникшую избыточность обрабатываемых нейросетью данных, мы используем сверточный слой из одного фильтра, приводящий количество отсчетов к количеству отсчетов в исходных данных. После данного преобразования размерность обрабатываемых данных становится равной $batch_size \times 512 \times 1$. Для устранения последнего измерения мы используем вызов Tensorflow `tf.squeeze`. Заметим, что в слоях `self.denseout` и `self.convout2` (слой `conv1d_1` на рисунках 7 и 8) не используется активация (`activation='linear'`, что эквивалентно отсутствию слоя активации).

Это связано с тем, что данные слои осуществляют переход от внутренних представлений исходного сигнала обратно к тому виду, в котором представлен исходный сигнал. Использование активации в данных слоях может привести к искажению выходных последовательностей (самый простой пример – ReLU-активация, которая устраняет отрицательные значения из сигнала, что приведет к занулению отрицательной части колебаний в получаемом результате, тогда как исходный сигнал представляет собой звук в знаковой РСМ и содержит как отрицательную, так и положительную составляющую колебаний). Далее производится поэлементное сложение получаемой временной последовательности с семплами исходного сигнала с помощью обратной связи (слой `add_1` на рисунках 7 и 8). Цель введения данной обратной связи приблизительно аналогична цели ее введения в блоке частотного восстановления: изменить функциональную зависимость, которую сложно воспроизвести с помощью большого количества слоев. На этом будем считать рассмотрение временной части архитектуры нейросети завершенным.

Все предыдущее повествование строилось вокруг рассмотрения абстрактных рекуррентных слоев без конкретизации их архитектуры. Чтобы устранить данный пробел, расскажем немного о двух вариантах архитектуры рекуррентных слоев, которые мы будем использовать, и об их отличиях друг от друга. Напомним, что мы предлагаем архитектуру разработанной нейросети в двух вариантах – с использованием простых рекуррентных слоев SimpleRNN и длинных цепей элементов краткосрочной памяти LSTM. Как можно понять из названия, первый вариант рекуррентного слоя имплементирует «классический» вариант архитектуры рекуррентного слоя. Простой рекуррентный слой можно описать следующим набором формул:

$$s_t = f_1(W^{sx} x_t + W^{ss} s_{t-1} + W_0^{ss}) \quad (3.15)$$

$$y_t = s_t \quad (3.16)$$

Здесь:

f_1 – внутренняя функция активации рекуррентного слоя

x_t – входной вектор рекуррентного слоя в момент времени t ($x_t \in \mathbb{R}^{l \times 1}$)

s_t – состояние рекуррентного слоя в момент времени t ($s_t \in \mathbb{R}^{m \times 1}$)

y_t – выходной вектор рекуррентного слоя в момент времени t ($y_t \in \mathbb{R}^{m \times 1}$)

W^{sx}, W^{ss}, W_0^{ss} – матрицы весов нейросети ($W^{sx} \in \mathbb{R}^{m \times l}, W^{ss} \in \mathbb{R}^{m \times m}, W_0^{ss} \in \mathbb{R}^{m \times 1}$)

Таким образом, здесь формула 3.15 выражает то, как изменяется состояние рекуррентного слоя, а формула 3.16 выражает связь между текущим состоянием рекуррентного слоя и выходным вектором нейросети. Как видно из приведенных формул, данный слой реализует относительно несложную функциональную зависимость входа и выхода, возвращая свое внутреннее состояние s_t в качестве выходного вектора и содержит всего три матрицы весов. При этом в качестве f_1 в нашем случае используется ReLU (параметр activation), и слой возвращает свое внутреннее состояние в качестве выхода (параметр return_sequences=True). Поскольку данный слой возвращает свое состояние в качестве выходного вектора, размерность последнего измерения выходного вектора данного слоя совпадает с размерностью его состояния (в пояснениях к формулам 3.15 и 3.16 эта размерность обозначена буквой m). Основной проблемой SimpleRNN слоя является проблема исчезающих градиентов, приводящая к «забыванию» слоем отдаленных от текущего момента времени событий.

Теперь скажем несколько слов о втором варианте архитектуры – LSTM. Данная архитектура была разработана с целью решения проблемы исчезающих градиентов при обратном распространении ошибки во времени. Это достигается благодаря использованию так называемых «вентилей» - специальных функциональных блоков рекуррентной нейронной сети, которые используются для контроля потоков информации на входах и на выходах памяти данных блоков. Архитектура простого LSTM блока представлена на рисунке 9.

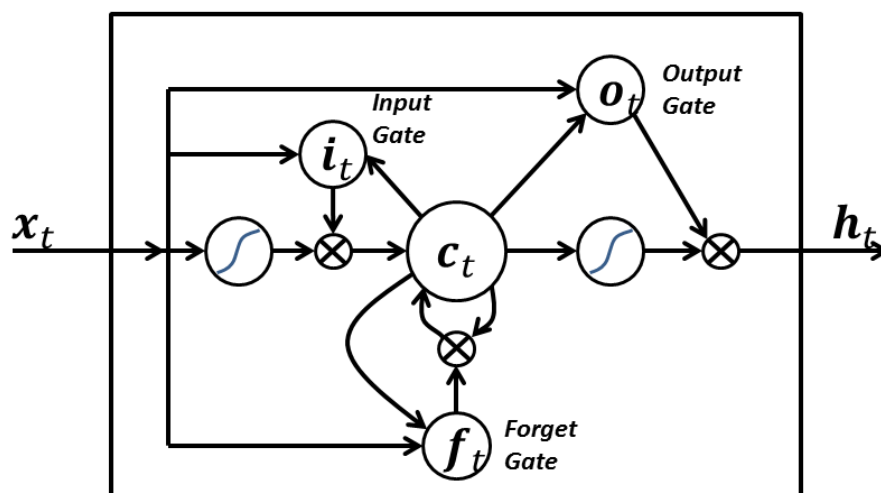


Рис. 9 Архитектура простого LSTM-блока [21]

Представленная архитектура содержит в себе 3 вентиля: вентиль входа, выхода, и забывания. Каждый из вентиля возвращает вектор, поэлементное перемножение на который используется для частичного допуска или запрещения потока информации внутрь и наружу памяти. Отметим, что каждый вентиль содержит свой набор весов, определяющий его выход на основании входного вектора и вектора состояний. Таким образом, данный слой содержит кратно большее число весов чем SimpleRNN слой (чтобы убедиться в этом, достаточно взглянуть на рисунки 7 и 8: версия архитектуры со слоем SimpleRNN содержит примерно 1,7 миллиона весов, тогда как архитектура со слоем LSTM содержит примерно 6,7 миллионов) из-за чего обучение с его применением как правило оказывается более ресурсозатратным. При этом данный слой как правило обеспечивает существенный прирост качества получаемых результатов относительно SimpleRNN слоя. Это достигается благодаря тому, что механизм вентиля позволяет эффективно обрабатывать и прогнозировать временные ряды в случаях, когда важные события разделены временными лагами с неопределенной продолжительностью и границами.

Последней темой, которую мы рассмотрим в данной главе, является само обучение разработанной нейросети. Для решения данной проблемы был разработан скрипт, запускающий обучение разработанной нейросети с заданным набором параметров. Имя файла данного скрипта – script.py (код

этого скрипта можно найти в приложениях к работе). Данный скрипт принимает следующий набор аргументов (аргументы перечислены в порядке, соответствующем порядку их передачи разработанному скрипту):

- имя WAV-файла с неискаженной звуковой дорожкой (файл обязательно должен содержать RIFF 64 контейнер);
- имя WAV-файла с искаженной звуковой дорожкой (файл обязательно должен содержать RIFF 64 контейнер);
- размер пачки, используемой для обучения нейросети;
- версия архитектуры нейросети, два возможных значения параметра: 0 – SimpleRNN архитектура, 1 – LSTM архитектура;

Эталонные параметры обучения, которые использовались нами для обучения нейросети с разработанной архитектурой следующие:

- размер пачки – 512 последовательностей;
- размер кэша – 20 пачек;
- перемешивание последовательностей и пачек – включено;
- функция потерь – среднеквадратичное отклонение последовательностей во временной области (MSE);
- оптимизатор (алгоритм, используемый для минимизации функции потерь) – AdamW с шагом обучения (learning rate), равным 10^{-4} и угасанием весов (weight decay) равным 0.01;

AdamW – версия стандартного алгоритма стохастического градиентного спуска Adam с использованием так называемого «разъединенного угасания весов» вместо L2-регуляризации, используемой алгоритмом Adam [22]. Данная модификация позволяет достигать лучшего уровня производительности модели за меньшее число эпох обучения [22].

Процесс обучения длился приблизительно 25 эпох для каждого из двух вариантов архитектуры, одна эпоха обучения для версии с SimpleRNN слоем занимала приблизительно 1 час 20 минут, для версии с LSTM слоем – приблизительно 2 часа. Обучение производилось с использованием версии

Tensorflow для GPU на видеокарте NVIDIA GeForce RTX 2080 Ti. Операционная система, используемая на машине, которая применялась для обучения – Debian 12. Скрипт для обучения запускался в виртуальном окружении Python. Все модули и их версии сохранены в файле requirements.txt (приведен в приложении к работе). Чтобы установить соответствующие версии модулей, необходимых для работы скриптов в виртуальном окружении нужно сделать вызов `pip install -r requirements.txt`.

3.4. Восстановление звуковых дорожек с помощью разработанной нейросети

В данной главе мы рассмотрим скрипт для восстановления звуковой дорожки с помощью разработанной нейросети. Имя данного скрипта – `inference_batch_prediction.py` (код скрипта можно найти в приложениях к данной работе). Сам разработанный скрипт принимает следующий набор аргументов:

- имя сохраненного файла разработанной нейросети (расширение файла – `.keras`);
- имя восстанавливаемого WAV-файла;
- путь к WAV-файлу, куда будет сохранен результат обучения;
- относительный порог фиксации клиппинга α (действительное число в интервале $(0,1)$, реализация аналогична реализации данного порога в программе для интерполяции кубическими сплайнами);
- T – абсолютный порог маскирования спектрограммы при постобработке восстановленного сигнала (действительно значение, если значение $T < 10^{-4}$ маскирование спектрограммы не производится);

Обсудим детали имплементации алгоритма восстановления звука с помощью разработанной нейросети. Приблизительный алгоритм этого восстановления выглядит следующим образом:

1. чтение массива нормированных семплов входного файла;

2. разбиение прочитанного массива на последовательности, длины которых соответствуют размерности входа разработанной нейросети;
3. восстановление сразу всего массива входных последовательностей с помощью вызова Keras `predict_on_batch`, восстанавливающего сразу целую пачку входных данных;
4. маскирование амплитудной спектрограммы всей входной звуковой дорожки (с порогом маскирования – T , имплементация маскирования будет подробно описана далее), если оно требуется;
5. маскирование полученных результатов во временной области (с относительным порогом фиксации клиппинга α);
6. дополнение результатов семплами исходной дорожки до количества семплов во входном файле;
7. запись полученных результатов в WAV-файл;

Поясним все вышеперечисленные этапы алгоритма восстановления звуковой дорожки. Первым важным этапом данного алгоритма является второй этап – разбиение прочитанного массива на последовательности. Экспериментально было выяснено, что точность восстановления пиков исходной звуковой дорожки выше в середине каждой из исходных последовательностей (возможно, в силу краевых эффектов при применении сверточных слоев и дискретного оконного преобразования Фурье). Поэтому исходная прочитанная звуковая дорожка (чтение осуществляется с помощью функции `read_wav_as_float`, нормирующей прочитанную последовательность семплов) разбивается на последовательности из 512 семплов, идущие друг за другом с шагом в 256 семплов. Иными словами, если первый семпл первой последовательности имеет нулевой индекс, индексы ее семплов – с нулевого по 511-ый включительно, тогда как индексы следующей сразу за данной последовательности – с 256-го по 767-ой включительно. После данного разбиения производится восстановление полученного массива последовательностей с помощью загруженной в память модели через вызов Keras `predict_on_batch`. Далее производится маскирование амплитудной

спектрограммы полученной звуковой дорожки. Это связано с тем, что сгенерированный нейросетью восстановленный сигнал содержит вычислительный шум, созданный нейросетью. Один из типовых способов шумоподавления звуковых сигналов – маскирование (зануление) значений меньше установленного порога в амплитудной спектрограмме. Рассмотрим подробнее, как это делается. Сначала полученный на предыдущем этапе массив последовательностей приводится к виду одномерного массива семплов. Далее к этому массиву семплов применяется оконное дискретное преобразование Фурье со следующими параметрами:

- окно – Ханна (Хэннинга) с центрированием;
- размер окна R (и используемого ДПФ) – 256 отсчетов (эквивалентно $\frac{256}{44100} \approx 5,80$ мс аудиодорожки);
- шаг (сдвиг) окна L – 128 отсчетов (эквивалентно $\frac{128}{44100} \approx 2,90$ мс аудиодорожки);

Данное преобразование имплементировано в скрипте с помощью вызова Tensorflow `tf.signal.stft`. Полученный комплексный спектр исходного сигнала разбивается на амплитудную и фазовую составляющие. После этого производится зануление отсчетов амплитудной спектрограммы, значение которых меньше значения T с объединением амплитудной и фазовой частей спектрограммы в исходную комплексную спектрограмму. Все описанные действия начиная с разбиения комплексной спектрограммы на амплитудную и фазовую части производятся внутри функции `mask_stft_amplitude`, объявленной внутри модуля базовых объявлений `base.py`. После этого производится обратное оконное дискретное преобразование Фурье, восстанавливающее временное представление сигнала. Отметим, что отношение $\frac{R}{L} = 2$, а используемое окно в преобразовании – окно Ханна с центрированием. Как уже было доказано в главе 3.2, данные параметры дискретного оконного преобразования Фурье обеспечивают идеальную

инвертируемость полученной спектрограммы. Поскольку после маскирования мы получаем одномерный массив отсчетов сигнала во временной области, с помощью вызова `reshape` мы приводим одномерный массив обратно к двумерному массиву последовательностей по 512 семплов. При этом если переданное значение порога маскирования было меньше 10^{-4} , маскирование не производится и все перечисленные выше шаги преобразования пропускаются.

Следующим этапом восстановления также является маскирование, но уже во временной области. Напомним, что имеется ввиду. Предположим, мы имеем два массива, x и y , одинаковой длины N , а также относительный порог фиксации клиппинга α . Пусть массив x – массив искаженных семплов исходного сигнала, а y – массив результатов восстановления массива нейросетью. Очевидно, что массив x содержит два вида семплов: искаженные клиппингом и неискаженные. Значения искаженных клиппингом семплов будут приблизительно равняться уровню отсечения сигнала клиппингом. Нетрудно заметить, что значения семплов массива искаженных семплов x будут ограничены уровнем клиппинга. Поэтому для выделения частей исходного массива x , содержащих клиппинг мы просто найдем абсолютный максимум значений семплов исходного массива x и будем фиксировать клиппинг в массиве x если абсолютное значение семпла приблизительно равняется абсолютному максимуму значений. Разброс значений, для которых будет фиксироваться клиппинг как раз и определяется относительным порогом фиксации клиппинга α . Теперь переформулируем описанное условие на языке математики:

$$abs(x[i]) \geq \alpha \cdot max(abs(x)) \quad (3.17)$$

Здесь:

$x[i]$ – i -ый семпл исходного сигнала x

α – относительный порог фиксации клиппинга

$abs(\cdot)$ – абсолютное значение, если аргумент – действительное число, возвращает его модуль, если аргумент – вектор, возвращает поэлементное абсолютное значение данного вектора

$max(\cdot)$ – нахождение максимального значения вектора

Условие 3.17 проверяется для каждого элемента вектора x . Результаты возвращаются в виде маски – там, где условие выполняется значение маски равно 1, там же, где оно не выполняется – значение маски равно 0. После вычисления маски маскирование результатов производится по следующей формуле:

$$(1 - mask) \odot x + mask \odot y \quad (3.18)$$

Здесь:

$mask$ – вычисленная по формуле 3.17 маска

x – исходный вектор искаженных семплов

y – вектор восстановленных нейросетью семплов

$\odot$ – поэлементное перемножение двух векторов

$+$ – поэлементное сложение элементов векторов

$-$ – поэлементное вычитание элементов векторов (если один операнд число, а другой – вектор, поэлементное вычитание числа из каждого элемента вектора, или каждого элемента вектора из числа с получением результирующего вектора)

Маскирование, как оно описано формулами 3.17 и 3.18 имплементировано в функции `threshold_mask` скрипта `base.py`, содержащего базовые объявления, необходимые для обучения нейросети и восстановления с помощью нее.

Следующий этап восстановления, о котором нужно сказать – дополнение выходного массива до длины входного. Почему вообще возникает такая необходимость? Вспомним про то, что мы используем как восстановленные семплы только семплы из середины накладывающихся последовательностей, обработанных нейросетью. Таким образом, в восстановленной дорожке не хватает 128 (четверть длины одной последовательности) семплов в начале. Также в восстановленное дорожке

может не хватать какого-то количества семплов в конце, поскольку если последней последовательности в разбиении входного массива отсчетов на последовательности не хватает отсчетов, данная последовательность отбрасывается. С помощью двух последовательных вызовов `append` производится дополнение длины восстановленной дорожки семплами исходной дорожки таким образом, чтобы длины исходного и восстановленного файлов в точности совпадали друг с другом. Отметим, что данный процесс несущественно ухудшает качество восстановленного звука, поскольку независимо от длины входной дорожки, при дополнении придется вставить в восстановленную дорожку не более 639 (128 семплов в начале и 511 в конце) семплов, что составляет всего $\frac{639}{44100} \approx 14.49$ миллисекунд звука при частоте дискретизации в 44.1 кГц.

Напомним, что разработанная нами модель для интерполяции существует в двух вариантах: с архитектурой на основе SimpleRNN слоя и с архитектурой на основе LSTM слоя. Обе версии модели вместе с их архитектурой были сохранены как файлы с расширением `.keras`. Скрипт восстановления корректно загружает модели из файлов для обоих представленных типов архитектуры. Сохраненная в виде файла модель в версии архитектуры с SimpleRNN слоем занимает примерно 20 мегабайт памяти, тогда как в версии архитектуры с LSTM слоем – примерно 80 мегабайт памяти.

3.5. Выводы

В данном разделе были получены следующие результаты:

1. Предложен способ формирования обучающей выборки для модели компенсации клиппинга, разработан скрипт, формирующий такую выборку на основе заданных параметров.
2. Предложена оригинальная архитектура нейронной сети для интерполяции цифрового звука, разработаны скрипты для обучения модели и сохранения результатов обучения на диск.

3. Предложен способ восстановления звука с использованием нейросети и улучшения качества получаемых результатов за счет постобработки, разработан скрипт, осуществляющий нахождение областей клиппинга в исходном звуковом файле, их восстановление, и запись полученных результатов в файл.

Эффективность разработанной архитектуры нейросети для восстановления звука будет численно оценена и сопоставлена с эффективностью интерполяции кубическими сплайнами как метода восстановления звука в следующем разделе данной работы.

4. АНАЛИЗ ЭФФЕКТИВНОСТИ РАЗРАБОТАННЫХ МЕТОДОВ НА РЕАЛЬНЫХ ДАННЫХ

В данном разделе будет проведен статистический анализ результатов восстановления с помощью двух методов восстановления дорожек, содержащих клиппинг: с использованием интерполяции кубическими сплайнами, и с использованием разработанной нейросети. Также будет произведено сравнение производительности разных архитектур разработанной нейросети и разных значений порогов маскирования и даны конкретные рекомендации по улучшению получаемых результатов.

4.1. Методы оценки эффективности восстановления цифрового звука

Вообще говоря, существует два подхода к оценке качества восстановленного звука, и соответствующие им два набора показателей – показатели качества на основе «эталонного» результата и без него. Первый подход подразумевает, что исходными данными алгоритма оценки качества восстановления будут являться сразу два звуковых файла: первый из них – «эталонный» файл (ожидаемый результат восстановления, который явился бы результатом алгоритма в случае, если сам алгоритм обеспечивал идеальное восстановление), второй – восстановленный файл, представляющий собой результаты алгоритма восстановления звука. Другой подход принимает как исходные данные только искаженный звуковой файл без необходимости получения эталонного файла. Каждый подход обладает своим набором плюсов и минусов: если первый подход более точен и надежен, второй подход более универсален. Поскольку искажения, рассматриваемые в данной работе, легко воспроизводятся любым звуковым редактором для любых исходных данных, нет необходимости ограничиваться вторым подходом, когда можно получить максимально точные и надежные результаты с помощью первого. Именно поэтому все показатели качества восстановления, вокруг которых будет строиться дальнейшее повествование будут опираться на некоторый «эталонный» результат.

Для оценки базового набора показателей (тех показателей, вычисление которых представляется относительно простым и прямолинейным) нами была разработана программа на языке Си. Данная программа принимает на вход 2 WAV-файла и вычисляет набор показателей, которые выражают степень ошибки искаженного файла относительно эталонного. Программа поддерживает до 4 каналов WAV-аудио с PCM модуляцией (WAV-аудио с PCM-модуляцией – формат данных, который обрабатывают оба представленных нами решения для восстановления звука, искаженного клиппингом). При работе с многоканальным аудио, набор показателей программой вычисляется для каждого канала в отдельности. В данном разделе работы будут рассмотрены основные показатели, вычисление которых реализовано в разработанной программе. Отметим, что позднее, при анализе результатов восстановления, в дополнение к набору показателей, которые рассчитывает разработанная программа, мы будем также пользоваться показателем PESQ как более подходящим показателем качества восприятия восстановления речи человеческим ухом. Итак, перечислим набор показателей, вычисляемых разработанной программой.

Первый такой показатель - среднеквадратичная ошибка (англ. MSE, Mean Square Error). Для семплов звукового файла данный показатель может быть вычислен по следующей формуле:

$$MSE = \sqrt{\frac{\sum_{i=0}^{N-1} (\hat{y}_i - y_i)^2}{N}} \quad (4.1)$$

Здесь, соответственно, $\hat{y}_i$ – семплы восстановленного звукового файла, y_i – семплы оригинального звукового файла, N – количество семплов звуковых файлов.

Заметим, что данная и все последующие формулы в этом разделе подразумевает одинаковое количество семплов в восстановленном и оригинальном звуковых файлах. Важным недостатком показателя, вычисляемого по формуле 4.1 является то, что он показывает абсолютный

уровень шума, никак не соотнося его с мощностью сигнала: при большей мощности сигнала аналогичный уровень шума будет вносить меньший негативный эффект, что данным показателем не учитывается.

Следующий показатель, вычисляемый разработанной программой, менее «наивен», т.к. учитывает относительный уровень шума, а не абсолютный, на что намекает его название – отношение сигнала к шуму (англ. SNR, Signal-to-Noise Ratio). Вычисляется он по следующей формуле:

$$\text{SNR} = 10 \log_{10} \frac{\sum_{i=0}^{N-1} y_i^2}{\sum_{i=0}^{N-1} (\hat{y}_i - y_i)^2} \quad (4.2)$$

Как и во всех предыдущих формулах, здесь: $\hat{y}_i$ – семплы восстановленного звукового файла, y_i – семплы оригинального звукового файла, N – количество семплов звуковых файлов. Однако в отличие от MSE, значение, вычисляемое по формуле 4.2 выражается не в абсолютных единицах, а в децибелах.

Заметим, что изменение громкости оригинального файла также будет приводить к уменьшению получаемого по формуле 4.2 значения SNR. Очевидно, это является нежелательным эффектом, поскольку изменение громкости не приводит к какой-либо потере звуковой информации (оговоримся, что это происходит при не слишком большом изменении громкости, которое не приводит к возникновению клиппинга или занулению большого числа семплов исходного файла при целочисленном делении). Чтобы избавиться от этого эффекта, была разработана модификация данного показателя, называемая «масштабонезависимое отношение сигнала к шуму» (англ. SI-SNR, Scale-Invariant Signal-to-Noise Ratio), рассчитываемое по следующей формуле:

$$\text{SI-SNR} = 10 \log_{10} \frac{\sum_{i=0}^{N-1} (\alpha y_i)^2}{\sum_{i=0}^{N-1} (\alpha y_i - \hat{y}_i)^2} \quad (4.3)$$

Здесь: $\alpha = \frac{\sum_{i=0}^{N-1} \hat{y}_i y_i}{\sum_{i=0}^{N-1} y_i^2}$ – коэффициент приведения масштаба, $\hat{y}_i$ – семплы восстановленного звукового файла, y_i – семплы оригинального звукового файла, N – количество семплов звуковых файлов.

Результат, получаемый согласно формуле 4.3 не зависит от громкости звука [23]. Попробуем это доказать. Пусть громкость восстановленного звука была изменена в k раз. Тогда $\hat{y}_i = k \cdot (y_i + \delta_i)$, где δ_i – небольшая ($\forall i \delta_i \ll y_i$) аддитивная составляющая шума восстановленного звукового файла.

Следовательно,

$$\alpha = \frac{\sum_{i=0}^{N-1} \hat{y}_i \cdot y_i}{\sum_{i=0}^{N-1} y_i^2} = \frac{\sum_{i=0}^{N-1} k \cdot (y_i + \delta_i) \cdot y_i}{\sum_{i=0}^{N-1} y_i^2} = k \frac{\sum_{i=0}^{N-1} (y_i^2 + \delta_i \cdot y_i)}{\sum_{i=0}^{N-1} y_i^2} = k \frac{\sum_{i=0}^{N-1} y_i^2 + \sum_{i=0}^{N-1} \delta_i \cdot y_i}{\sum_{i=0}^{N-1} y_i^2} \quad (4.4)$$

Т.к. $\forall i \delta_i \ll y_i$, $\frac{\sum_{i=0}^{N-1} y_i^2 + \sum_{i=0}^{N-1} \delta_i \cdot y_i}{\sum_{i=0}^{N-1} y_i^2} \approx 1$. Следовательно, из формулы 4.4 получаем, что $\alpha \approx k$. Тогда:

$$SI - SNR = 10 \log_{10} \frac{\sum_{i=0}^{N-1} (\alpha y_i)^2}{\sum_{i=0}^{N-1} (\alpha y_i - \hat{y}_i)^2} = 10 \log_{10} \frac{\sum_{i=0}^{N-1} (k y_i)^2}{\sum_{i=0}^{N-1} (k y_i - k \cdot (y_i + \delta_i))^2} \quad (4.5)$$

Преобразуя знаменатель в формуле 4.5, получаем $\sum_{i=0}^{N-1} (k y_i - k \cdot (y_i + \delta_i))^2 = \sum_{i=0}^{N-1} (k y_i - k y_i - k \delta_i)^2 = \sum_{i=0}^{N-1} (-k \delta_i)^2 = k^2 \cdot \sum_{i=0}^{N-1} \delta_i^2$. Таким образом, $SI - SNR = 10 \log_{10} \frac{k^2 \cdot \sum_{i=0}^{N-1} y_i^2}{k^2 \cdot \sum_{i=0}^{N-1} \delta_i^2} = 10 \log_{10} \frac{\sum_{i=0}^{N-1} y_i^2}{\sum_{i=0}^{N-1} \delta_i^2}$.

Как мы видим, в итоговом результате мы получили отношение суммы квадратов значений семплов сигнала к сумме квадратов аддитивной ошибки, то есть ожидаемое для не масштабированного сигнала значение SNR, что доказывает корректность приведенной формулы. Отметим, что в силу того, что коэффициент приведения масштаба α в том числе учитывает сами шумы звукового файла как отклонения масштаба (хотя при условии, что энергия шума не так велика на фоне энергии сигнала эти отклонения незначительны) мы можем гарантировать лишь приблизительное равенство $\alpha \approx 1$ даже когда изменения громкости вовсе не было. Именно это вызывает обычно незначительное отклонение показателя SI-SNR от SNR для идентичных входных данных.

Еще один показатель, вычисляемый разработанной программой это пиковое отношение сигнал-шум (англ. PSNR, Peak Signal-to-Noise Ratio). Нетрудно догадаться по названию, что данный показатель отражает

отношение максимума амплитуды сигнала к шуму на логарифмической шкале, иными словами, может быть вычислен по следующей формуле:

$$\text{PSNR} = 10 \log_{10} \frac{y_{\max}^2}{\sum_{i=0}^{N-1} \frac{(\hat{y}_i - y_i)^2}{N}} \quad (4.6)$$

Здесь: $y_{\max}$ – максимальное абсолютное (под максимальным абсолютным здесь подразумевается наибольшее значение по модулю) значение семпла звукового файла, $\hat{y}_i$ – семплы восстановленного звукового файла, y_i – семплы оригинального звукового файла, N – количество семплов звуковых файлов. Отметим, что формула 4.6 чаще всего используется для оценки качества изображений, нежели для звуковой информации.

Зачастую более информативным показателем по сравнению с показателями, опирающимися на временное представление сигналов, является их сравнение в частотной области. Одним из таких показателей, который вычисляется разработанной программой, является логарифмическое спектральное расстояние (англ. LSD, Log-Spectral Distance). Данный показатель вычисляется по следующей формуле:

$$\text{LSD} = \left\{ \frac{1}{N} \sum_{f=1}^N \left(10 \log_{10} \frac{|X(f)|^2}{|\hat{X}(f)|^2} \right)^p \right\}^{\frac{1}{p}} \quad (4.7)$$

Здесь:

X – результат ДПФ исходного (эталонного) набора семплов

$\hat{X}$ – результат ДПФ восстановленного набора семплов

p – некоторое натуральное число, определяющее норму, с помощью которой выражается показатель

N – количество семплов звуковых файлов

$|\cdot|$ – модуль комплексного числа (в данном случае, комплексных отсчетов ДПФ исходного и восстановленного сигналов)

В разработанной программе используется версия формулы 4.7 для $p = 2$, а для вычисления спектра сигнала используется быстрое преобразование Фурье исходного сигнала для длин исходных массивов, равных степени двойки.

Соответственно, перед вычислением БПФ массив дополняется нулями до ближайшей длины, выражаемой степенью двойки. Результатом данного дополнения в частотной области является интерполированный спектр более сглаженной формы [24], который и используется для вычисления логарифмического спектрального расстояния. Отметим, что дополнение нулями не влияет на вычислительную сложность алгоритма вычисления БПФ. Действительно, $O(2N \log 2N) = O(2N \cdot (\log N + c)) = 2O(N \log N) + O(2c N) = O(N \log N)$.

Таким образом, разработанная программа имплементирует вычисление пяти различных показателей, которые заданы формулами 4.1, 4.2, 4.3, 4.6, 4.7. Заметим, что значения показателей SNR и PSNR зависят от порядка переданных файлов для сравнения (иными словами, значения показателей меняются если передать «восстановленный» файл как «эталонный», а «эталонный» как «восстановленный»), тогда как значения MSE, SI-SNR и LSD – не зависят. Также отметим, что большим значениям показателей MSE и LSD соответствует больший уровень искажений, тогда как большим значениям показателей SNR, SI-SNR и PSNR наоборот меньший. Это важно учитывать при численном анализе получаемых результатов.

Ну и скажем пару слов о самой программе. Программа работает с интерфейсом командной строки и принимает на вход 2 аргумента: имена/путь к эталонному и искаженному файлам. После расчета перечисленных показателей для 2 переданных файлов программа выводит в консоли полученные значения показателей. Напомним, что программа поддерживает до 4 каналов WAV-аудио с 8,16,32-битной PCM-модуляцией и рассчитывает весь набор показателей для каждого канала аудио в отдельности. Программа использует тот же код для чтения данных и заголовка WAV-файла, что и программа для интерполяции кубическими сплайнами. Напомним, что связанная с чтением WAV-файлов с PCM-модуляцией часть кода была вынесена в заголовочный файл «wave.h» (код данного заголовочного файла приведен в приложениях к работе), используемый обеими разработанными

программами. На рисунке 10 приведен пример вывода разработанной программы для расчета показателей качества восстановления.

```
C:\Users\abcde\Downloads\CubicSplinesInterp>compare 1.wav output12_1.wav
Original file: C:\Users\abcde\Downloads\CubicSplinesInterp\1.wav
Clipped file: C:\Users\abcde\Downloads\CubicSplinesInterp\output12_1.wav
-----Channel 1-----
MSE = 0.000238
SNR = 18.233547 dB
SI-SNR = 18.512531 dB
PSNR = 35.441933 dB
LSD = 14.668330 dB

C:\Users\abcde\Downloads\CubicSplinesInterp>
```

Рис. 10 Вывод разработанной программы оценки качества восстановления звука

Все показатели, которые мы перечислили выше, имеют один существенный недостаток – они оценивают скорее объективную численную точность воспроизведения семплов «эталонного» звукового файла в «восстановленном», а не субъективное восприятие человеческим ухом искажений относительно эталонного звукового файла. Чтобы получить более информативные результаты, естественным решением будет использовать показатель, отражающий субъективное восприятие качества звука человеческим ухом. В качестве такого показателя в представленной работе мы будем использовать показатель PESQ (англ. Perceptual Evaluation of Speech Quality – воспринимаемая оценка качества речи).

PESQ – показатель, разработанный специально для оценки качества передачи речи телефонными сетями и различными кодеками. Данный показатель был стандартизован сектором стандартизации электросвязи Международного союза электросвязи в 2001 году. Сам показатель был разработан на основе двух других алгоритмов, ранее хорошо себя зарекомендовавших в ходе экспериментов. В общих чертах рассмотрим, как производится вычисление данного показателя. Функциональная схема вычисления PESQ представлена на рисунке 11.

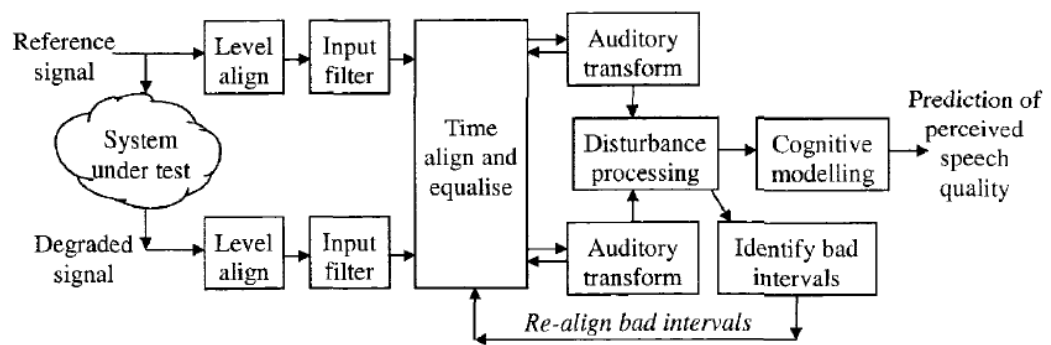


Рис. 11 Функциональная схема вычисления PESQ [25]

Первым шагом данной схемы является приведение уровня громкости сигналов к стандартному уровню громкости (блок «level align» на рисунке 11). Результат фильтруется с помощью входного фильтра, моделирующего стандартную телефонную трубку («input filter» на рисунке 11). Далее производится выравнивание входных сигналов во времени (в нашем случае этот шаг не требуется, так как между входом и выходом отсутствует временная задержка). Следующий этап – слуховое преобразование (блок «auditory transform» на рисунке 11). Слуховое преобразование – психоакустическая модель, которая сопоставляет сигналу во временной области представление воспринимаемой громкости в зависимости от времени и частоты [25]. Далее происходит обработка помех и моделирование восприятия. В рамках этих этапов обработки звуковая дорожка «искаженного» сигнала декомпозируется на ряд спектральных компонент, для каждой из которых оценивается пороговое значение уровня шума. Далее производится маскирование шума ниже этого порогового значения, квантование и кодирование полученных спектральных составляющих, и наконец вычисление ошибки между полученными спектральными составляющими. Результирующее воспринимаемое качество звука выражают по шкале MOS (англ. Mean Opinion Score – средний балл мнения). Для стандартных входных данных значение MOS варьируется в диапазоне от 1 (плохое качество) до 4,5 (нет искажений) [25]. При экстремально сильных искажениях MOS может падать ниже единицы, но это большая редкость. Показатель PESQ можно применять как для узкой полосы исходных сигналов (англ. «narrowband»), так и для широкой

полосы (англ. «wideband»). Узкополосная версия показателя использует только узкую низкочастотную полосу сигнала от 0 до примерно 3,5 кГц для сравнения, тогда как широкополосная версия использует всю доступную с учетом частоты дискретизации полосу. Для большинства практических приложений, не использующих узкополосные системы связи рекомендуется использовать широкополосную версию данного показателя. Существенно так же и то, что данный показатель можно использовать только для сравнения сигналов с частотой дискретизации не более 16 кГц (если исходные звуковые сигналы записывались с большей частотой дискретизации единственным выходом из ситуации становится их передискретизация). Так как мы будем использовать разработанные решения для восстановления звука с частотой дискретизации 44.1 кГц, нам потребуется передискретизация исходных звуковых дорожек. Таким образом, мы будем сравнивать только частотную полосу исходного сигнала от 0 до 8 кГц, т.е. частотная полоса от 8 до 22,5 кГц исходного сигнала будет отброшена при передискретизации, однако, при сравнении речевых сигналов это несущественно, поскольку в человеческой речи вклад составляющих выше 8 кГц несущественен (большинство формант, определяющих разборчивость человеческой речи расположены в полосе частот от 300 до 3400 Гц[26]).

4.2. Статистический анализ разработанных методов восстановления звука

В данном подразделе представленной работы будет производиться сравнение имплементированных методов интерполяции цифрового звука. Чтобы обеспечить корректность получаемых результатов, была собрана выборка тестовых фрагментов человеческого голоса с разными тембрами, записанных в разных студиях с использованием различного оборудования. Естественно, фрагменты звука, используемые для оценки качества работы представленных методов никак не пересекаются с обучающей выборкой разработанной нейросети. При этом в самих фрагментах используется клиппинг глубиной от 5 до 15 децибел (что соответствует уровням клиппинга,

использовавшимся в обучающей выборке нейросети). В качестве основных показателей качества получаемых результатов мы будем использовать рассмотренные ранее показатели SI-SNR и PESQ.

В ходе предварительных экспериментов были подобраны два оптимальных относительных порога фиксации клиппинга, обеспечивающие наилучшие результаты восстановления. Это значения $\alpha = 0,95$ и $\alpha = 0,99$. Данные значения параметра α будут использоваться для проведения статистической оценки качества восстановления разработанных методов. В случае относительного порога маскирования спектрограммы итогового сигнала ситуация более сложная. Это связано с тем, что оптимальный уровень маскирования спектрограммы варьируется в зависимости от глубины клиппинга. Действительно, большие значения T могут привести к маскированию большого количества отсчетов спектрограммы сигнала, что приведет к «упрощению» формы волны. При относительно небольшой глубине клиппинга длина самой области клиппинга как правило небольшая, и такое «упрощение» может быть оправданным, тогда как при большой глубине клиппинга это может привести к потере некоторых «деталей» восстановленного сигнала, важных в первую для восприятия результата человеческим ухом. Данный вопрос является достаточно сложным и может представлять интерес для отдельного детального исследования. В представленной работе в качестве компромиссного варианта мы будем использовать значение $T = 0,5$, выбранное в ходе предварительных экспериментов. Мы убедимся, что данное значение является неплохим начальным выбором, который обеспечивает улучшение результатов восстановления в большинстве случаев (относительно случая, когда маскирование спектрограмм при восстановлении не применяется).

В таблице 1 представлен показатель SI-SNR для интерполяции кубическими сплайнами с значениями $\alpha = 0,95$ и $\alpha = 0,99$ для множества выборок цифрового звука.

Таблица 1 Показатель SI-SNR интерполяции кубическими сплайнами

	мин.	средн.	макс.
ориг.	8,03	12,70	20,87
$\alpha = 0,95$	14,78	23,10	34,42
$\alpha = 0,99$	15,98	23,88	35,65

В таблице 2 представлен показатель PESQ для интерполяции кубическими сплайнами с значениями $\alpha = 0,95$ и $\alpha = 0,99$ для множества выборок цифрового звука.

Таблица 2 Показатель PESQ интерполяции кубическими сплайнами

	мин.	средн.	макс.
ориг.	1,40	1,96	3,03
$\alpha = 0,95$	2,49	3,32	4,11
$\alpha = 0,99$	2,48	3,38	4,15

Результаты, представленные в Таблице 1 и Таблице 2 демонстрируют эффективность исследуемого метода и оценивают в среднем его эффективность как прибавку порядка 1,4 баллов показателя PESQ и порядка 10 децибел показателя SI-SNR. Заметим, что ощутимого прироста воспринимаемого качества звука увеличение показателя α со значения 0,95 до значения 0,99 не дает.

Теперь изучим качество восстановления с использованием разработанной нейросети. Мы проведем анализ результатов для всех комбинаций следующих параметров восстановления: значение α – $\alpha = 0,95$ и $\alpha = 0,99$, с рекуррентными слоями SimpleRNN и LSTM, с маскированием спектрограмм восстановленного сигнала и без ($T = 0$ и $T = 0,5$). Полученные результаты представлены в таблицах 3,4,5 и 6.

Таблица 3 Показатель SI-SNR восстановления с использованием архитектуры SimpleRNN

	мин.	средн.	макс.
ориг.	8,03	12,70	20,87
$\alpha = 0,95 (T = 0)$	16,94	23,13	32,24
$\alpha = 0,99 (T = 0)$	17,00	23,16	32,27
$\alpha = 0,95 (T = 0,5)$	17,19	23,84	34,24
$\alpha = 0,99 (T = 0,5)$	17,26	23,91	34,35

Таблица 4 Показатель PESQ восстановления с использованием архитектуры SimpleRNN

	мин.	средн.	макс.
ориг.	1,40	1,96	3,03
$\alpha = 0,95 (T = 0)$	2,44	3,02	3,81
$\alpha = 0,99 (T = 0)$	2,41	3,02	3,81
$\alpha = 0,95 (T = 0,5)$	2,71	3,36	4,19
$\alpha = 0,99 (T = 0,5)$	2,66	3,34	4,17

Представленные в таблицах 3 и 4 результаты свидетельствуют о том, что восстановление с использованием архитектуры SimpleRNN обеспечивает примерно сопоставимое качество восстановления с интерполяцией кубическими сплайнами. Средний прирост показателя SI-SNR для восстановления без маскирования результирующей спектрограммы равен 10 децибелам, а для восстановления с маскированием результирующей спектрограммы – 11 децибелам. При этом прирост показателя PESQ несколько ниже – порядка 1 балла в случае без маскирования спектрограмм, и порядка 1,3 балла в случае маскирования спектрограмм.

Таблица 5 Показатель SI-SNR восстановления с использованием архитектуры LSTM

	мин.	средн.	макс.
ориг.	8,03	12,70	20,87
$\alpha = 0,95 (T = 0)$	17,20	23,58	30,42
$\alpha = 0,99 (T = 0)$	17,24	23,66	30,57
$\alpha = 0,95 (T = 0,5)$	17,41	24,36	31,53
$\alpha = 0,99 (T = 0,5)$	17,46	24,43	31,65

Таблица 6 Показатель PESQ восстановления с использованием архитектуры LSTM

	мин.	средн.	макс.
ориг.	1,40	1,96	3,03
$\alpha = 0,95 (T = 0)$	2,77	3,45	4,10
$\alpha = 0,99 (T = 0)$	2,76	3,46	4,13
$\alpha = 0,95 (T = 0,5)$	3,02	3,79	4,43
$\alpha = 0,99 (T = 0,5)$	2,99	3,77	4,42

Представленные в таблицах 5 и 6 результаты свидетельствуют о том, что эффективность описанного метода восстановления выше эффективности интерполяции кубическими сплайнами. Средний прирост показателя SI-SNR для восстановления без маскирования результирующей спектрограммы равен 11 децибелам, а для восстановления с маскированием результирующей спектрограммы – 12 децибелам. При этом прирост показателя PESQ следующий – порядка 1,5 балла в случае без маскирования спектрограмм, и порядка 1,8 балла в случае маскирования спектрограмм.

Таким образом, архитектура с использованием слоев SimpleRNN демонстрирует сопоставимые с интерполяцией кубическими сплайнами результаты, тогда как архитектура с LSTM слоями – существенно лучшие результаты. Причина этого косвенно упомянута во второй главе: принимая гипотезу о том, что форма сигнала в области клиппинга может быть выражена полиномом третьей степени, мы существенно ограничиваем количество

деталей в форме сигнала после восстановления. Проиллюстрировать это можно представленной на рисунке 12 формой восстановленных сигналов.

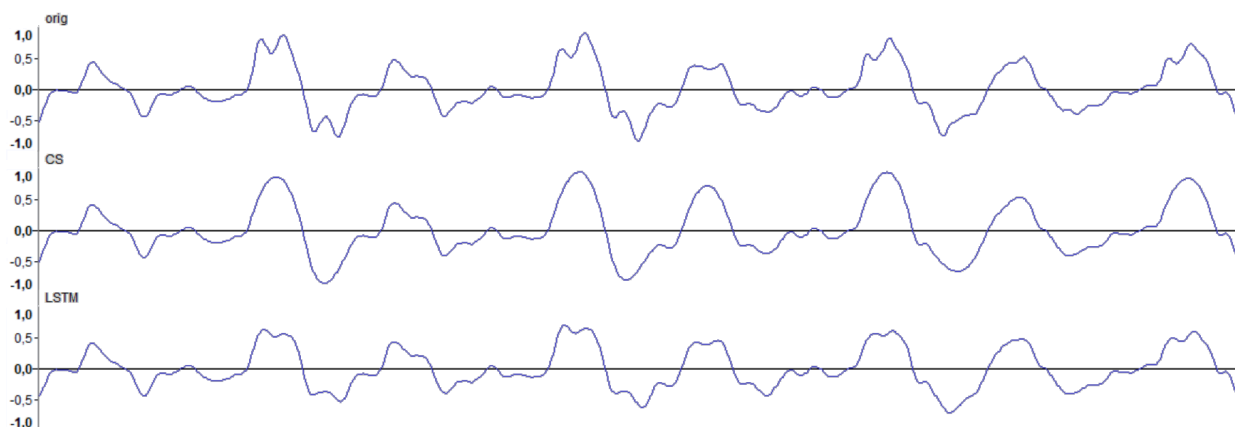


Рис. 12 Форма восстановленных сигналов

На рисунке 12 представлена форма сигнала, восстановленного двумя способами: с использованием интерполяции кубическими сплайнами и с использованием архитектуры LSTM (соответственно средняя и нижняя кривые, оригинальный сигнал представлен верхней кривой). Заметим, что восстановление с использованием нейросетей позволяет получать сложную форму восстановленного сигнала в области клиппинга, что является одной из возможных причин увеличения показателя PESQ восстановленного звука при использовании нейросетей относительно получаемых результатов при интерполяции кубическими сплайнами.

Подводя итог всему вышесказанному, рекомендуемым вариантом интерполяции является использование архитектуры с LSTM-слоем, значением $\alpha = 0,95$ и маскированием амплитудной части спектрограммы получаемого сигнала. В качестве порога маскирования как изначальное значение рекомендуется значение $T = 0,5$, но почти всегда возможно улучшение получаемых результатов через варьирование данного порогового значения.

4.3. Выводы

В данном разделе были получены следующие результаты:

1. Предложены методы оценки качества восстановления звука и разработана программа, реализующая вычисление данного набора показателей для звуковых файлов.

2. Произведен количественный анализ результатов восстановления с использованием интерполяции кубическими сплайнами и с использованием разработанной архитектуры нейросети.

Полученные результаты свидетельствуют о большей эффективности интерполяции с использованием разработанной архитектуры нейросети чем у классического метода интерполяции с использованием кубических сплайнов.

ЗАКЛЮЧЕНИЕ

Целью представленной работы было исследование и практическая имплементация эффективных методов компенсации нелинейных искажений цифрового звука с жестким ограничением сигнала по амплитуде, называемых клиппингом. Кратко перечислим полученные в данной работе результаты. В первом разделе сформирована необходимая теоретическая база: детально изучены основы цифровой обработки сигналов, определена специфика задачи компенсации клиппинга и проанализированы существующие решения (как классические, так и с использованием нейросетей). Во втором разделе реализован базовый алгоритм восстановления на основе кубической эрмитовой интерполяции: разработан метод обнаружения участков клиппинга с пороговым параметром α , выведена формула интерполяции и создана программа на языке Си, служащая эталоном для оценки последующих результатов. В третьем разделе предложена и внедрена собственная нейросетевая архитектура для восстановления аудиосигнала: разработаны инструменты автоматизированного формирования обучающей выборки и скрипты, обеспечивающие обучение модели, поиск искажённых фрагментов и их постобработку. Наконец, в четвертом разделе проведён сравнительный анализ результатов восстановления, в котором введён набор объективных метрик качества (MSE, SNR, LSD и др.) и реализована утилита для их вычисления. Эксперименты продемонстрировали, что нейросетевая модель стабильно превосходит классическую интерполяцию кубическими сплайнами, обеспечивая более качественное восстановление звука.

Таким образом, в данной работе продемонстрировано, что применение нейросетевых методов для компенсации клиппинга цифрового звука является более эффективным по сравнению с традиционными алгоритмами интерполяции. Полученные результаты обосновывают перспективность дальнейшей оптимизации нейросетевых архитектур и их адаптации к более сложным сценариям компенсации искажений, связанных с мягким и жестким ограничением сигнала по амплитуде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Айфичер, Э.С. Цифровая обработка сигналов: практический подход / Э.С. Айфичер, Б.У. Джервис. — 2-е изд. — М.: Вильямс, 2004. — 992 с.
2. Разделение в вопросе 18!!! // Файловый архив для студентов. StudFiles URL: <https://studfile.net/preview/9570047/page:3/> (дата обращения: 26.05.2025).
3. Сато, Ю. Без паники! Цифровая обработка сигналов / Ю. Сато. — пер. с яп. Селиной Т.Г. — М.: Додэка XXI, 2010. — 176 с.: ил.
4. Microsoft WAVE soundfile format. — Текст: электронный // Center for Computer Research in Music and Acoustics: [сайт]. — URL: <https://ccrma.stanford.edu/courses/422-winter-2014/projects/WaveFormat/> (дата обращения: 26.05.2025).
5. Лайонс Р. Цифровая обработка сигналов: Второе издание. Пер с англ. — 2-е изд. — М.: ООО «Бином-Пресс», 2006. — 656 с.
6. Сергиенко А. Б. Цифровая обработка сигналов: учеб. пособие. — 3-е изд. — СПб.: БХВ-Петербург, 2011. — 768 с.: ил.
7. Клиппинг (аудио) // Википедия URL: [https://ru.wikipedia.org/wiki/Клиппинг_\(аудио\)](https://ru.wikipedia.org/wiki/Клиппинг_(аудио)) (дата обращения: 26.05.2025).
8. Fanhu Bie, Dong Wang, Jun Wang [и др.] Detection and reconstruction of clipped speech for speaker recognition // Speech Communication. - 2015. - №72. - С. 218-231.
9. Hirofumi Nakajima, Kazuhiro Nakadai, Shigeki Miyabe [и др.] Restoration of clipped audio signal using recursive vector projection // TENCON 2011. - Bali: Electrical Engineering Department, Universitas Indonesia, 2011. - С. 787-790.
10. Kitić, S., Bertin, N., Gribonval, R. Sparsity and cosparsity for audio declipping: a flexible non-convex approach // Latent Variable Analysis and Signal Separation. - Liberec: Springer, 2015. - С. 243-250.

11. An Efficient Algorithm for Clipping Detection and Declipping Audio // Music Informatics Group URL: https://musicinformatics.gatech.edu/wp-content_nondefault/uploads/2016/09/Laguna_Lerch_2016_An-Efficient-Algorithm-For-Clipping-Detection-And-Declipping-Audio.pdf (дата обращения: 26.05.2025).
12. Johannes Imort, Giorgio Fabbro, Marco A. Martínez Ramírez [и др.] Distortion Audio Effects: Learning How to Recover the Clean Signal // Proceedings of the 23rd International Society for Music Information Retrieval Conference, ISMIR 2022. - 2022. - №1. - С. 218-225.
13. J. Yi, J. Koo and K. Lee DDD: A Perceptually Superior Low-Response-Time DNN-Based Declipper // ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). - Seoul: IEEE, 2024. - С. 801-805.
14. Сплайн // Википедия URL: <https://ru.wikipedia.org/wiki/Сплайн> (дата обращения: 26.05.2025).
15. RF64: An extended File Format for Audio // EBU Technology & Innovation URL: https://tech.ebu.ch/docs/tech/tech3306v1_0.pdf (дата обращения: 26.05.2025).
16. Mitra, Sanjit K. Digital Signal Processing: A Computer-Based Approach. - 2-е изд. - New York: McGraw-Hill, 2001. - 866 с.
17. Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows. // Fermilab | Holometer URL: https://holometer.fnal.gov/GH_FFT.pdf (дата обращения: 26.05.2025).
18. Invertibility of overlap-add processing // Learn Data Science URL: <https://gauss256.github.io/blog/cola.html> (дата обращения: 26.05.2025).
19. 3.5. Short-time processing of speech signals // Introduction to Speech Processing — Introduction to Speech Processing URL: https://speechprocessingbook.aalto.fi/Representations/Short-time_processing.html (дата обращения: 26.05.2025).

20. Meinard Müller Fundamentals of Music Processing Audio, Analysis, Algorithms, Applications. - 1-е изд. - Erlangen: Springer, 2015. - 487 с.
21. Долгая краткосрочная память // Википедия URL: https://ru.wikipedia.org/wiki/Долгая_краткосрочная_память (дата обращения: 26.05.2025).
22. DECOUPLED WEIGHT DECAY REGULARIZATION // arXiv.org e-Print archive URL: <https://arxiv.org/pdf/1711.05101> (дата обращения: 26.05.2025).
23. J. L. Roux, S. Wisdom, H. Erdogan [и др.] SDR – Half-baked or Well Done? // IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). - Brighton: IEEE, 2019. - С. 626-630.
24. ПРЕОБРАЗОВАНИЕ ФУРЬЕ И КЛАССИЧЕСКИЙ ЦИФРОВОЙ СПЕКТРАЛЬНЫЙ АНАЛИЗ. // Вибродиагностика URL: http://www.vibration.ru/preobraz_fur.shtml (дата обращения: 26.05.2025).
25. Rix, A.W., Beerends, J.G., Hollier, M.P. [и др.] Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs // 2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings. - 2001. - №2. - С. 749-752.
26. Частота голоса // Википедия URL: https://ru.wikipedia.org/wiki/Частота_голоса (дата обращения: 26.05.2025).

Приложение А. Задание на магистерскую диссертацию



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт	ИВТИ
Кафедра	ВМСС

**ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(МАГИСТЕРСКУЮ ДИССЕРТАЦИЮ)**

Направление	09.04.01 Информатика и вычислительная техника
	(код и наименование)

Образовательная программа	Вычислительные машины, комплексы, системы и сети
---------------------------	--

Форма обучения	очная
	(очная/очно-заочная/заочная)

Тема:	Исследование способов компенсации нелинейных искажений цифровых аудиозаписей
-------	--

Студент	А-07М-23	Винокуров Р.Н.
	группа	подпись
		фамилия и инициалы

Руководитель ВКР	К.Т.Н.	доцент	Гольцов А.Г.
	уч. степень	должность	подпись
			фамилия и инициалы

Заведующий кафедрой	К.Т.Н.	доцент	Вишняков С.В.
	уч. степень	звание	подпись
			фамилия и инициалы

Место выполнения работы	ФГБОУ ВО «НИУ «МЭИ»
-------------------------	---------------------

1. Обоснование выбора темы выпускной квалификационной работы

Клиппинг – эффект ограничения амплитуды сигнала при достижении максимально (или минимально) возможного ее значения. При возникновении данного эффекта часть переносимой сигналом информации безвозвратно теряется, поэтому универсального и полностью результативного (восстанавливающего исходный сигнал в точности таким какой он был до искажений) решения этой проблемы не существует (что, однако не мешает создать решения, обеспечивающие приемлемое качество восстановления для большинства случаев). Более того, в случае возникновения данного эффекта в цифровых аудиозаписях он приводит к нелинейным искажениям, очень явно различимым человеком на слух, что делает проблему устранения последствий клиппинга актуальной.

Для решения задачи интерполяции искаженного клиппингом звука в профессиональной и любительской звукозаписи необходимо исследовать и сравнить различные подходы к решению проблемы компенсации вызванных клиппингом нелинейных искажений. Исследование подразумевает создание программ и скриптов, решающих проблему восстановления звука одним из 3 следующих способов: с применением интерполяции кубическими сплайнами, с применением простой рекуррентной нейронной сети, с применением длинной цепи элементов краткосрочной памяти, а также сравнение результатов работы данных программ на тестовом наборе звуковых файлов. Для решения задачи интерполяции первым способом (с помощью кубических сплайнов) необходимо разработать приложение на компилируемом языке программирования, выявляющее области клиппинга в звуковом файле формата WAVE, интерполирующее эти области и сохраняющее полученный результат в звуковой файл формата WAVE. Для решения задачи интерполяции вторым и третьим способом подразумевается разработка двух скриптов на интерпретируемом языке программирования с применением библиотек и фреймворков для машинного обучения, которые принимают на вход либо два файла (искаженный и исходный) на основе которых формируется обучающая

выборка и происходит обучение нейросети, либо один файл, который требуется интерполировать. Чтобы решить задачу статистического исследования результатов необходимо разработать приложение на компилируемом языке программирования принимающее на вход два файла (исходный и восстановленный) и вычисляющее статистические показатели, выражающие степень искажения восстановленного файла (среднеквадратичное отклонение, пиковое отношение сигнал-шум).

Научный руководитель Гольцов А.Г. **дата** 21.11.2024

Студент Винокуров Р.Н. **дата** 21.11.2024

2. План выполнения выпускной квалификационной работы

№ п\п	Содержание разделов	Срок выпол- нения	Трудоём- кость, %
I.	Теоретическая часть		
	1) Обзор литературы	20.02.2025	20%
	2) Разработка алгоритмов интерполяции	02.03.2025	10%
	3) Подбор критериев статистического исследования	15.03.2025	10%
II.	Экспериментальная часть		
	1) Разработка и отладка программ и скриптов	06.04.2025	20%
	2) Подготовка тестовых примеров	18.04.2025	10%
	3) Сбор результатов работы программ	30.04.2025	10%
III.	Публикации	30.04.2025	0%

IV.	Оформление диссертации		
	1) Написание рукописи	10.05.2025	10%
	2) Вычитка текста рукописи	20.05.2025	5%
	3) Подготовка иллюстрирующих материалов: рисунки, таблицы	30.05.2025	5%

3. Рекомендуемая литература

1. Айфичер, Э.С. Цифровая обработка сигналов: практический подход / Э.С. Айфичер, Б.У. Джервис. — 2-е изд. — М. : Вильямс, 2004. — 992 с.
2. Microsoft WAVE soundfile format. — Текст : электронный // Center for Computer Research in Music and Acoustics : [сайт]. — URL: <https://ccrma.stanford.edu/courses/422-winter-2014/projects/WaveFormat/> (дата обращения: 27.10.2024).
3. Сергиенко А. Б. Цифровая обработка сигналов: учеб. пособие. — 3-е изд. — СПб.: БХВ-Петербург, 2011. — 768 с.: ил.
4. LSTM и GRU. — Текст : электронный // Хабр : [сайт]. — URL: <https://habr.com/ru/companies/mvideo/articles/780774/> (дата обращения: 27.10.2024).

Примечания:

1. Задание брошюруется вместе с выпускной работой после титульного листа (страницы задания имеют номера 2, 3, 4, 5).
2. Отзыв руководителя, рецензия(и), отчет о проверке на объем заимствований и согласие студента на размещение работы в открытом доступе вкладываются в конверт (файловую папку) под обложкой работы.

Приложение Б. Листинг кода разработанных программ и скриптов

Файл wave.h

```
#pragma once

#define TRUE 1
#define FALSE 0

//структура для чтения заголовка WAV-файла
#pragma pack(push,1) //отключить выравнивание полей структуры
struct HEADER {
    unsigned char riff[4]; //строка 'RIFF'
    unsigned int overall_size; //общий размер файла в байтах
    unsigned char wave[4]; //строка 'WAVE'
    unsigned char fmt_chunk_marker[4]; //строка 'fmt ' с завершающим нулевым символом
    unsigned int length_of_fmt; //размер чанка 'fmt'
    unsigned short format_type; //формат хранения аудиоданных: 1 - PCM,
    остальные значения подразумевают ту или иную форму сжатия
    unsigned short channels; //количество каналов аудио
    unsigned int sample_rate; //частота дискретизации файла (количество
    семплов в секунду, герцы)
    unsigned int byterate; //битрейт = sample_rate * channels *
    bits_per_sample / 8
    unsigned short block_align; //количество байт на семпл = channels *
    bits_per_sample / 8 // NumChannels * BitsPerSample/8
    unsigned short bits_per_sample; //количество бит на семпл
    unsigned char data_chunk_header[4]; //заголовок чанка данных
    unsigned int data_size; //размер чанка данных в байтах, равен
    num_samples * channels * bits_per_samples / 8, где num_samples - количество семплов аудио в
    этом чанке
};
#pragma pack(pop)

//функция чтения данных WAV-файла
//fp - файловый указатель WAV-файла (файл должен быть открыт для чтения в двоичном режиме)
//obj - указатель на структуру заголовка файла (используется для сохранения данных
прочитанного заголовка файла)
//возвращаемое значение - указатель на двумерный массив прочитанных из файла семплов
//((семплы нормируются на отрезке [-1,1], если в файле несколько каналов - каналы записываются
в различные массивы)
float** read_file_data(FILE* fp, struct HEADER* obj) {
    float** arrf = NULL; //если по той или иной причине прочесть данные не удалось,
    возвращаем NULL указатель
    int read = 0;
    //действительный диапазон значений амплитуды, вычисляется на основе количества бит
    на семпл в заголовке файла
    long long low_limit = 0;
    long long high_limit = 0;
    //в заголовке WAV-файла используются 2 и 4-байтные поля, объявляем переменные для их
    хранения
    unsigned char buffer4[4];
    unsigned char buffer2[2];

    read = fread(obj->riff, sizeof(obj->riff), 1, fp);
```

```

read = fread(buffer4, sizeof(buffer4), 1, fp);

//конвертировать 4-байтное целое little-endian значение в формат big-endian
obj->overall_size = buffer4[0] |
    (buffer4[1] << 8) |
    (buffer4[2] << 16) |
    (buffer4[3] << 24);

read = fread(obj->wave, sizeof(obj->wave), 1, fp);
read = fread(obj->fmt_chunk_marker, sizeof(obj->fmt_chunk_marker), 1, fp);
read = fread(buffer4, sizeof(buffer4), 1, fp);

//конвертировать 4-байтное целое little-endian значение в формат big-endian
obj->length_of_fmt = buffer4[0] |
    (buffer4[1] << 8) |
    (buffer4[2] << 16) |
    (buffer4[3] << 24);

read = fread(buffer2, sizeof(buffer2), 1, fp);
obj->format_type = buffer2[0] | (buffer2[1] << 8);
read = fread(buffer2, sizeof(buffer2), 1, fp);
obj->channels = buffer2[0] | (buffer2[1] << 8);
read = fread(buffer4, sizeof(buffer4), 1, fp);
obj->sample_rate = buffer4[0] |
    (buffer4[1] << 8) |
    (buffer4[2] << 16) |
    (buffer4[3] << 24);

read = fread(buffer4, sizeof(buffer4), 1, fp);
obj->byterate = buffer4[0] |
    (buffer4[1] << 8) |
    (buffer4[2] << 16) |
    (buffer4[3] << 24);

read = fread(buffer2, sizeof(buffer2), 1, fp);
obj->block_align = buffer2[0] |
    (buffer2[1] << 8);
read = fread(buffer2, sizeof(buffer2), 1, fp);
obj->bits_per_sample = buffer2[0] |
    (buffer2[1] << 8);

read = fread(obj->data_chunk_header, sizeof(obj->data_chunk_header), 1, fp);

read = fread(buffer4, sizeof(buffer4), 1, fp);

obj->data_size = buffer4[0] |
    (buffer4[1] << 8) |
    (buffer4[2] << 16) |
    (buffer4[3] << 24);

arrf = (float**)malloc(obj->channels * sizeof(float*));

```

```

//вычислить количество семплов звуковой дорожки
long num_samples = (8 * obj->data_size) / (obj->channels * obj->bits_per_sample);

long size_of_each_sample = (obj->channels * obj->bits_per_sample) / 8;

//вычислить продолжительность файла в секундах
float duration_in_seconds = (float)obj->overall_size / obj->byterate;
long bytes_in_each_channel = (size_of_each_sample / obj->channels);
if (obj->format_type == 1) { //если в файле используется модуляция, отличная от PCM,
    чтение данных не производится
    long i = 0;
    char* data_buffer = (char*)malloc(size_of_each_sample);
    int size_is_correct = TRUE;

    //убедиться что количество байт на семпл делится на число каналов без остатка
    if ((bytes_in_each_channel * obj->channels) != size_of_each_sample) {
        size_is_correct = FALSE;
    }

    if (size_is_correct) {

        switch (obj->bits_per_sample) {
        case 8:
            low_limit = -128;
            high_limit = 127;
            break;
        case 16:
            low_limit = -32768;
            high_limit = 32767;
            break;
        case 32:
            low_limit = -2147483648.0;
            high_limit = 2147483647;
            break;
        }
        //выделение памяти для всех семплов файла (с учетом каналов)
        for (int k = 0; k < obj->channels; k++)
            arrf[k] = (float*)malloc(num_samples * sizeof(float));
        for (i = 1; i <= num_samples; i++) {
            read = fread(data_buffer, size_of_each_sample, 1, fp);
            if (read == 1) {
                //извлечение значения семпла из прочитанных данных
                unsigned int xchannels = 0;
                int data_in_channel = 0;
                int offset = 0; //смещение относительно начала аудиоданных
                в файле
                for (xchannels = 0; xchannels < obj->channels; xchannels++) {
                    //конвертировать данные из формата little-endian в
                    формат big-endian, используя количество байт на семпл
                    if (bytes_in_each_channel == 4) {
                        data_in_channel = (data_buffer[offset] & 0x00ff)

```

```

|                                     ((data_buffer[offset + 1] & 0x00ff) << 8)
|
|                                     ((data_buffer[offset + 2] & 0x00ff) << 16)
|
|                                     (data_buffer[offset + 3] << 24);
|                                     }
|                                     else if (bytes_in_each_channel == 2) {
|                                         data_in_channel = (data_buffer[offset] & 0x00ff)
|
|                                         (data_buffer[offset + 1] << 8);
|                                     }
|                                     else if (bytes_in_each_channel == 1) {
|                                         data_in_channel = data_buffer[offset] & 0x00ff;
|                                         data_in_channel -= 128; //в формате WAVE 8-
битное аудио как правило без знака, смещаем до знакового
|                                     }
|
|                                     offset += bytes_in_each_channel;
|                                     arrf[xchannels][i - 1] = data_in_channel / fabsf(low_limit
* 1.0f);
|                                     if (arrf[xchannels][i - 1] > 1)
|                                     {
|                                         arrf[xchannels][i - 1] = 1;
|                                     }
|                                     else if (arrf[xchannels][i - 1] < -1)
|                                     {
|                                         arrf[xchannels][i - 1] = -1;
|                                     }
|                                     //проверяем, что прочитанное значение попадает в
диапазон представимых
|                                     if (data_in_channel < low_limit || data_in_channel >
high_limit)
|                                     printf("**value out of range \n");
|                                     }
|                                     }
|                                     else {
|                                         printf("Error reading file. %d bytes\n", read);
|                                         break;
|                                     }
|                                     }
|                                     free(data_buffer);
|                                     }
|                                     }
|                                     fclose(fp);
|                                     return arrf;
|                                     }

```

Файл restore.c

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <io.h>

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "wave.h" //заголовочный файл с объявлениями, нужными для работы с WAV-файлами

FILE* ptr;
FILE* fptr;
char* filename;
struct HEADER header;

#define SLOPE_LENGTH 4
//структура для хранения одного канала данных входного файла
typedef struct {
    float* samples;
    int length;
} AudioBuffer;

//параметры устранения клиппинга во входном файле
typedef struct {
    float threshold_ratio;
    float gain_linear;
    int buffer_size;
} ClipFixParams;

//прототипы функций, используемых программой, см. описание в месте объявления функций
void clipfix_process(AudioBuffer* audio, ClipFixParams* params, float peak_level);
void processBuffer(float* buffer, int bufferLength, float threshold);
void interpolate(float* buffer, int t0, int t1, int slopeLength);
float db_to_linear(float db);

int main(int argc, char** argv) {
    float** arrf = NULL; //массив семплов исходного файла
    filename = (char*)malloc(sizeof(char) * 1024);
    if (filename == NULL) {
        printf("Error in malloc\n");
        return -1;
    }

    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL) { //получить путь к текущей директории
        strcpy_s(filename, 1024, cwd);
        if (argc < 5) {
            printf("No enough arguments specified\n");
            return -2;
        }

        strcat_s(filename, 1024, "\\");
        strcat_s(filename, 1024, argv[1]); //получить имя файла из командной строки
    }

    fopen_s(&ptr, filename, "rb"); //открыть входной файл
    if (ptr == NULL) {
        printf("Error opening input file\n");
    }
}

```

```

        return -3;
    }

    arrf = read_file_data(ptr, &header);    //прочсть массив семплов из входного файла
    используя файловый указатель ptr
    if (arrf == NULL) {
        printf("Error reading input file\n");
        return -4;
    }

    //извлекаем используемые при обработке входных данных значения из заголовка
    входного файла
    long num_samples = (8 * header.data_size) / (header.channels * header.bits_per_sample);
    long bytes_in_each_channel = header.bits_per_sample / 8;
    long limit = 32768;    //абсолютный предел шкалы представимых значений
    switch (bytes_in_each_channel) {
        case 1:
            limit = 128;
            break;
        case 4:
            limit = 2147483648.0;
            break;
    }
    free(filename);
    //находим пиковые значения нормированных семплов отдельно для каждого канала
    (входной файл не должен содержать более 4 каналов)
    float mval[4] = { 0.0,0.0,0.0,0.0 };
    for (int chan = 0; chan < header.channels; chan++)
    {
        mval[chan] = arrf[chan][0];
        for (int i = 0; i < num_samples; i++) {
            if (fabs(arrf[chan][i]) > fabs(mval[chan]))
                mval[chan] = fabs(arrf[chan][i]);
        }
    }
    AudioBuffer mybuffer[4];
    ClipFixParams params;
    //извлекаем параметры восстановления звукового файла и записываем их в структуру
    params
    params.buffer_size = num_samples;
    params.gain_linear = db_to_linear(strtof(argv[4], NULL));
    params.threshold_ratio = strtoc(argv[3], NULL);
    //восстанавливаем каждый канал исходного аудио используя отдельный набор
    параметров
    for (int chan = 0; chan < header.channels; chan++)
    {
        mybuffer[chan].length = num_samples;
        mybuffer[chan].samples = arrf[chan];
        clipfix_process(&mybuffer[chan], &params, mval[chan]);
    }
    strcat_s(cwd, 1024, "\\");
    strcat_s(cwd, 1024, argv[2]);    //формирование пути к файлу для записи полученного
    результата

```



```

    _unlink(cwd);
    fopen_s(&fptr, cwd, "wb");
    if (fptr == NULL) {
        printf("Error opening output file\n");
        return -5;
    }
    fwrite(&header, (size_t)sizeof(header), 1, fptr); //записываем заголовок в результирующий
    файл
    int var;
    int value;
    //записать восстановленные двоичные данные в результирующий файл
    for (int j = 0; j < num_samples; j++)
    {
        for (int m = 0; m < header.channels; m++)
        {
            value = floor(arrf[m][j] * abs(limit));
            if (bytes_in_each_channel == 1)
                var = value & 0xff + 128;
            else if (bytes_in_each_channel == 2)
                var = (value & 0xff00) | (value & 0xff);
            else if (bytes_in_each_channel == 4)
                var = value;
            fwrite(&var, bytes_in_each_channel, 1, fptr);
        }
    }
    fclose(fptr);
    return 0;
}

//функция пересчета уровня громкости в дБ в относительное изменение громкости
//db - изменение громкости в децибелах
//возвращаемое значение - относительное изменение громкости соответствующее db децибел
float db_to_linear(float db) {
    return powf(10.0f, db / 20.0f);
}

//функция интерполяции кубическим эрмитовым сплайном отсчетов сигнала
//buffer - массив отсчетов интерполируемого сигнала
//t0 - индекс семпла начала отрезка, внутри которого будут интерполироваться значения
//t1 - индекс семпла конца отрезка, внутри которого будут интерполироваться значения
//иными словами, интерполируются семплы с индексами начиная с t0+1 до t1-1 включительно)
//dur - количество предшествующих семплов используемых для нахождения разностной
производной на каждом из концов отрезка
//возвращаемое значение - нет, результаты записываются в buffer
void interpolate(float* buffer, int t0, int t1, int dur) {
    float d0 = (buffer[t0] - buffer[t0 - dur]) / dur;
    float d1 = (buffer[t1 + dur] - buffer[t1]) / dur;
    float m = (d1 + d0) / ((t1 - t0) * (t1 - t0));
    float b = (d1 / (t1 - t0)) - (m * t1);

    for (int j = t0 + 1; j < t1; j++) {
        float term1 = (t1 - j) * (buffer[t0] / (t1 - t0));
        float term2 = (j - t0) * (buffer[t1] / (t1 - t0));
        float term3 = (j - t0) * (j - t1) * (m * j + b);
    }
}

```

```

        buffer[j] = term1 + term2 + term3;
    }
}
//функция восстановления множества интервалов клиппинга по заданным параметрам
//buffer - массив отсчетов интерполируемого сигнала
//buffer_length - длина массива buffer
//threshold - уровень фиксации клиппинга (уже пересчитанный с учетом относительного порога
фиксации клиппинга)
//возвращаемое значение - нет, результаты записываются в buffer
void process_buffer(float* buffer, int buffer_length, float threshold) {
    int* exit_list = NULL;
    int* return_list = NULL;
    int* exit_list_ptr = NULL;
    int* return_list_ptr = NULL;
    int exit_count = 0;
    int return_count = 0;
    const int last_sample = buffer_length - SLOPE_LENGTH; //если справа недостаточно семплов
- не удастся вычислить разностную производную
    //выявить превышения уровня клиппинга
    //exit_list_ptr - массив индексов начала последовательностей клиппинга
    //return_list_ptr - массив индексов конца последовательностей клиппинга
    for (int i = SLOPE_LENGTH; i < last_sample; i++) {
        if (fabsf(buffer[i]) >= threshold) {
            if (fabsf(buffer[i - 1]) < threshold) {
                if( (exit_list_ptr = realloc(exit_list_ptr, (exit_count + 1) * sizeof(int))) ==
NULL) {
                    printf("Error while reallocating in exit_list_ptr!\n");
                    free(exit_list_ptr);
                    exit(1);
                }
                exit_list_ptr[exit_count++] = i - 1;
            }
        }
        else {
            if (fabsf(buffer[i - 1]) >= threshold) {
                if( (return_list_ptr = realloc(return_list_ptr, (return_count + 1) *
sizeof(int))) == NULL) {
                    printf("Error while reallocating in return_list_ptr!\n");
                    free(return_list_ptr);
                    exit(1);
                }
                return_list_ptr[return_count++] = i;
            }
        }
    }
    return_list = return_list_ptr;
    exit_list = exit_list_ptr;
    //обработать граничный случай когда аудио начинается с клиппинга
    if (exit_count > 0 && return_count > 0 &&
        fabsf(buffer[SLOPE_LENGTH - 1]) >= threshold) {
        return_list++;
        return_count--;
    }
}

```

```

//процесс восстановления массива областей клиппинга
const int slope_len = SLOPE_LENGTH - 1;
for (int i = 0; i < exit_count && i < return_count; i++) {
    interpolate(buffer, exit_list[i], return_list[i], slope_len);
}
if (exit_list_ptr != NULL)
    free(exit_list_ptr);
if (return_list_ptr != NULL)
{
    free(return_list_ptr);
    return_list_ptr = NULL;
}
}

//процесс исправления клиппинга в буфере
//audio - буфер с восстанавливаемыми аудио данными
//params - параметры, используемые для восстановления исходных звуковых данных
//peak_level - абсолютный пик амплитуды звуковой дорожке, используется для нахождения
порога фиксации клиппинга
//возвращаемое значение - нет, изменяются семплы буфера audio
void clipfix_process(AudioBuffer* audio, ClipFixParams* params, float peak_level) {
    const float threshold = params->threshold_ratio * peak_level;
    const float gain = params->gain_linear;
    const int total_samples = audio->length;

    //обработать исходный массив как множество чанков
    for (int i = 0; i < total_samples; i += params->buffer_size) {
        int chunk_size = (i + params->buffer_size > total_samples)
            ? total_samples - i
            : params->buffer_size;
        process_buffer(audio->samples + i, chunk_size, threshold);
    }

    //применить изменение громкости (необходимо для предотвращения клиппинга при
восстановлении звука)
    for (int i = 0; i < total_samples; i++) {
        audio->samples[i] *= gain;
    }
}

```

Файл compare.c

```

#include <stdio.h>
#include <stdlib.h>
#define _USE_MATH_DEFINES
#include <math.h>
#include <io.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include "wave.h" //заголовочный файл с объявлениями, нужными для работы с WAV-файлами

```

```

//храним комплексные числа в виде действительной и мнимой частей в структуре
typedef struct {
    double Re;
    double Im;
} complex;

//функция проверки, является ли x степенью двойки
//x - целое число
//возвращаемое значение - булево, true, если x - степень двойки, false - в противном случае
bool IsPowerOfTwo(long x)
{
    return (x != 0) && ((x & (x - 1)) == 0);
}

//быстрое преобразование Фурье для длин массива, выражаемых степенью двойки
//v - комплексные отсчеты исходного сигнала
//n - количество комплексных отсчетов исходного сигнала
//возвращаемое значение - флаг ошибки при выполнении БПФ (неверная длина входного массива
- 1, иначе - 0)
int fft(complex* v, int n)
{
    //проверяем, является ли длина входного массива степенью двойки
    if (!IsPowerOfTwo(n))
        return 1;

    //выделяем память для результатов ДПФ (результат - массив комплексных значений
    //равный по длине входному массиву)
    complex* tmp = (complex*)malloc(n * sizeof(complex));
    if (n > 1) { //если массив не содержит хотя бы 2 элементов - искомое
        БПФ уже найдено, ничего не делаем
        int k, m;
        complex z, w, * vo, * ve;
        ve = tmp; vo = tmp + n / 2; //ve - указатель на начало первой половины tmp, vo - на
        начало второй половины tmp
        //кладем четные отсчеты v в ve, нечетные в vo
        for (k = 0; k < n / 2; k++) {
            ve[k] = v[2 * k];
            vo[k] = v[2 * k + 1];
        }
        fft(ve, n / 2); //БПФ для первой половины массива tmp (четных
        элементов)
        fft(vo, n / 2); //БПФ для второй половины массива tmp (нечетных
        элементов)
        //объединение БПФ для n/2 нечетных и n/2 четных отсчетов в БПФ для всей длины
        n
        //осуществляется по формулам:
        //X_m=E_m+e^{-2\pi i\frac{m}{N}}\cdot O_m
        //X_{m+\frac{N}{2}}=E_m-e^{-2\pi i\frac{m}{N}}\cdot O_m
        //здесь: O_m - ДПФ для элемента x_{2m+1}
        //E_m - ДПФ для элемента x_{2m}
        //m - целое число, такое что 0 <= m < N/2
        for (m = 0; m < n / 2; m++) {
            //находим e^{-2\pi i\frac{m}{N}} в виде действительной и мнимой частей,
            далее обозначим это значение как w

```

```

        w.Re = cos(2 * M_PI * m / (double)n);
        w.Im = sin(2 * M_PI * m / (double)n);
        z.Re = w.Re * vo[m].Re - w.Im * vo[m].Im;    //действительная часть w \cdot
O_m
        z.Im = w.Re * vo[m].Im + w.Im * vo[m].Re;    //мнимая часть w \cdot O_m
        v[m].Re = ve[m].Re + z.Re;    //действительная часть E_m + w \cdot
O_m
        v[m].Im = ve[m].Im + z.Im;    //мнимая часть E_m + w \cdot O_m
        v[m + n / 2].Re = ve[m].Re - z.Re;    //действительная часть E_m - w \cdot
O_m
        v[m + n / 2].Im = ve[m].Im - z.Im;    //мнимая часть E_m - w \cdot O_m
    }
}
free(tmp);
return 0;
}
//файловые указатели, строки имен и структуры заголовков для двух сравниваемых файлов
FILE* ptr1;
FILE* ptr2;
char* filename1;
char* filename2;
struct HEADER header1;
struct HEADER header2;
//функция сравнения заголовков обрабатываемых программой файлов
//hd1 - заголовок первого обрабатываемого файла
//hd2 - заголовок второго обрабатываемого файла
//возвращаемое значение - булево, true - заголовки совпадают, false - заголовки различаются
bool compare_headers(struct HEADER hd1, struct HEADER hd2) {
    //для числовых полей сравнение через ==, для строк - проверяем равенство нулю
    //результата strcmp
    bool res = (strcmp(hd1.riff, hd2.riff) == 0) && \
        (strcmp(hd1.wave, hd2.wave) == 0) && (strcmp(hd1.fmt_chunk_marker,
hd2.fmt_chunk_marker, 4) == 0) && \
        (hd1.length_of_fmt == hd2.length_of_fmt) && (hd1.format_type == hd2.format_type)
&& (hd1.channels == hd2.channels) && \
        (hd1.sample_rate == hd2.sample_rate) && (hd1.byterate == hd2.byterate) &&
(hd1.block_align == hd2.block_align) && \
        (hd1.bits_per_sample == hd2.bits_per_sample) && (strcmp(hd1.data_chunk_header,
hd2.data_chunk_header, 4) == 0) && (hd1.data_size == hd2.data_size);
    return res;
}

int main(int argc, char** argv) {
    //массивы для хранения семплов прочитанных файлов
    float** samples1 = NULL;
    float** samples2 = NULL;

    filename1 = (char*)malloc(sizeof(char) * 1024);
    filename2 = (char*)malloc(sizeof(char) * 1024);
    if (filename1 == NULL || filename2 == NULL) {
        printf("Error in malloc\n");
        exit(1);
    }
}

```

```

char cwd[1024];
if (getcwd(cwd, sizeof(cwd)) != NULL) { //получить путь к текущему рабочему каталогу

    strcpy_s(filename1, 1024, cwd);
    strcpy_s(filename2, 1024, cwd);
    if (argc < 3) {
        printf("No wave file specified\n");
        return -1;
    }
    //получить имена файлов из командной строки
    strcat_s(filename1, 1024, "\\");
    strcat_s(filename1, 1024, argv[1]);
    strcat_s(filename2, 1024, "\\");
    strcat_s(filename2, 1024, argv[2]);
    printf("Original file: %s\n", filename1);
    printf("Clipped file: %s\n", filename2);
}
//открыть исходные файлы в двоичном режиме чтения
fopen_s(&ptr1, filename1, "rb");
fopen_s(&ptr2, filename2, "rb");
if (ptr1 == NULL || ptr2 == NULL) {
    printf("Error opening file\n");
    return -2;
}
//samples1 - массив семплов оригинального файла, samples2 - искаженного
samples1 = read_file_data(ptr1, &header1);
samples2 = read_file_data(ptr2, &header2);
if (!compare_headers(header1, header2)) { //если заголовки не совпадают, сообщить о
проблеме и вывести дампы заголовков
    printf("Given files are not in the same format(new)!\n");
    printf("Header 1:\n\t");
    unsigned char* tmp = &header1;
    for (int i = 0; i < sizeof(header1); i++) {
        printf("%02X ", tmp[i]);
    }
    printf("\n");
    printf("Header 2:\n\t");
    tmp = &header2;
    for (int i = 0; i < sizeof(header2); i++) {
        printf("%02X ", tmp[i]);
    }
    printf("\n");
    return -3;
}
//значения MSE и максимальные по модулю значения семплов в файлах (отдельные для
каждого канала)
double* mse = (double*)malloc(header1.channels * sizeof(double));
double* mse1 = (double*)malloc(header1.channels * sizeof(double));
double* maxs = (double*)malloc(header1.channels * sizeof(double));
double* mse2 = (double*)malloc(header1.channels * sizeof(double));
if (mse == NULL || mse1 == NULL || mse2 == NULL || maxs == NULL) {
    printf("Error in malloc\n");
    return -4;
}

```

```

    }
    long num_samples = (8 * header1.data_size) / (header1.channels * header1.bits_per_sample);
    long real_num_samples = pow(2, ceil(log(num_samples) / log(2)));
    double norm = 0.0;
    double alpha = 0.0;
    //цикл нахождения заданного набора 5 показателей для каждого из каналов звуковых
    файлов
    for (int channel = 0; channel < header1.channels; channel++)
    {
        //нахождение коэффициента приведения масштаба alpha
        for (int i = 0; i < num_samples; i++)
            norm += samples1[channel][i] * samples1[channel][i];
        for (int i = 0; i < num_samples; i++)
            alpha += samples1[channel][i] * samples2[channel][i] / norm;
        mse[channel] = 0.0f;
        mse1[channel] = 0.0f;
        mse2[channel] = 0.0f;
        maxs[channel] = samples1[channel][0];
        //БПФ будет применяться к массивам семплов исходных файлов, дополненных
        нулевыми отсчетами до длины, выражаемой степенью двойки
        complex* FFT1 = (complex*)calloc(1, real_num_samples * sizeof(complex));
        complex* FFT2 = (complex*)calloc(1, real_num_samples * sizeof(complex));
        if (FFT1 == NULL || FFT2 == NULL)
        {
            printf("Error in malloc\n");
            return -5;
        }
        for (int i = 0; i < num_samples; i++)
        {
            FFT1[i].Re = (double)samples1[channel][i];
            FFT2[i].Re = (double)samples2[channel][i];
            if (fabs(samples1[channel][i]) > fabs(maxs[channel]))
                maxs[channel] = samples1[channel][i]; //максимальное значение
            семпла в каждом канале исходного файла
            mse1[channel] += pow(samples1[channel][i], 2) / num_samples;
            //среднеквадратичное значение семпла исходного файла
            mse[channel] += pow((samples1[channel][i] - samples2[channel][i]), 2) /
            num_samples; //MSE двух файлов
            mse2[channel] += pow((alpha*samples1[channel][i] - samples2[channel][i]), 2) /
            num_samples; //MSE двух файлов с поправкой
        }
        //находим БПФ двух исходных файлов, результаты используются для нахождения
        LSD
        fft(FFT1, real_num_samples);
        fft(FFT2, real_num_samples);
        double LSD = 0.0;
        //нахождение значения LSD^2
        for (int i = 0; i < real_num_samples; i++)
            LSD += 100/((double)real_num_samples * pow(log10((FFT1[i].Re * FFT1[i].Re +
            FFT1[i].Im * FFT1[i].Im) / (FFT2[i].Re * FFT2[i].Re + FFT2[i].Im * FFT2[i].Im)), 2);
        //нахождение SNR, SI-SNR, PSNR, LSD
        LSD = sqrt(LSD);
        double SDR = 10 * log10f(mse1[channel] / mse[channel]);
    }

```

```

double SISDR = 10 * log10f(alpha*alpha*mse1[channel] / mse2[channel]);
double PSNR = 10 * log10f(maxs[channel] * maxs[channel] / mse[channel]);
//вывод найденных значений для каждого канала с фиксированной точностью
вывода
printf("-----Channel %d-----\n",channel+1);
printf("MSE = %12.9lf\n", mse[channel]);
printf("SNR = %12.9lf dB\n", SDR);
printf("SI-SNR = %12.9lf dB\n", SISDR);
printf("PSNR = %12.9lf dB\n", PSNR);
printf("LSD = %12.9lf dB\n", LSD);
free(FFT1);
free(FFT2);
}
//очистка выделенных из кучи данных
for (int k = 0; k < header1.channels; k++)
    free(samples1[k]);
free(samples1);
for (int k = 0; k < header1.channels; k++)
    free(samples2[k]);
free(samples2);
free(mse);
free(mse1);
free(mse2);
free(maxs);
free(filename1);
free(filename2);
}

```

Файл rf64_clip.py

```

import struct
import numpy as np
import random
import sys

#функция пересчета уровня клиппинга в децибелах в относительный порог отсечения
#dbvalue - отрицательный уровень клиппинга в децибелах
#возвращаемое значение - относительный (т.е. значение лежит на полуотрезке (0,1] для
положительного dbvalue) порог клиппинга
def db_to_multiplier(dbvalue):
    return 10 ** (-dbvalue / 20.0)

#функция чтения заголовка чанка файла
#f - дескриптор открытого файла
#возвращаемое значение - кортеж из двух значений (tag,size), где tag - название чанка, size - его
размер в байтах
def read_chunk_header(f):
    hdr = f.read(8)
    if len(hdr) < 8:
        return None, None #если прочитано меньше 8 байт - чтение не удалось, возвращаем None
    #парсим прочитанные байты в значения переменных
    #'<4sL' - формат данных: < - порядок байтов little-endian, 4s - первые 4 прочитанных байта это
строка из 4 символов, L - последние 4 прочитанных байта это длинное целое значение без знака
    tag, size = struct.unpack('<4sL', hdr)

```



```

return tag, size
if __name__ == '__main__':
    #недостаточно аргументов для запуска скрипта - сообщение об ошибке
    if(len(sys.argv)<5):
        print('No enough arguments passed!')
    else:
        #пользовательские параметры
        source_file_path = sys.argv[1] #имя входного файла для внесения клиппинга
        destination_file_path = sys.argv[2] #имя выходного файла, в который будет записана дорожка
        после внесения клиппинга
        clip_thrsd_db_min = float(sys.argv[3]) #минимальный уровень клиппинга в децибелах
        clip_thrsd_db_max = float(sys.argv[4]) #максимальный уровень клиппинга в децибелах
        chunk_size = 44100 #длина блока, к которому применяется один уровень клиппинга

        #открыть входной файл и распарсить заголовок
        with open(source_file_path, 'rb') as fin:
            #прочитать первые 12 байт: подпись, размер файла, строку "WAVE"
            riff_header = fin.read(12)
            #не удалось прочитать заголовок - выбросить исключение
            if len(riff_header) < 12:
                raise ValueError("File too short")
            #прочитать подпись файла, если медиаконтейнер не RIFF 64 - выбросить исключение
            signature = riff_header[:4]
            if signature != b'RF64':
                raise ValueError("Not an RF64 file (signature = {}).format(signature))

            header_bytes = riff_header #мы соберем байты заголовка в этой переменной
            data_chunk_found = False
            data_chunk_size = None
            extended_data_size = None #в этой переменной будет храниться количество байт
            аудиоданных (считывается из ds64 чанка)

            #считывать чанки из файла пока не найдем чанк 'data'
            while not data_chunk_found:
                tag, size = read_chunk_header(fin)
                if tag is None:
                    raise ValueError("Reached end of file without finding data chunk")
                header_bytes += struct.pack('<4sL', tag, size)
                if tag == b'ds64':
                    ds64_data = fin.read(size)
                    header_bytes += ds64_data
                    #распаковать первые 24 байта: поля riffSize (размер блока RF64), dataSize (размер чанка
                    'data'), sampleCount (фактическое количество семплов); все поля 64-битные little-endian
                    if size >= 24:
                        riffSize_ext, data_size_ext, sample_count_ext = struct.unpack('<QQQ', ds64_data[:24])
                        extended_data_size = data_size_ext
                elif tag == b'fmt ':
                    fmt_data = fin.read(size)
                    header_bytes += fmt_data
                    sampwidth = struct.unpack('<H',fmt_data[14:16])[0]//8 #число байт на семпл
                    print("Sample width:",sampwidth)
                elif tag == b'data':
                    data_chunk_found = True

```

```

        data_chunk_size = size
        #достигли чанка 'data' - завершаем цикл
    else:
        #для любого другого чанка, скопируем его целиком
        chunk_data = fin.read(size)
        header_bytes += chunk_data

    if data_chunk_size is None:
        raise ValueError("Data chunk not found in source file")

    #сопоставить количеству бит на семпл тип элементов массива dtype
    if sampwidth == 2:
        dtype = np.int16
    elif sampwidth == 4:
        dtype = np.int32
    else:
        raise ValueError(f"Unsupported sample width: {sample_width}")

    #если 32-битный размер в чанке 'data' равен 0xFFFFFFFF в 16-ричной системе, использовать
    64-битный размер из чанка ds64
    if data_chunk_size == 0xFFFFFFFF and extended_data_size is not None:
        data_chunk_size = extended_data_size

    print("Header length:", len(header_bytes))
    print("Data chunk size (bytes):", data_chunk_size)

    #в этот момент, fin располагается в начале чанка data
    with open(destination_file_path, 'wb') as fout: #открыть файл результата в двоичном режиме
записи
        #записать заголовок в точности как он был прочтен
        fout.write(header_bytes)
        #обработать чанк данных по частям
        bytes_remaining = data_chunk_size

        #при PCM со знаком, максимальное по модулю значение семпла равно
        2^(sample_width*8-1)
        full_scale = 2**((sampwidth*8-1)

        while bytes_remaining > 0:
            dbval = random.uniform(clip_thrshd_db_min, clip_thrshd_db_max)
            clip_threshold = int(db_to_multiplier(dbval) * full_scale)
            #принудительное устанавливаем количество читаемых байт кратное числу байт на
семпл
            to_read = min(chunk_size * sampwidth, (bytes_remaining // sampwidth) * sampwidth)
            raw_chunk = fin.read(to_read)
            if not raw_chunk:
                break
            #конвертировать массив сырых байтов в массив NumPy
            samples = np.frombuffer(raw_chunk, dtype=dtype)
            #произвести клиппинг прочитанных данных
            clipped = np.clip(samples, -clip_threshold, clip_threshold)
            #записать обработанный чанк в выходной файл
            fout.write(clipped.tobytes())

```

```

        bytes_remaining -= len(raw_chunk)
    print("Processing complete. Processed file saved as:", destination_file_path)

```

Файл base.py

```

import sys
import contextlib
import wave_bwf_rf64 #модуль wave-bwf-rf64 для работы с медиаконтейнером RIFF 64
import os
from random import shuffle, seed
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0' #отключить кастомные OneDNN операции

import wave
import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import Sequence
from tensorflow.keras.callbacks import ModelCheckpoint
SQNC_LENGTH = 512 #длина входной и выходной последовательностей нейросети
FSTEP = 8 #размер шага при оконном преобразовании Фурье

#функция очищения комплексной спектрограммы звука
#stft_complex - комплексная спектрограмма звука
#T - порог значения амплитуды спектрограммы, меньшие значения амплитуды спектрограммы
будут маскироваться
#возвращаемое значение - маскированная комплексная спектрограмма звука той же формы как и
входная
def mask_stft_amplitude(stft_complex: tf.Tensor, T: float) -> tf.Tensor:
    #разделение комплексной входной спектрограммы на амплитудную и фазовую составляющие,
    состоящие из действительных значений
    amplitude = tf.abs(stft_complex)
    phase = tf.math.angle(stft_complex)

    #маскирование небольших амплитуд, если амплитуда отсчета < T, присвоить амплитуде отсчета
    значение 0
    masked_amp = tf.where(amplitude < T,
                          tf.zeros_like(amplitude),
                          amplitude)

    #восстановить действительную и мнимую части сигнала
    real_part = masked_amp * tf.cos(phase)
    imag_part = masked_amp * tf.sin(phase)

    #воссоздать комплексный тензор аналогичного размера из действительной и мнимой частей
    return tf.complex(real_part, imag_part)

#функция для маскирования последовательностей с клиппингом и восстановленных нейросетью
последовательностей
#reference - исходная последовательность с клиппингом
#restored - восстановленная нейросетью последовательность
#threshold_ratio - относительный порог фиксации клиппинга (по умолчанию - 95% абсолютного
пикового значения)
#возвращаемое значение - маскированная последовательность, в которой в позициях без
клиппинга использованы семплы из reference, где он есть - семплы из restored

```

```
def threshold_mask(reference, restored, threshold_ratio=0.95):
    abs_x = np.abs(reference)
    max_val = np.max(abs_x) #находим абсолютный максимум последовательности с клиппингом
    mask = (abs_x >= (threshold_ratio * max_val)).astype(int) #маскирование по относительному
    #порогу: 1 - клиппинг текущего семпла обнаружен, 0 - в противном случае
    return (1-mask)*reference + restored*mask #маска = 1 - используем семпл из restored, маска = 0 -
    семпл из reference
```

#функция для чтения WAV-файла и возвращения его семплов как NumPy массива нормированных семплов в виде значений типа float32
 #file_path - путь к открываемому файлу
 #возвращаемое значение - массив нормированных (значения на отрезке [-1,1]) семплов файла

```
def read_wav_as_float(file_path):
    with wave.open(file_path, 'rb') as wav_file:
        #получить параметры из заголовка открываемого файла: количество каналов, количество байт
        #на семпл, количество семплов, частота дискретизации
        n_channels = wav_file.getnchannels()
        sample_width = wav_file.getsampwidth()
        n_frames = wav_file.getnframes()
        frame_rate = wav_file.getframerate()
        print(f"Channels: {n_channels}, Sample Width: {sample_width}, Frame Rate: {frame_rate}, Frames:
        {n_frames}")
```

```
        #прочитать данные исходного файла как массив значений байтов
        raw_data = wav_file.readframes(n_frames)
```

#определить тип данных отдельного семпла на основании количества байт на семпл: 2 байта - np.int16, 4 байта - np.int32

```
dtype = {2: np.int16, 4: np.int32}.get(sample_width)
if dtype is None:
    raise ValueError(f"Unsupported sample width: {sample_width}")
```

```
#конвертировать массив сырых байтов в массив значений заданного типа
int_data = np.frombuffer(raw_data, dtype=dtype)
```

#нормировать массив значений семплов, чтобы итоговые значения лежали в диапазоне [-1,1]
 max_val = float(2 ** (8 * sample_width - 1)) #максимальное значение семпла в знаковой PCM-модуляции когда на один семпл приходится sample_width байт
 float_data = int_data.astype(np.float32) / max_val

#если файл содержит несколько каналов - усреднить их значения и преобразовать результат в один канал

```
if n_channels > 1:
    float_data = float_data.reshape(-1, n_channels).mean(axis=1)
```

```
return float_data
```

#функция для записи нормированных значений семплов в 16-битный одноканальный WAV-файл
 #samples - массив нормированных на отрезке [-1,1] семплов звукового файла
 #sample_rate - частота дискретизации звуковой дорожки
 #output_path - путь по которому будет записан звуковой файл
 #возвращаемое значение - отсутствует

```
def write_float_samples_to_wav(samples, sample_rate, output_path):
```

```

#преобразовать массив семплов в массив NumPy
samples = np.array(samples, dtype=np.float32)

#ограничиваем значения в массиве samples отрезком [-1,1] чтобы избежать переполнения
samples = np.clip(samples, -1.0, 1.0)

#конвертировать массив семплов в формат 16-битной PCM
int_samples = (samples * 32768).astype(np.int16)

#записать семплы в 16-битный одноканальный WAV-файл
with wave.open(output_path, 'wb') as wav_file:
    #установить параметры WAV-файла
    wav_file.setnchannels(1) # Один канал
    wav_file.setsampwidth(2) # 16-битная PCM
    wav_file.setframerate(sample_rate) #установить частоту дискретизации равную sample_rate

    #записать семплы в аудио-файл
    wav_file.writeframes(int_samples.tobytes())

#прочитать количество семплов из заголовка WAV-файла с медиаконтейнером RIFF 64
#filename - имя файла для чтения
#возвращаемое значение - количество семплов в WAV-файле
def get_number_of_samples(filename):
    with contextlib.closing(wave_bwf_rf64.open(filename, 'rb')) as wf:
        nframes = wf.getnframes()
    return nframes

#функция чтения непрерывной последовательности семплов из WAV-файла с медиаконтейнером RIFF 64
#filename - путь к открываемому WAV-файлу
#start_index - индекс первого семпла читаемой последовательности (включительно - индекс первого семпла в последовательности = start_index)
#end_index - индекс последнего семпла читаемой последовательности (не включительно, т.е. реальный индекс последнего семпла = end_index-1)
#возвращаемое значение - нормированная на отрезке [-1,1] последовательность семплов файла начиная с start_index (первый семпл) и заканчивая end_index-1 (последний семпл)
def read_samples_segment(filename, start_index, end_index):
    #открыть файл в двоичном режиме для чтения
    with contextlib.closing(wave_bwf_rf64.open(filename, 'rb')) as wf:
        #извлечь из файла количество семплов в нем
        nframes = wf.getnframes()
        #индекс семпла выходит за границы массива семплов файла - выбросить исключение
        if start_index < 0 or end_index > nframes or start_index >= end_index:
            raise ValueError("Invalid start or end index.")

        #установить указатель в позицию начала читаемой последовательности
        wf.setpos(start_index)
        #прочитать нужное количество семплов из файла в массив frames
        frames = wf.readframes(end_index - start_index)

    sample_width = wf.getsampwidth()
    num_channels = wf.getnchannels()

```

```

#сопоставить количеству бит на семпл тип элементов массива dtype
if sample_width == 2:
    dtype = np.int16
elif sample_width == 4:
    dtype = np.int32
else:
    raise ValueError(f"Unsupported sample width: {sample_width}")

#конвертировать массив сырых байтов в массив NumPy
samples = np.frombuffer(frames, dtype=dtype)/(2**((sample_width*8-1))
#если в файле содержался больше чем 1 канал, выделить отдельное измерение для каналов
звука
if num_channels > 1:
    samples = samples.reshape(-1, num_channels)
return samples

#класс генератора обучающей выборки нейросети
#возвращает кортеж (X_seq,y_seq), где X_seq - массив (пачка) последовательностей с клиппингом
(входная), y_seq - массив (пачка) последовательностей без клиппинга (выходная)
class AudioDataGenerator(Sequence):
    #функция инициализации класса генератора
    #self - ссылка на экземпляр объекта класса AudioDataGenerator
    #original_file - путь к файлу с последовательностями без клиппинга (оригинальным данным)
    #clipped_file - путь к файлу с последовательностями с клиппингом (длина в семплах должна
совпадать с длиной файла без клиппинга)
    #SQNC_LENGTH - длина последовательностей возвращаемых генератором
    #batch_size - размер массива(пачки) последовательностей
    #cache_size - размер (количество пачек последовательностей) непрерывного блока данных
(кэша), единоразово загружаемого в оперативную память с диска
    #shuffle - булево значение, определяющее, будет ли производиться перемешивание
возвращаемой обучающей выборки
    #возвращаемое значение - нет
    def __init__(self, original_file, clipped_file, SQNC_LENGTH, batch_size = 32, cache_size = 100,
shuffle=True):
        #запоминание значений инициализации в объекте генератора
        self.SQNC_LENGTH = SQNC_LENGTH
        self.batch_size = batch_size
        self.shuffle = shuffle
        self.ofname = original_file
        self.cfname = clipped_file
        self.cache_size = cache_size
        #получение количества семплов в оригинальном файле
        self.nofsamples = get_number_of_samples(original_file)
        self.step_size = SQNC_LENGTH // 2 #вычисление шага для извлечения последовательностей и
генерации обучающей выборки (используем 50% наложения)
        self.indices = list(range(0, self.nofsamples - SQNC_LENGTH + 1, self.step_size)) #находим массив
индексов начала последовательностей
        #сквозные индексы первой и последней последовательности кэша
        self.startindex = -1
        self.endindex = -1
        #массивы семплов исходного файла и файла с клиппингом
        self.cache_samples_orig = []
        self.cache_samples_clip = []

```

```

        #если shuffle=True, перемешать индексы начала последовательностей в self.indices в рамках
каждого кэша (self.batch_size*self.cache_size последовательностей)
        if self.shuffle:
            for i in range(int(np.ceil(len(self.indices)/(self.batch_size*self.cache_size)))):
                cache = self.indices[self.batch_size*self.cache_size*i:self.batch_size*self.cache_size*(i+1)]
#извлечь self.batch_size*self.cache_size индексов
                np.random.shuffle(cache) #перемешать
                self.indices[self.batch_size*self.cache_size*i:self.batch_size*self.cache_size*(i+1)] = cache
#записать результат в массив self.indices

#функция, возвращающая длину последовательности
#self - ссылка на экземпляр объекта класса AudioDataGenerator
#возвращаемое значение - количество пачек, приходящихся на одну эпоху
def __len__(self):
    return int(np.ceil(len(self.indices) / self.batch_size))

#функция, возвращающая следующую пачку (массив из self.batch_size последовательностей)
для обучения нейросети
#self - ссылка на экземпляр объекта класса AudioDataGenerator
#idx - индекс пачки (число от 0 до len(obj)-1, где obj - объект генератора)
#возвращаемое значение - пачка данных для обучения нейросети
def __getitem__(self, idx):
    #проверить, нужно ли обновлять кэш
    if ((idx//self.cache_size)*self.cache_size*self.batch_size != self.startindex):
        #обновление кэша
        cachestart_in_batch_num = (idx//self.cache_size)*self.cache_size #номер пачки начала кэша
        self.startindex = cachestart_in_batch_num*self.batch_size #индекс первой
последовательности кэша
        #cache size in batches

        cachesz = self.cache_size*self.batch_size #размер кэша в пачках
        self.endindex = self.startindex + cachesz - 1 #находим индекс последней последовательности
кэша
        #idx - номер запрошенной пачки последовательностей
        #self.startindex и self.endindex - индексы первой (включительно) и последней (не
включительно) пачки последовательностей, хранящихся в кэше
        #пример: 3627-ая последовательность -> self.startindex = 3600, self.endindex = 3699 (для
self.cache_size = 100)
        if (self.endindex > len(self.indices)-1):
            self.endindex = len(self.indices)-1
            fsqstartabsolute = self.startindex*self.step_size #абсолютный индекс семпла начала кэша во
всем файле
            lsqstartabsolute = self.endindex*self.step_size #абсолютный индекс семпла начала
последней последовательности кэша во всем файле
            lsqendabsolute = lsqstartabsolute + self.SQNC_LENGTH - 1 #абсолютный индекс семпла конца
последней последовательности кэша во всем файле
            #читаем сразу весь кэш с диска в оперативную память
            self.cache_samples_orig =
read_samples_segment(self.obufname,fsqstartabsolute,lsqendabsolute+1)
            self.cache_samples_clip = read_samples_segment(self.cfname,fsqstartabsolute,lsqendabsolute+1)
#получить глобальные индексы первой и последней последовательностей для текущей пачки
            start_batch = idx * self.batch_size
            end_batch = (idx + 1) * self.batch_size

```

```

#если пачка длиннее всего файла - усекаем индекс конца пачки
if(end_batch>len(self.indices)-1):
    end_batch= len(self.indices)-1
#извлечь из self.indices массив с индексами начала последовательностей пачки
batch_indices = self.indices[start_batch : end_batch]
#X_batch - пачка последовательностей с клиппингом
#y_batch - пачка последовательностей без клиппинга
X_batch = []
y_batch = []
#поэлементно заполняем пачку последовательностями
for start in batch_indices:
    #извлечь две последовательности длины SQNC_LENGTH из кэша
    cachestartabs = self.startindex*self.step_size #сквозной индекс во всем файле с которого
    начинается кэш
    X_seq = self.cache_samples_clip[start - cachestartabs : start - cachestartabs + self.SQNC_LENGTH]
    y_seq = self.cache_samples_orig[start - cachestartabs : start - cachestartabs +
self.SQNC_LENGTH]
    X_batch.append(X_seq)
    y_batch.append(y_seq)
#конвертировать списки последовательностей в массивы NumPy
return np.array(X_batch), np.array(y_batch)

def on_epoch_end(self):
    #если self.shuffle = True, перемешать куски, которыми читается кэш в эпохе, и
    последовательности в каждом из кэшей в конце каждой эпохи
    if self.shuffle:
        batch_indices = list(range(int(np.ceil(len(self.indices))/(self.batch_size*self.cache_size))))
        np.random.shuffle(batch_indices) #перемешать последовательные куски которыми читается
        кэш в каждой эпохе
        for i in batch_indices:
            cache = self.indices[self.batch_size*self.cache_size*i:self.batch_size*self.cache_size*(i+1)]
            np.random.shuffle(cache) #перемешать последовательности в кэше
            self.indices[self.batch_size*self.cache_size*i:self.batch_size*self.cache_size*(i+1)] = cache

#кастомный слой, оборачивающий весь алгоритм обработки входных последовательностей с
помощью спектрограмм
class SpectrogramModelLayer(tf.keras.layers.Layer):
    #функция инициализации слоя восстановления
    #self - ссылка на экземпляр объекта класса SpectrogramModelLayer
    #sq_length - длина входной последовательности, обрабатываемой слоем
    #rnn_layer - тип рекуррентного слоя, используемого нейросетью (SimpleRNN или LSTM)
    #activation - активация, используемая в рекуррентном слое (с SimpleRNN лучше использовать
    relu, с LSTM - tanh)
    #**kwargs - словарь именованных аргументов функции
    #возвращаемое значение - нет
    def __init__(self, sq_length, rnn_layer=tf.keras.layers.SimpleRNN, activation='relu', **kwargs):
        super(SpectrogramModelLayer, self).__init__(**kwargs)
        #запоминание значений инициализации в объекте слоя
        self.sq_length = sq_length
        self.rnn_layer = rnn_layer
        self.activation = activation
        self.frame_step = FSTEP
        self.frame_length = FSTEP * 2

```



```

self.F_const = self.frame_step + 1 #количество отсчетов спектрограммы по частоте (на 1 больше
из-за "нулевой" частоты - постоянной составляющей сигнала)
self.M_const = sq_Lngth // self.frame_step + 1 #количество отсчетов спектрограммы по времени
(поскольку окно преобразования сдвигается с шагом self.frame_step, данное значение равно
sq_Lngth // self.frame_step + 1)

#прототипы используемых рекуррентных слоев, преобразуют спектрограммы сигнала
self.rnn = self.rnn_layer(units=sq_Lngth, activation=self.activation, return_sequences=True)
self.rnn_s = self.rnn_layer(units=sq_Lngth, activation=self.activation, return_sequences=True)
#слои, используемые непосредственно для преобразования спектрограмм сигнала
self.rnn1 = tf.keras.layers.Bidirectional(self.rnn,merge_mode='ave') #двунаправленный слой с
усреднением результатов работы в двух направлениях
self.dropout = tf.keras.layers.Dropout(0.25) #слой регуляризации, выбрасывает часть нейронов
при обучении
self.rnn2 = tf.keras.layers.Bidirectional(self.rnn_s,merge_mode='ave') #двунаправленный слой с
усреднением результатов работы в двух направлениях
self.dense = tf.keras.layers.Dense(units=self.F_const, activation='linear') #полносвязный слой
#слои, используемые для преобразования сигнала во временной области
self.convout = tf.keras.layers.Conv1D(filters=32, kernel_size=3, activation='relu',
padding='same') #сверточный слой с relu-активацией, 32 фильтра
self.rnnout = self.rnn_layer(units=32, activation=self.activation,
return_sequences=True) #рекуррентный слой из 32 нейронов
self.rnnout2 = self.rnn_layer(units=sq_Lngth//2, activation=self.activation,
return_sequences=True) #рекуррентный слой из sq_Lngth//2 нейронов
self.denseout = tf.keras.layers.Dense(units=sq_Lngth//2, activation='linear') #полносвязный слой
из sq_Lngth//2 нейронов
self.convout2 = tf.keras.layers.Conv1D(filters=1, kernel_size=3, activation='linear',
padding='same') #сверточный слой для устранения искажений во временной области
#функция восстановления сигнала с помощью разработанной нейросети
#self - ссылка на экземпляр объекта класса SpectrogramModelLayer
#inputs - восстанавливаемая последовательность во временной области (или пачка из таких
последовательностей, т.е. форма входа (None,sq_Lngth))
#возвращаемое значение - последовательность или пачка последовательностей такой же
формы как inputs
def call(self, inputs):
    #форма входной последовательности (batch, self.sq_Lngth)
    #преобразуем сигнал в спектрограмму с помощью оконного преобразования Фурье
    #self.frame_length = FSTEP * 2, self.frame_step = FSTEP, степень наложения - 50%
    #mag - амплитудная составляющая комплексных отсчетов спектрограммы
    #phase - фазовая составляющая комплексных отсчетов спектрограммы
    #при восстановлении преобразуем только амплитудную составляющую спектрограммы,
    фазовую оставляем неизменной
    mag, phase = tf.keras.ops.stft(inputs,self.frame_length,self.frame_step,self.frame_length)
    batch_size = tf.shape(mag)[0] #извлекаем количество последовательностей в пачке
    #используем последовательные рекуррентные слои для преобразования спектрограммы,
    вход (self.M_const,self.F_const)->(self.M_const,self.sq_Lngth)
    #рекуррентные слои идеально подходят для обработки изменяющихся срезов спектрограммы
    по времени за счет наличия "состояния"
    x = self.rnn1(mag)
    x = self.dropout(x) #отбрасываем часть нейронов при градиентном спуске чтобы избежать
переобучения
    x = self.rnn2(x)

```

```

x = self.dense(x) #возвращаем спектрограмму в исходную форму, устраняя избыточность
(self.M_const,self.sq_lngth)->(self.M_const,self.F_const)
x = mag + x #прибавляем получившийся результат к спектрограмме входной
последовательности
#восстановить сигнал во временной области с помощью обратного оконного преобразования
Фурье, для правильной инверсии используем те же значения параметров что и при прямом
преобразовании
reconstructed = tf.keras.ops.istft([x,phase],self.frame_length,self.frame_step,self.frame_length)

#преобразование последовательности во временной области
reconstructed = tf.reshape(reconstructed, [batch_size, self.sq_lngth, 1]) #добавляем
дополнительное измерение сигнала
#сверточный слой, преобразует форму сигнала (batch_size, self.sq_lngth, 1)->(batch,
self.sq_lngth, 32), т.е. добавляется избыточность
rec_proc1 = self.convout(reconstructed)
#преобразуем входные данные через рекуррентные слои
rec_proc = self.rnnout(rec_proc1) #(batch, self.sq_lngth, 32)->(batch, self.sq_lngth, 32), степень
избыточности та же
rec_proc = self.dropout(rec_proc) #отбрасываем часть нейронов при градиентном спуске
чтобы избежать переобучения
rec_proc = self.rnnout2(rec_proc) #(batch, self.sq_lngth, 32)->(batch, sq_lngth, self.sq_lngth//2),
степень избыточности увеличена
rec_proc = self.denseout(rec_proc) #полносвязный слой, (batch, self.sq_lngth, self.sq_lngth//2)-
>(batch, self.sq_lngth, self.sq_lngth//2)
rec_proc = self.convout2(rec_proc) #убираем избыточность через свертку->(batch,
self.sq_lngth, self.sq_lngth//2)->(batch, sq_lngth, 1)
rec_proc = tf.squeeze(rec_proc, axis=-1) #убираем дополнительное измерение (batch, sq_lngth,
1)->(batch, sq_lngth)

#обратная связь, складываем получившиеся восстановленные пики во временной области с
входным сигналом
return inputs + rec_proc

#функция, возвращающая словарь с параметрами слоя: нужна для правильного сохранения
модели в файл и загрузки модели из файла
#self - ссылка на экземпляр объекта класса SpectrogramModelLayer
#возвращаемое значение - словарь, содержащий пары (ключ,значение) вида
(название_параметра, значение)
def get_config(self):
    config = super(SpectrogramModelLayer, self).get_config()
    #возвращать дополнительно к основным параметрам слоя: длину последовательности, тип
рекуррентного слоя, активацию
    config.update({
        "sq_lngth": self.sq_lngth,
        "rnn_layer": self.rnn_layer.__name__,
        "activation": self.activation
    })
    return config

#функция, создающая слой на основании его конфигурации
#cls - класс, объект которого инстанцируется (в данном случае, SpectrogramModelLayer)
#config - словарь с параметрами слоя, содержащий пары (ключ,значение) вида
(название_параметра, значение)
#возвращаемое значение - объект класса cls, инициализированный параметрами из словаря
config

```

```

@classmethod
def from_config(cls, config):
    #преобразуем строковое название слоя в класс слоя
    rnn_layer_name = config.pop("rnn_layer")
    if rnn_layer_name == "LSTM":
        config["rnn_layer"] = tf.keras.layers.LSTM
    elif rnn_layer_name == "SimpleRNN":
        config["rnn_layer"] = tf.keras.layers.SimpleRNN
    else:
        raise ValueError(f"Unsupported rnn_layer: {rnn_layer_name}") #если такой строке никакой
        слой не соответствует, возвращаем ошибку
    #инстанцируем объект данного класса
    return cls(**config)

#функция, реализующая модель на основании заданных параметров
#sq_length - длина последовательности
#layer - рекуррентный слой, используемый для инициализации модели
#activation - функция активации, применяемая сразу после рекуррентных слоев модели
#возвращаемое значение - объект модели для обучения с заданными параметрами
def build_rnn_spectrogram_model(sq_length, layer, activation):
    model = tf.keras.Sequential([
        tf.keras.layers.InputLayer(shape=(sq_length,)),
        SpectrogramModelLayer(sq_length=sq_length, rnn_layer=layer, activation=activation)
    ])
    model.compile(
        optimizer=tf.keras.optimizers.AdamW(learning_rate=0.0001, weight_decay=0.01),
        loss='mse',
        metrics=['mse']
    )
    return model

```

Файл script.py

```

from base import * #базовый набор методов и объявлений для обучения и применения нейросети
if __name__ == '__main__':
    #недостаточно аргументов для запуска скрипта - сообщение об ошибке
    if(len(sys.argv)<5):
        print('No enough arguments passed!')
    else:
        #фиксируем генераторы случайных чисел чтобы обеспечить воспроизводимость результатов
        обучения
        os.environ['PYTHONHASHSEED']=str(42)
        seed(42)
        np.random.seed(42)
        tf.random.set_seed(42)
        wav_file_path = sys.argv[1] #путь к исходной аудио дорожке обучающей выборки (без
        клиппинга, обязательно использовать медиаконтейнер RIFF 64)
        wav_file_path1 = sys.argv[2] #путь к аудио дорожке с клиппингом обучающей выборки
        (обязательно использовать медиаконтейнер RIFF 64)
        modes = [(tf.keras.layers.SimpleRNN, 'relu'), (tf.keras.layers.LSTM, 'tanh')] #режимы (архитектуры
        рекуррентного слоя) используемые для обучения нейросети
        index = int(sys.argv[4])
        model = build_rnn_spectrogram_model(SQNC_LENGTH, modes[index][0], modes[index][1])

```

```

model.summary()
early_stopping = tf.keras.callbacks.EarlyStopping(monitor='loss', patience=20,
restore_best_weights=True) #если обучение уже не дает прогресса - завершаем
batch_size = int(sys.argv[3]) #размер пачки - целое число
train_gen = AudioDataGenerator(wav_file_path, wav_file_path1, SQNC_LENGTH,
batch_size=batch_size, cache_size=20, shuffle=True) #генерируем обучающую выборку используя
кэш
checkpointer =
ModelCheckpoint('model_checkpoint.keras',monitor='mse',verbose=1,save_best_only=True,save_weights_only=False,mode='min',save_freq='epoch') #сохранение модели при уменьшении MSE,
проверять уменьшение MSE каждую эпоху
model.fit(train_gen,
          verbose=1,
          epochs=1000,
          callbacks=[early_stopping,checkpointer]) #запустить обучение модели

```

Файл inference_batch_prediction.py

```

from base import *
import tensorflow as tf
from tensorflow.keras.mixed_precision import Policy
from tensorflow.keras.utils import custom_object_scope
if __name__ == '__main__':
    #передано меньше 5 аргументов - выводим сообщение об ошибке
    if(len(sys.argv)<6):
        print('No enough arguments passed!')
    else:
        model_for_load_path = sys.argv[1] #путь к загружаемому файлу обученной модели
        file_for_restoration_path = sys.argv[2] #путь к восстанавливаемому WAV-файлу
        output_path = sys.argv[3] #путь сохранения восстановленного WAV-файла
        alpha = float(sys.argv[4]) #относительный порог фиксации клиппинга - действительное число на
интервале (0,1)
        T = float(sys.argv[5]) #порог маскирования спектрограммы при очищении файла от остаточных
шумов
        #класс, используемый для корректной загрузки входного слоя (отбрасывание измерения,
отвечающего за размер пачки)
        class CustomInputLayer(tf.keras.layers.InputLayer):
            @classmethod
            def from_config(cls, config):
                #если в InputLayer в сохраненном файле есть измерение с размером пачки, удалить его
                if 'batch_shape' in config:
                    config.pop('batch_shape')
                return super(CustomInputLayer, cls).from_config(config)

        #загрузить частоту дискретизации из заголовка восстанавливаемого файла
        with wave.open(file_for_restoration_path, 'rb') as wav_file:
            fs = wav_file.getframerate()

        #загрузка модели из файла с обученной моделью с использованием объявленных слоев и
функций из base
        model = tf.keras.models.load_model(
            model_for_load_path,
            custom_objects={

```

```

        'SpectrogramModelLayer': SpectrogramModelLayer,
        'InputLayer': CustomInputLayer,
        'DTypePolicy': Policy
    }
)

samples_input_file = read_wav_as_float(file_for_restoration_path) #чтение семплов из
восстанавливаемого звукового файла
restored_samples_overlap = []
overlap_input_sequences = []
step_size = SQNC_LENGTH // 2 #наложение между соседними последовательностями - 50%
j = 0
#нахождение максимума и минимума значений семплов восстанавливаемого файла
maxv = np.max(np.array(samples_input_file))
minv = np.min(np.array(samples_input_file))
#разбиение массива семплов восстанавливаемого файла на последовательности из
SQNC_LENGTH семплов с наложением в 50%
while j < len(samples_input_file):
    #если индекс конца извлекаемой последовательности в пределах массива семплов -
    добавляем
    if(j+SQNC_LENGTH < len(samples_input_file)):
        overlap_input_sequences.append(samples_input_file[j:j+SQNC_LENGTH])
    else: #иначе - завершаем цикл
        break
    j += step_size
overlap_input_sequences = np.array(overlap_input_sequences)
nn_restored = model.predict_on_batch(overlap_input_sequences) #восстановление массива
последовательностей с помощью загруженной нейросети
#если T = 0, маскирование ничего не изменяет
if(T>0.0001):
    nn_restored = nn_restored.flatten() #приведение к одномерному массиву
    nn_restored = tf.signal.stft(nn_restored,256,128,256) #дискретное оконное преобразование
    Фурье, размер преобразования - 256, шаг - 128, окно Хэмминга (наложение 50% + окно Ханна -
    NOLA, обеспечивается инвертируемость)
    nn_restored = mask_stft_amplitude(nn_restored,T) #маскирование спектрограммы для
    удаления остаточных шумов, оставляемых нейросетью
    nn_restored =
    tf.signal.inverse_stft(nn_restored,256,128,256>window_fn=tf.signal.inverse_stft_window_fn(128))
#инвертирование дискретного оконного преобразования Фурье
    nn_restored = np.array(nn_restored)
    nn_restored =
nn_restored.reshape(nn_restored.shape[0]//SQNC_LENGTH,SQNC_LENGTH) #приведение данных
обратно к двумерному массиву
i = 0
#получение окончательного результата восстановления - где нет клиппинга берем семплы из
восстанавливаемого файла, где есть - результаты работы нейросети
for sqnc in overlap_input_sequences:
    #если в последовательности есть клиппинг - вставляем маскированные результаты
    восстановления нейросети
    if(max(sqnc)>(maxv*alpha) or min(sqnc)<(minv*alpha)):
        #извлекаем семплы из середины последовательностей исходных и восстановленных
        семплов
        restored = nn_restored[i][SQNC_LENGTH//4:(SQNC_LENGTH*3)//4]

```

```

reference = np.array(sqnc[SQNC_LENGTH//4:(SQNC_LENGTH*3)//4])
#объединяем результаты в итоговую последовательность, используя относительный
порог клиппинга alpha
restored_samples_overlap.append(threshold_mask(reference, restored, alpha))
#в противном случае - вставляем последовательность из исходного файла
else:
    restored_samples_overlap.append(np.array(sqnc[SQNC_LENGTH//4:(SQNC_LENGTH*3)//4]))
    i += 1
restored_samples_overlap = np.array(restored_samples_overlap).flatten()
#вычисляем количество семплов в конце, не покрытых последней последовательностью
add = len(samples_input_file) - SQNC_LENGTH//4 - restored_samples_overlap.shape[0]
#дополняем массив SQNC_LENGTH//4 семплами из исходного файла в начале (были
пропущены по причине того, что мы вставляем семплы из середины каждой последовательности)
restored_samples_overlap =
np.append(np.array(samples_input_file[0:SQNC_LENGTH//4]),restored_samples_overlap)
#дополнение массива add семплами на конце
if(add>0):
    restored_samples_overlap = np.append(restored_samples_overlap,np.array(samples_input_file[-
add:]))
#запись результатов восстановления по пути output_path
write_float_samples_to_wav(restored_samples_overlap, fs, output_path)
print(f"WAV file written to {output_path}")

```

Файл requirements.txt

```

absi-py==2.1.0
astunparse==1.6.3
certifi==2025.1.31
charset-normalizer==3.4.1
flatbuffers==25.2.10
gast==0.6.0
google-pasta==0.2.0
grpcio==1.71.0
h5py==3.13.0
idna==3.10
keras==3.9.0
libclang==18.1.1
llvmlite==0.44.0
Markdown==3.7
markdown-it-py==3.0.0
MarkupSafe==3.0.2
mdurl==0.1.2
ml_dtypes==0.5.1
namex==0.0.8
numba==0.61.0
numpy==2.1.3
nvidia-cublas-cu12==12.5.3.2
nvidia-cuda-cupti-cu12==12.5.82
nvidia-cuda-nvcc-cu12==12.5.82
nvidia-cuda-nvrtc-cu12==12.5.82
nvidia-cuda-runtime-cu12==12.5.82
nvidia-cudnn-cu12==9.3.0.75
nvidia-cufft-cu12==11.2.3.61

```

nvidia-curand-cu12==10.3.6.82
nvidia-cusolver-cu12==11.6.3.83
nvidia-cuspars-cu12==12.5.1.3
nvidia-nccl-cu12==2.23.4
nvidia-nvjitlink-cu12==12.5.82
opt_einsum==3.4.0
optree==0.14.1
packaging==24.2
protobuf==5.29.3
Pygments==2.19.1
requests==2.32.3
rich==13.9.4
six==1.17.0
tensorboard==2.19.0
tensorboard-data-server==0.7.2
tensorflow==2.19.0
tensorflow-io-gcs-filesystem==0.37.1
termcolor==2.5.0
typing_extensions==4.12.2
urllib3==2.3.0
wave-bwf-rf64==2.0.1
Werkzeug==3.1.3
wrapt==1.17.2